

Spring Data JPA

Spring Data JPA provides repository support for the Jakarta Persistence API (JPA). It eases development of applications with a consistent programming model that need to access JPA data sources.

JPA JPA and JPA Repositories

Envers Support for Envers Revision Repositories

Wiki What's New, Upgrade Notes, Supported Versions, additional cross-version information.

Oliver Gierke, Thomas Darimont, Christoph Strobl, Mark Paluch, Jay Bryant, Greg Turnquist

© 2008-2026 VMware, Inc.

Copies of this document may be made for your own use and for distribution to others, provided that you do not charge any fee for such copies and further provided that each copy contains this Copyright Notice, whether distributed in print or electronically.

Upgrading Spring Data

Instructions for how to upgrade from earlier versions of Spring Data are provided on the project [wiki](#). Follow the links in the [release notes section](#) to find the version that you want to upgrade to.

Upgrading instructions are always the first item in the release notes. If you are more than one release behind, please make sure that you also review the release notes of the versions that you jumped.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

JPA

This chapter points out the specialties for repository support for JPA. This builds on the core repository support explained in [Working with Spring Data Repositories](#). Make sure you have a sound understanding of the basic concepts explained there.

Section Summary

- [Getting Started](#)
- [Core concepts](#)
- [Defining Repository Interfaces](#)
- [Configuration](#)
- [Persisting Entities](#)
- [Defining Query Methods](#)
- [JPA Query Methods](#)
- [Value Expressions Fundamentals](#)
- [Projections](#)
- [Stored Procedures](#)
- [Specifications](#)
- [Query by Example](#)
- [Vector Search](#)
- [Transactionality](#)
- [Locking](#)
- [Auditing](#)
- [Merging persistence units](#)
- [CDI Integration](#)
- [Custom Repository Implementations](#)
- [Publishing Events from Aggregate Roots](#)
- [Null Handling of Repository Methods](#)
- [Spring Data Extensions](#)
- [Repository query keywords](#)
- [Repository query return types](#)
- [Ahead of Time Optimizations](#)
- [Frequently Asked Questions](#)

- [Glossary](#)



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Getting Started

Getting Started

Examples Repository

Hello World

An easy way to bootstrap setting up a working environment is to create a Spring-based project via start.spring.io or create a Spring project in [Spring Tools](#).

Examples Repository

The GitHub [spring-data-examples repository](#) hosts several examples that you can download and play around with to get a feel for how the library works.

Hello World

Let's start with a simple entity and its corresponding repository:

```
@Entity
class Person {

    @Id @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    private String name;

    // getters and setters omitted for brevity
}

interface PersonRepository extends Repository<Person, Long> {

    Person save(Person person);

    Optional<Person> findById(long id);
}
```

JAVA

Create the main application to run, as the following example shows:

JAVA

```
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

    @Bean
    CommandLineRunner runner(PersonRepository repository) {
        return args -> {

            Person person = new Person();
            person.setName("John");

            repository.save(person);
            Person saved =
repository.findById(person.getId()).orElseThrow(NoSuchElementException::new);
        };
    }
}
```

Even in this simple example, there are a few notable things to point out:

- Repository instances are automatically implemented. When used as parameters of `@Bean` methods, these will be autowired without further need for annotations.
- The basic repository extends `Repository`. We suggest to consider how much API surface you want to expose towards your application. More complex repository interfaces are `ListCrudRepository` or `JpaRepository`.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered

trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Core concepts

The central interface in the Spring Data repository abstraction is `Repository`. It takes the domain class to manage as well as the identifier type of the domain class as type arguments. This interface acts primarily as a marker interface to capture the types to work with and to help you to discover interfaces that extend this one.

TIP

Spring Data considers domain types to be entities, more specifically aggregates. So you will see the term "entity" used throughout the documentation that can be interchanged with the term "domain type" or "aggregate".

As you might have noticed in the introduction it already hinted towards domain-driven concepts. We consider domain objects in the sense of DDD. Domain objects have identifiers (otherwise these would be identity-less value objects), and we somehow need to refer to identifiers when working with certain patterns to access data. Referring to identifiers will become more meaningful as we talk about repositories and query methods.

The [CrudRepository](#) and [ListCrudRepository](#) interfaces provide sophisticated CRUD functionality for the entity class that is being managed.

CrudRepository Interface

```
public interface CrudRepository<T, ID> extends Repository<T, ID> {  
  
    <S extends T> S save(S entity);           1  
  
    Optional<T> findById(ID primaryKey);      2  
  
    Iterable<T> findAll();                     3  
  
    long count();                             4  
  
    void delete(T entity);                    5  
  
    boolean existsById(ID primaryKey);        6  
  
    // ... more functionality omitted.  
}
```

JAVA

- 1 Saves the given entity.
- 2 Returns the entity identified by the given ID.
- 3 Returns all entities.
- 4 Returns the number of entities.
- 5 Deletes the given entity.
- 6 Indicates whether an entity with the given ID exists.

The methods declared in this interface are commonly referred to as CRUD methods.

`ListCrudRepository` offers equivalent methods, but they return `List` where the `CrudRepository` methods return an `Iterable`.

| IMPORTANT

The repository interface implies a few reserved methods like `findById(ID identifier)` that target the domain type identifier property regardless of its property name. Read more about this in [“Defining Query Methods”](#).

You can annotate your query method with `@Query` to provide a custom query if a property named `Id` doesn't refer to the identifier. Following that path can easily lead to confusion and is discouraged as you will quickly hit type limits if the `ID` type and the type of your `Id` property deviate.

| NOTE

We also provide persistence technology-specific abstractions, such as `JpaRepository` or `MongoRepository`. Those interfaces extend `CrudRepository` and expose the capabilities of the underlying persistence technology in addition to the rather generic persistence technology-agnostic interfaces such as `CrudRepository`.

Additional to the `CrudRepository`, there are [PagingAndSortingRepository](#) and [ListPagingAndSortingRepository](#) which add additional methods to ease paginated access to entities:

PagingAndSortingRepository interface

```
public interface PagingAndSortingRepository<T, ID> {  
  
    Iterable<T> findAll(Sort sort);  
  
    Page<T> findAll(Pageable pageable);  
}
```

JAVA

| NOTE

Extension interfaces are subject to be supported by the actual store module. While this documentation explains the general scheme, make sure that your store module supports the interfaces that you want to use.

To access the second page of `User` by a page size of 20, you could do something like the following:

JAVA

```
PagingAndSortingRepository<User, Long> repository = // ... get access to a bean
Page<User> users = repository.findAll(PageRequest.of(1, 20));
```

`ListPagingAndSortingRepository` offers equivalent methods, but returns a `List` where the `PagingAndSortingRepository` methods return an `Iterable`.

In addition to pagination, scrolling provides a more fine-grained access to iterate through chunks of larger result sets.

In addition to query methods, query derivation for both count and delete queries is available. The following list shows the interface definition for a derived count query:

Derived Count Query

JAVA

```
interface UserRepository extends CrudRepository<User, Long> {

    long countByLastname(String lastname);
}
```

The following listing shows the interface definition for a derived delete query:

Derived Delete Query

JAVA

```
interface UserRepository extends CrudRepository<User, Long> {

    long deleteByLastname(String lastname);

    List<User> removeByLastname(String lastname);
}
```



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and

copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Defining Repository Interfaces

Defining Repository Interfaces

Fine-tuning Repository Definition

Using Repositories with Multiple Spring Data Modules

To define a repository interface, you first need to define a domain class-specific repository interface. The interface must extend `Repository` and be typed to the domain class and an ID type. If you want to expose CRUD methods for that domain type, you may extend `CrudRepository`, or one of its variants instead of `Repository`.

Fine-tuning Repository Definition

There are a few variants how you can get started with your repository interface.

The typical approach is to extend `CrudRepository`, which gives you methods for CRUD functionality. CRUD stands for Create, Read, Update, Delete. With version 3.0 we also introduced `ListCrudRepository` which is very similar to the `CrudRepository` but for those methods that return multiple entities it returns a `List` instead of an `Iterable` which you might find easier to use.

If you are using a reactive store you might choose `ReactiveCrudRepository`, or `RxJava3CrudRepository` depending on which reactive framework you are using.

If you are using Kotlin you might pick `CoroutineCrudRepository` which utilizes Kotlin's coroutines.

Additionally you can extend `PagingAndSortingRepository`, `ReactiveSortingRepository`, `RxJava3SortingRepository`, or `CoroutineSortingRepository` if you need methods that allow to specify a `Sort` abstraction or in the first case a `Pageable` abstraction. Note that the various sorting repositories no longer extended their respective CRUD repository as they did in Spring Data Versions pre 3.0. Therefore, you need to extend both interfaces if you want functionality of both.

If you do not want to extend Spring Data interfaces, you can also annotate your repository interface with `@RepositoryDefinition`. Extending one of the CRUD repository interfaces exposes a complete set of methods to manipulate your entities. If you prefer to be selective about the methods being exposed,

copy the methods you want to expose from the CRUD repository into your domain repository. When doing so, you may change the return type of methods. Spring Data will honor the return type if possible. For example, for methods returning multiple entities you may choose `Iterable<T>`, `List<T>`, `Collection<T>` or a VAVR list.

If many repositories in your application should have the same set of methods you can define your own base interface to inherit from. Such an interface must be annotated with `@NoRepositoryBean`. This prevents Spring Data to try to create an instance of it directly and failing because it can't determine the entity for that repository, since it still contains a generic type variable.

The following example shows how to selectively expose CRUD methods (`findById` and `save`, in this case):

Selectively exposing CRUD methods

JAVA

```
@NoRepositoryBean
interface MyBaseRepository<T, ID> extends Repository<T, ID> {

    Optional<T> findById(ID id);

    <S extends T> S save(S entity);
}

interface UserRepository extends MyBaseRepository<User, Long> {
    User findByEmailAddress(EmailAddress emailAddress);
}
```

In the prior example, you defined a common base interface for all your domain repositories and exposed `findById(...)` as well as `save(...)`. These methods are routed into the base repository implementation of the store of your choice provided by Spring Data (for example, if you use JPA, the implementation is `SimpleJpaRepository`), because they match the method signatures in `CrudRepository`. So the `UserRepository` can now save users, find individual users by ID, and trigger a query to find `Users` by email address.

NOTE

The intermediate repository interface is annotated with `@NoRepositoryBean`. Make sure you add that annotation to all repository interfaces for which Spring Data should not create instances at runtime.

Using Repositories with Multiple Spring Data Modules

Using a unique Spring Data module in your application makes things simple, because all repository interfaces in the defined scope are bound to the Spring Data module. Sometimes, applications require us-

ing more than one Spring Data module. In such cases, a repository definition must distinguish between persistence technologies. When it detects multiple repository factories on the class path, Spring Data enters strict repository configuration mode. Strict configuration uses details on the repository or the domain class to decide about Spring Data module binding for a repository definition:

1. If the repository definition extends the module-specific repository, it is a valid candidate for the particular Spring Data module.
2. If the domain class is annotated with the module-specific type annotation, it is a valid candidate for the particular Spring Data module. Spring Data modules accept either third-party annotations (such as JPA's `@Entity`) or provide their own annotations (such as `@Document` for Spring Data MongoDB and Spring Data Elasticsearch).

The following example shows a repository that uses module-specific interfaces (JPA in this case):

Example 1. Repository definitions using module-specific interfaces

```
interface MyRepository extends JpaRepository<User, Long> { }

@NoRepositoryBean
interface MyBaseRepository<T, ID> extends JpaRepository<T, ID> { ... }

interface UserRepository extends MyBaseRepository<User, Long> { ... }
```

JAVA

`MyRepository` and `UserRepository` extend `JpaRepository` in their type hierarchy. They are valid candidates for the Spring Data JPA module.

The following example shows a repository that uses generic interfaces:

Example 2. Repository definitions using generic interfaces

```
interface AmbiguousRepository extends Repository<User, Long> { ... }

@NoRepositoryBean
interface MyBaseRepository<T, ID> extends CrudRepository<T, ID> { ... }

interface AmbiguousUserRepository extends MyBaseRepository<User, Long> { ... }
```

JAVA

`AmbiguousRepository` and `AmbiguousUserRepository` extend only `Repository` and `CrudRepository` in their type hierarchy. While this is fine when using a unique Spring Data module, multiple modules cannot distinguish to which particular Spring Data these repositories should be bound.

The following example shows a repository that uses domain classes with annotations:

Example 3. Repository definitions using domain classes with annotations

```
interface PersonRepository extends Repository<Person, Long> { ... }

@Entity
class Person { ... }

interface UserRepository extends Repository<User, Long> { ... }

@Document
class User { ... }
```

JAVA

`PersonRepository` references `Person`, which is annotated with the JPA `@Entity` annotation, so this repository clearly belongs to Spring Data JPA. `UserRepository` references `User`, which is annotated with Spring Data MongoDB's `@Document` annotation.

The following bad example shows a repository that uses domain classes with mixed annotations:

Example 4. Repository definitions using domain classes with mixed annotations

```
interface JpaPersonRepository extends Repository<Person, Long> { ... }

interface MongoDBPersonRepository extends Repository<Person, Long> { ... }

@Entity
@Document
class Person { ... }
```

JAVA

This example shows a domain class using both JPA and Spring Data MongoDB annotations. It defines two repositories, `JpaPersonRepository` and `MongoDBPersonRepository`. One is intended for JPA and the other for MongoDB usage. Spring Data is no longer able to tell the repositories apart, which leads to undefined behavior.

Repository type details and distinguishing domain class annotations are used for strict repository configuration to identify repository candidates for a particular Spring Data module. Using multiple persistence technology-specific annotations on the same domain type is possible and enables reuse of domain types across multiple persistence technologies. However, Spring Data can then no longer determine a unique module with which to bind the repository.

The last way to distinguish repositories is by scoping repository base packages. Base packages define the starting points for scanning for repository interface definitions, which implies having repository definitions located in the appropriate packages. By default, annotation-driven configuration uses the package of the configuration class. The base package in XML-based configuration is mandatory.

The following example shows annotation-driven configuration of base packages:

Annotation-driven configuration of base packages

JAVA

```
@EnableJpaRepositories(basePackages = "com.acme.repositories.jpa")
@EnableMongoRepositories(basePackages = "com.acme.repositories.mongo")
class Configuration { ... }
```



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Configuration

Configuration

Annotation-based Configuration

Spring Namespace

Custom Namespace Attributes

Bootstrap Mode

Recommendations

This section describes configuring Spring Data JPA through either:

- “Annotation-based Configuration” (Java configuration)
- “Spring Namespace” (XML configuration)

Annotation-based Configuration

The Spring Data JPA repositories support can be activated through both JavaConfig as well as a custom XML namespace, as shown in the following example:

Example 1. Spring Data JPA repositories using JavaConfig

```
@Configuration
@EnableJpaRepositories
@EnableTransactionManagement
class ApplicationConfig {

    @Bean
    public DataSource dataSource() {

        EmbeddedDatabaseBuilder builder = new EmbeddedDatabaseBuilder();
        return builder.setType(EmbeddedDatabaseType.HSQL).build();
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory() {

        HibernateJpaVendorAdapter vendorAdapter = new HibernateJpaVendorAdapter();
```

JAVA

```

        vendorAdapter.setGenerateDdl(true);

        LocalContainerEntityManagerFactoryBean factory = new
LocalContainerEntityManagerFactoryBean();
        factory.setJpaVendorAdapter(vendorAdapter);
        factory.setPackagesToScan("com.acme.domain");
        factory.setDataSource(dataSource());
        return factory;
    }

    @Bean
    public PlatformTransactionManager transactionManager(EntityManagerFactory
entityManagerFactory) {

        JpaTransactionManager txManager = new JpaTransactionManager();
        txManager.setEntityManagerFactory(entityManagerFactory);
        return txManager;
    }
}

```

NOTE

You must create `LocalContainerEntityManagerFactoryBean` and not `EntityManagerFactory` directly, since the former also participates in exception translation mechanisms in addition to creating `EntityManagerFactory`.

The preceding configuration class sets up an embedded HSQL database by using the `EmbeddedDatabaseBuilder` API of `spring-jdbc`. Spring Data then sets up an `EntityManagerFactory` and uses Hibernate as the sample persistence provider. The last infrastructure component declared here is the `JpaTransactionManager`. Finally, the example activates Spring Data JPA repositories by using the `@EnableJpaRepositories` annotation, which essentially carries the same attributes as the XML namespace. If no base package is configured, it uses the one in which the configuration class resides.

Spring Namespace

The JPA module of Spring Data contains a custom namespace that allows defining repository beans. It also contains certain features and element attributes that are special to JPA. Generally, the JPA repositories can be set up by using the `repositories` element, as shown in the following example:

Example 2. Setting up JPA repositories by using the namespace

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

```

XML

```
xmlns:jpa="http://www.springframework.org/schema/data/jpa"
xsi:schemaLocation="http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/data/jpa
    https://www.springframework.org/schema/data/jpa/spring-jpa.xsd">

<jpa:repositories base-package="com.acme.repositories" />

</beans>
```

TIP

Which is better, JavaConfig or XML? XML is how Spring was configured long ago. In today's era of fast-growing Java, record types, annotations, and more, new projects typically use as much pure Java as possible. While there is no immediate plan to remove XML support, some of the newest features MAY not be available through XML.

Using the `repositories` element it activates persistence exception translation for all beans annotated with `@Repository`, to let exceptions being thrown by the JPA persistence providers be converted into Spring's `DataAccessException` hierarchy.

Custom Namespace Attributes

Beyond the default attributes of the `repositories` element, the JPA namespace offers additional attributes to let you gain more detailed control over the setup of the repositories:

Table 1. Custom JPA-specific attributes of the `repositories` element

entity-manager-factory-ref	Explicitly wire the <code>EntityManagerFactory</code> to be used with the repositories being detected by the <code>repositories</code> element. Usually used if multiple <code>EntityManagerFactory</code> beans are used within the application. If not configured, Spring Data automatically looks up the <code>EntityManagerFactory</code> bean with the name <code>entityManagerFactory</code> in the <code>ApplicationContext</code> .
transaction-manager-ref	Explicitly wire the <code>PlatformTransactionManager</code> to be used with the repositories being detected by the <code>repositories</code> element. Usually only necessary if multiple transaction managers or <code>EntityManagerFactory</code> beans have been configured. Default to a single defined <code>PlatformTransactionManager</code> inside the current <code>ApplicationContext</code> .

NOTE

Spring Data JPA requires a `PlatformTransactionManager` bean named `transactionManager` to be present if no explicit `transaction-manager-ref` is defined.

Bootstrap Mode

By default, Spring Data JPA repositories are default Spring beans. They are singleton scoped and eagerly initialized. During startup, they already interact with the JPA `EntityManager` for verification and meta-data analysis purposes. Spring Framework supports the initialization of the JPA `EntityManagerFactory` in a background thread because that process usually takes up a significant amount of startup time in a Spring application. To make use of that background initialization effectively, we need to make sure that JPA repositories are initialized as late as possible.

As of Spring Data JPA 2.1 you can now configure a `BootstrapMode` (either via the `@EnableJpaRepositories` annotation or the XML namespace) that takes the following values:

- **DEFAULT** (default) — Repositories are instantiated eagerly unless explicitly annotated with `@Lazy`. The lazification only has effect if no client bean needs an instance of the repository as that will require the initialization of the repository bean.
- **LAZY** — Implicitly declares all repository beans lazy and also causes lazy initialization proxies to be created to be injected into client beans. That means, that repositories will not get instantiated if the client bean is simply storing the instance in a field and not making use of the repository during initialization. Repository instances will be initialized and verified upon first interaction with the repository.
- **DEFERRED** — Fundamentally the same mode of operation as **LAZY**, but triggering repository initialization in response to an `ContextRefreshedEvent` so that repositories are verified before the application has completely started.

Recommendations

If you're not using asynchronous JPA bootstrap stick with the default bootstrap mode.

In case you bootstrap JPA asynchronously, **DEFERRED** is a reasonable default as it will make sure the Spring Data JPA bootstrap only waits for the `EntityManagerFactory` setup if that itself takes longer than initializing all other application components. Still, it makes sure that repositories are properly initialized and validated before the application signals it's up.

LAZY is a decent choice for testing scenarios and local development. Once you are pretty sure that repositories can properly bootstrap, or in cases where you are testing other parts of the application, running verification for all repositories might unnecessarily increase the startup time. The same applies to local development in which you only access parts of the application that might need to have a single repository initialized.

Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Persisting Entities

Persisting Entities

Saving Entities

Entity State-detection Strategies

This section describes how to persist (save) entities with Spring Data JPA.

Saving Entities

Saving an entity can be performed with the `CrudRepository.save(...)` method. It persists or merges the given entity by using the underlying JPA `EntityManager`. If the entity has not yet been persisted, Spring Data JPA saves the entity with a call to the `entityManager.persist(...)` method. Otherwise, it calls the `entityManager.merge(...)` method.

Entity State-detection Strategies

Spring Data JPA offers the following strategies to detect whether an entity is new or not:

1. **Version-Property and Id-Property inspection (default):** By default Spring Data JPA inspects first if there is a Version-property of non-primitive type. If there is, the entity is considered new if the value of that property is `null`. Without such a Version-property Spring Data JPA inspects the identifier property of the given entity. If the identifier property is `null`, then the entity is assumed to be new. Otherwise, it is assumed to be not new. In contrast to other Spring Data modules, JPA considers `0` (zero) as the first inserted version of an entity and therefore, a primitive version property cannot be used to determine whether an entity is new or not.
2. **Implementing `Persistable`:** If an entity implements `Persistable`, Spring Data JPA delegates the new detection to the `isNew(...)` method of the entity. See the [JavaDoc](#) for details.
3. **Implementing `EntityInformation`:** You can customize the `EntityInformation` abstraction used in the `SimpleJpaRepository` implementation by creating a subclass of `JpaRepositoryFactory` and overriding the `getEntityInformation(...)` method accordingly. You then have to register the

custom implementation of `JpaRepositoryFactory` as a Spring bean. Note that this should be rarely necessary. See the [JavaDoc](#) for details.

Option 1 is not an option for entities that use manually assigned identifiers and no version attribute as with those the identifier will always be non- null . A common pattern in that scenario is to use a common base class with a transient flag defaulting to indicate a new instance and using JPA lifecycle callbacks to flip that flag on persistence operations:

Example 1. A base class for entities with manually assigned identifiers

```

@MappedSuperclass
public abstract class AbstractEntity<ID> implements Persistable<ID> {

    @Transient
    private boolean isNew = true; 1

    @Override
    public boolean isNew() {
        return isNew; 2
    }

    @PostPersist 3
    @PostLoad
    void markNotNew() {
        this.isNew = false;
    }

    // More code...
}

```

- 1 Declare a flag to hold the new state. Transient so that it's not persisted to the database.
- 2 Return the flag in the implementation of `Persistable.isNew()` so that Spring Data repositories know whether to call `EntityManager.persist()` or `...merge()`.
- 3 Declare a method using JPA entity callbacks so that the flag is switched to indicate an existing entity after a repository call to `save(...)` or an instance creation by the persistence provider.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Defining Query Methods

Defining Query Methods

Query Lookup Strategies

Query Creation

Reserved Method Names

Property Expressions

Repository Methods Returning Collections or Iterables

Using Streamable as Query Method Return Type

Returning Custom Streamable Wrapper Types

Support for Vavr Collections

Streaming Query Results

Asynchronous Query Results

Paging, Iterating Large Results, Sorting & Limiting

Which Method is Appropriate?

Paging and Sorting

Limiting Query Results

The repository proxy has two ways to derive a store-specific query from the method name:

- By deriving the query from the method name directly.
- By using a manually defined query.

Available options depend on the actual store. However, there must be a strategy that decides what actual query is created. The next section describes the available options.

Query Lookup Strategies

The following strategies are available for the repository infrastructure to resolve the query. With XML configuration, you can configure the strategy at the namespace through the `query-lookup-strategy` attribute. For Java configuration, you can use the `queryLookupStrategy` attribute of the `EnableJpaRepositories` annotation. Some strategies may not be supported for particular datastores.

- `CREATE` attempts to construct a store-specific query from the query method name. The general approach is to remove a given set of well known prefixes from the method name and parse the rest of

the method. You can read more about query construction in “Query Creation”.

- `USE_DECLARED_QUERY` tries to find a declared query and throws an exception if it cannot find one. The query can be defined by an annotation somewhere or declared by other means. See the documentation of the specific store to find available options for that store. If the repository infrastructure does not find a declared query for the method at bootstrap time, it fails.
- `CREATE_IF_NOT_FOUND` (the default) combines `CREATE` and `USE_DECLARED_QUERY`. It looks up a declared query first, and, if no declared query is found, it creates a custom method name-based query. This is the default lookup strategy and, thus, is used if you do not configure anything explicitly. It allows quick query definition by method names but also custom-tuning of these queries by introducing declared queries as needed.

Query Creation

The query builder mechanism built into the Spring Data repository infrastructure is useful for building constraining queries over entities of the repository.

The following example shows how to create a number of queries:

Query creation from method names

```
interface PersonRepository extends Repository<Person, Long> {  
  
    List<Person> findByEmailAddressAndLastname(EmailAddress emailAddress, String lastname);  
  
    // Enables the distinct flag for the query  
    List<Person> findDistinctPeopleByLastnameOrFirstname(String lastname, String firstname);  
    List<Person> findPeopleDistinctByLastnameOrFirstname(String lastname, String firstname);  
  
    // Enabling ignoring case for an individual property  
    List<Person> findByLastnameIgnoreCase(String lastname);  
    // Enabling ignoring case for all suitable properties  
    List<Person> findByLastnameAndFirstnameAllIgnoreCase(String lastname, String firstname);  
  
    // Enabling static ORDER BY for a query  
    List<Person> findByLastnameOrderByFirstnameAsc(String lastname);  
    List<Person> findByLastnameOrderByFirstnameDesc(String lastname);  
}
```

JAVA

Parsing query method names is divided into subject and predicate. The first part (`find...By` , `exists...By`) defines the subject of the query, the second part forms the predicate. The introducing clause (subject) can contain further expressions. Any text between `find` (or other introducing keywords) and `By` is con-

sidered to be descriptive unless using one of the result-limiting keywords such as a `Distinct` to set a distinct flag on the query to be created or `Top / First` to limit query results.

The appendix contains the [full list of query method subject keywords](#) and [query method predicate keywords including sorting and letter-casing modifiers](#). However, the first `By` acts as a delimiter to indicate the start of the actual criteria predicate. At a very basic level, you can define conditions on entity properties and concatenate them with `And` and `Or`.

The actual result of parsing the method depends on the persistence store for which you create the query. However, there are some general things to notice:

- The expressions are usually property traversals combined with operators that can be concatenated. You can combine property expressions with `AND` and `OR`. You also get support for operators such as `Between`, `LessThan`, `GreaterThan`, and `Like` for the property expressions. The supported operators can vary by datastore, so consult the appropriate part of your reference documentation.
- The method parser supports setting an `IgnoreCase` flag for individual properties (for example, `findByLastNameIgnoreCase(...)`) or for all properties of a type that supports ignoring case (usually `String` instances — for example, `findByLastNameAndFirstnameAllIgnoreCase(...)`). Whether ignoring cases is supported may vary by store, so consult the relevant sections in the reference documentation for the store-specific query method.
- You can apply static ordering by appending an `OrderBy` clause to the query method that references a property and by providing a sorting direction (`Asc` or `Desc`). To create a query method that supports dynamic sorting, see “Paging, Iterating Large Results, Sorting & Limiting”.

Reserved Method Names

While derived repository methods bind to properties by name, there are a few exceptions to this rule when it comes to certain method names inherited from the base repository targeting the *identifier* property. Those *reserved methods* like `CrudRepository#findById` (or just `findById`) are targeting the *identifier* property regardless of the actual property name used in the declared method.

Consider the following domain type holding a property `pk` marked as the identifier via `@Id` and a property called `id`. In this case you need to pay close attention to the naming of your lookup methods as they may collide with predefined signatures:

```
class User {  
    @Id Long pk;           1  
  
    Long id;              2
```

JAVA

```

// ...
}

interface UserRepository extends Repository<User, Long> {

    Optional<User> findById(Long id);           3

    Optional<User> findByPk(Long pk);          4

    Optional<User> findUserById(Long id);      5
}

```

- 1 The identifier property (primary key).
- 2 A property named `id`, but not the identifier.
- 3 Targets the `pk` property (the one marked with `@Id` which is considered to be the identifier) as it refers to a `CrudRepository` base repository method. Therefore, it is not a derived query using of `id` as the property name would suggest because it is one of the *reserved methods*.
- 4 Targets the `pk` property by name as it is a derived query.
- 5 Targets the `id` property by using the descriptive token between `find` and `by` to avoid collisions with *reserved methods*.

This special behaviour not only targets lookup methods but also applies to the `exists` and `delete` ones. Please refer to the “[Repository query keywords](#)” for the list of methods.

Property Expressions

Property expressions can refer only to a direct property of the managed entity, as shown in the preceding example. At query creation time, you already make sure that the parsed property is a property of the managed domain class. However, you can also define constraints by traversing nested properties. Consider the following method signature:

```
List<Person> findByAddressZipCode(ZipCode zipCode);
```

JAVA

Assume a `Person` has an `Address` with a `ZipCode`. In that case, the method creates the `x.address.zipCode` property traversal. The resolution algorithm starts by interpreting the entire part (`AddressZipCode`) as the property and checks the domain class for a property with that name (uncapitalized). If the algorithm succeeds, it uses that property. If not, the algorithm splits up the source at the camel-case parts from the right side into a head and a tail and tries to find the corresponding property — in our example, `AddressZip` and `Code`. If the algorithm finds a property with that head, it takes the

tail and continues building the tree down from there, splitting the tail up in the way just described. If the first split does not match, the algorithm moves the split point to the left (`Address` , `ZipCode`) and continues.

Although this should work for most cases, it is possible for the algorithm to select the wrong property. Suppose the `Person` class has an `addressZip` property as well. The algorithm would match in the first split round already, choose the wrong property, and fail (as the type of `addressZip` probably has no `code` property).

To resolve this ambiguity you can use `_` inside your method name to manually define traversal points. So our method name would be as follows:

```
List<Person> findByAddress_ZipCode(ZipCode zipCode);
```

JAVA

| NOTE

Because we treat underscores (`_`) as a reserved character, we strongly advise to follow standard Java naming conventions (that is, not using underscores in property names but applying camel case instead).

| CAUTION

Field Names starting with underscore:

Field names may start with underscores like `String _name` . Make sure to preserve the `_` as in `_name` and use double `_` to split nested paths like `user__name` .

Upper Case Field Names:

Field names that are all uppercase can be used as such. Nested paths if applicable require splitting via `_` as in `USER_name` .

Field Names with 2nd uppercase letter:

Field names that consist of a starting lower case letter followed by an uppercase one like `String qCode` can be resolved by starting with two upper case letters as in `QCode` . Please be aware of potential path ambiguities.

Path Ambiguities:

In the following sample the arrangement of properties `qCode` and `q` , with `q` containing a property called `code` , creates an ambiguity for the path `QCode` .

```
record Container(String qCode, Code q) {}  
record Code(String code) {}
```

Since a direct match on a property is considered first, any potential nested paths will not be considered and the algorithm picks the `qCode` field. In order to select the `code` field in `q` the underscore notation `Q_Code` is required.

Repository Methods Returning Collections or Iterables

Query methods that return multiple results can use standard Java `Iterable`, `List`, and `Set`. Beyond that, we support returning Spring Data's `Streamable`, a custom extension of `Iterable`, as well as collection types provided by [Vavr](#). Refer to the appendix explaining all possible [query method return types](#).

Using Streamable as Query Method Return Type

You can use `Streamable` as alternative to `Iterable` or any collection type. It provides convenience methods to access a non-parallel `Stream` (missing from `Iterable`) and the ability to directly `...filter(...)` and `...map(...)` over the elements and concatenate the `Streamable` to others:

Using Streamable to combine query method results

JAVA

```
interface PersonRepository extends Repository<Person, Long> {  
    Streamable<Person> findByFirstnameContaining(String firstname);  
    Streamable<Person> findByLastnameContaining(String lastname);  
}  
  
Streamable<Person> result = repository.findByFirstnameContaining("av")  
    .and(repository.findByLastnameContaining("ea"));
```

Returning Custom Streamable Wrapper Types

Providing dedicated wrapper types for collections is a commonly used pattern to provide an API for a query result that returns multiple elements. Usually, these types are used by invoking a repository method returning a collection-like type and creating an instance of the wrapper type manually. You can avoid that additional step as Spring Data lets you use these wrapper types as query method return types if they meet the following criteria:

1. The type implements `Streamable`.
2. The type exposes either a constructor or a static factory method named `of(...)` or `valueOf(...)` that takes `Streamable` as an argument.

The following listing shows an example:

JAVA

```
class Product {  
    MonetaryAmount getPrice() { ... }  
}  
  
@RequiredArgsConstructor(staticName = "of")  
class Products implements Streamable<Product> {  
    1  
    2
```

```

private final Streamable<Product> streamable;

public MonetaryAmount getTotal() {
    return streamable.stream()
        .map(Product::getPrice)
        .reduce(Money.of(0), MonetaryAmount::add);
}

@Override
public Iterator<Product> iterator() {
    return streamable.iterator();
}

interface ProductRepository implements Repository<Product, Long> {
    Products findAllByDescriptionContaining(String text);
}

```

- 1 A `Product` entity that exposes API to access the product's price.
- 2 A wrapper type for a `Streamable<Product>` that can be constructed by using `Products.of(...)` (factory method created with the Lombok annotation). A standard constructor taking the `Streamable<Product>` will do as well.
- 3 The wrapper type exposes an additional API, calculating new values on the `Streamable<Product>`.
- 4 Implement the `Streamable` interface and delegate to the actual result.
- 5 That wrapper type `Products` can be used directly as a query method return type. You do not need to return `Streamable<Product>` and manually wrap it after the query in the repository client.

Support for Vavr Collections

[Vavr](#) is a library that embraces functional programming concepts in Java. It ships with a custom set of collection types that you can use as query method return types, as the following table shows:

Vavr collection type	Used Vavr implementation type	Valid Java source types
<code>io.vavr.collection.Seq</code>	<code>io.vavr.collection.List</code>	<code>java.util.Iterable</code>
<code>io.vavr.collection.Set</code>	<code>io.vavr.collection.LinkedHashSet</code>	<code>java.util.Iterable</code>
<code>io.vavr.collection.Map</code>	<code>io.vavr.collection.LinkedHashMap</code>	<code>java.util.Map</code>

You can use the types in the first column (or subtypes thereof) as query method return types and get the types in the second column used as implementation type, depending on the Java type of the actual query result (third column). Alternatively, you can declare `Traversable` (the Vavr `Iterable` equivalent), and we then derive the implementation class from the actual return value. That is, a `java.util.List` is turned into a Vavr `List` or `Seq`, a `java.util.Set` becomes a Vavr `LinkedHashSet` `Set`, and so on.

Streaming Query Results

You can process the results of query methods incrementally by using a Java 8 `Stream<T>` as the return type. Instead of wrapping the query results in a `Stream`, data store-specific methods are used to perform the streaming, as shown in the following example:

Stream the result of a query with Java 8 `Stream<T>`

```
@Query("select u from User u")
Stream<User> findAllByCustomQueryAndStream();

Stream<User> readAllByFirstnameNotNull();

@Query("select u from User u")
Stream<User> streamAllPaged(Pageable pageable);
```

JAVA

| NOTE

A `Stream` potentially wraps underlying data store-specific resources and must, therefore, be closed after usage. You can either manually close the `Stream` by using the `close()` method or by using a Java 7 `try-with-resources` block, as shown in the following example:

Working with a `Stream<T>` result in a `try-with-resources` block

```
try (Stream<User> stream = repository.findAllByCustomQueryAndStream()) {
    stream.forEach(...);
}
```

JAVA

| NOTE

Not all Spring Data modules currently support `Stream<T>` as a return type.

Asynchronous Query Results

You can run repository queries asynchronously by using [Spring's asynchronous method running capability](#). This means the method returns immediately upon invocation while the actual query occurs in a task

that has been submitted to a Spring `TaskExecutor`. Asynchronous queries differ from reactive queries and should not be mixed. See the store-specific documentation for more details on reactive support. The following example shows a number of asynchronous queries:

```
@Async
Future<User> findByFirstname(String firstname); 1

@Async
CompletableFuture<User> findOneByFirstname(String firstname); 2
```

JAVA

- 1 Use `java.util.concurrent.Future` as the return type.
- 2 Use a Java 8 `java.util.concurrent.CompletableFuture` as the return type.

Paging, Iterating Large Results, Sorting & Limiting

To handle parameters in your query, define method parameters as already seen in the preceding examples. Besides that, the infrastructure recognizes certain specific types like `Pageable`, `Sort` and `Limit`, to apply pagination, sorting and limiting to your queries dynamically. The following example demonstrates these features:

Using `Pageable`, `Slice`, `ScrollPosition`, `Sort` and `Limit` in query methods

```
Page<User> findByLastname(String lastname, Pageable pageable);

Slice<User> findByLastname(String lastname, Pageable pageable);

Window<User> findTop10ByLastname(String lastname, ScrollPosition position, Sort sort);

List<User> findByLastname(String lastname, Sort sort);

List<User> findByLastname(String lastname, Sort sort, Limit limit);

List<User> findByLastname(String lastname, Pageable pageable);
```

JAVA

IMPORTANT

APIs taking `Sort`, `Pageable` and `Limit` expect non- null values to be handed into methods. If you do not want to apply any sorting or pagination, use `Sort.unsorted()`, `Pageable.unpaged()` and `Limit.unlimited()`.

The first method lets you pass an `org.springframework.data.domain.Pageable` instance to the query method to dynamically add paging to your statically defined query. A `Page` knows about the total number of elements and pages available. It does so by the infrastructure triggering a count query to calcu-

late the overall number. As this might be expensive (depending on the store used), you can instead return a `Slice`. A `Slice` knows only about whether a next `Slice` is available, which might be sufficient when walking through a larger result set.

Sorting options are handled through the `Pageable` instance, too. If you need only sorting, add an `org.springframework.data.domain.Sort` parameter to your method. As you can see, returning a `List` is also possible. In this case, the additional metadata required to build the actual `Page` instance is not created (which, in turn, means that the additional count query that would have been necessary is not issued). Rather, it restricts the query to look up only the given range of entities.

NOTE

To find out how many pages you get for an entire query, you have to trigger an additional count query. By default, this query is derived from the query you actually trigger.

IMPORTANT

Special parameters may only be used once within a query method. Some special parameters described above are mutually exclusive. Please consider the following list of invalid parameter combinations.

Parameters	Example	Reason
Pageable and Sort	<code>findBy...(Pageable page, Sort sort)</code>	Pageable already defines Sort
Pageable and Limit	<code>findBy...(Pageable page, Limit limit)</code>	Pageable already defines a limit.

The `Top` keyword used to limit results can be used to along with `Pageable` whereas `Top` defines the total maximum of results, whereas the `Pageable` parameter may reduce this number.

Which Method is Appropriate?

The value provided by the Spring Data abstractions is perhaps best shown by the possible query method return types outlined in the following table below. The table shows which types you can return from a query method

Table 1. Consuming Large Query Results

Method	Amount of Data Fetched	Query Structure	Constraints
--------	------------------------	-----------------	-------------

Method	Amount of Data Fetched	Query Structure	Constraints
<code>List<T></code>	All results.	Single query.	Query results haust all men Fetching all d time-intensiv
<code>Streamable<T></code>	All results.	Single query.	Query results haust all men Fetching all d time-intensiv
<code>Stream<T></code>	Chunked (one-by-one or in batches) depending on <code>Stream</code> consumption.	Single query using typically cursors.	Streams must after usage to source leaks.
<code>Flux<T></code>	Chunked (one-by-one or in batches) depending on <code>Flux</code> consumption.	Single query using typically cursors.	Store module vide reactive infrastructure
<code>Slice<T></code>	<code>Pageable.getPageSize() + 1</code> at <code>Pageable.getOffset()</code>	One to many queries fetching data starting at <code>Pageable.getOffset()</code> applying limiting.	A <code>Slice</code> can gate to the n <code>Slice</code> details v there is to fetch - Offset-l queries inefficie the offs large be databas to mate full resu

Method	Amount of Data Fetched	Query Structure	Constraints
Offset-based Window<T>	limit + 1 at OffsetScrollPosition.getOffset()	One to many queries fetching data starting at OffsetScrollPosition.getOffset() applying limiting.	A window ca igate to the n window . - Window details v there is to fetch - Offset-l queries inefficie the offs large be databas to mate full resu
Page<T>	Pageable.getPageSize() at Pageable.getOffset()	One to many queries starting at Pageable.getOffset() applying lim- iting. Additionally, COUNT(...) query to determine the total number of ele- ments can be required.	Often times, queries are re that are costl Offset-l queries inefficie the offs large be databas to mate full resu

Method	Amount of Data Fetched	Query Structure	Constraints
Keyset-based Window<T>	limit + 1 using a rewritten WHERE condition	One to many queries fetching data starting at KeysetScrollPosition.getKeys() applying limiting.	A Window ca igate to the n Window . - Window details v there is to fetch - Keyset-l queries proper i structur cient qu - Most de do not v when Ke based q sults coi values. - Results pose all keys in t sults rec projecti lect pot more pr than rec the actu nroiecti

Paging and Sorting

You can define simple sorting expressions by using property names. You can concatenate expressions to collect multiple criteria into one expression.

Defining sort expressions

```
Sort sort = Sort.by("firstname").ascending()
    .and(Sort.by("lastname").descending());
```

JAVA

For a more type-safe way to define sort expressions, start with the type for which to define the sort expression and use method references to define the properties on which to sort.

Defining sort expressions by using the type-safe API

JAVA

```
TypedSort<Person> person = Sort.sort(Person.class);

Sort sort = person.by(Person::getFirstname).ascending()
    .and(person.by(Person::getLastname).descending());
```

NOTE

`TypedSort.by(...)` makes use of runtime proxies by (typically) using CGLib, which may interfere with native image compilation when using tools such as Graal VM Native.

If your store implementation supports Querydsl, you can also use the generated metamodel types to define sort expressions:

Defining sort expressions by using the Querydsl API

JAVA

```
QSort sort = QSort.by(QPerson.firstname.asc())
    .and(QSort.by(QPerson.lastname.desc()));
```

Limiting Query Results

In addition to paging it is possible to limit the result size using a dedicated `Limit` parameter. You can also limit the results of query methods by using the `First` or `Top` keywords, which you can use interchangeably but may not be mixed with a `Limit` parameter. You can append an optional numeric value to `Top` or `First` to specify the maximum result size to be returned. If the number is left out, a result size of 1 is assumed. The following example shows how to limit the query size:

Limiting the result size of a query with `Top` and `First`

JAVA

```
List<User> findByLastname(String lastname, Limit limit);

User findFirstByOrderByLastnameAsc();

User findTopByLastnameOrderByAgeDesc(String lastname);

Page<User> queryFirst10ByLastname(String lastname, Pageable pageable);

Slice<User> findTop3By(Pageable pageable);

List<User> findFirst10ByLastname(String lastname, Sort sort);

List<User> findTop10ByLastname(String lastname, Pageable pageable);
```

The limiting expressions also support the `Distinct` keyword for datastores that support distinct queries. Also, for the queries that limit the result set to one instance, wrapping the result into with the `Optional` keyword is supported.

If pagination or slicing is applied to a limiting query pagination (and the calculation of the number of available pages), it is applied within the limited result.

NOTE

Limiting the results in combination with dynamic sorting by using a `Sort` parameter lets you express query methods for the 'K' smallest as well as for the 'K' biggest elements.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

JPA Query Methods

JPA Query Methods

- Query Lookup Strategies

 - Declared Queries

- Query Creation

 - Annotation-based Configuration

- Using JPA Named Queries

 - XML Named Query Definition

 - Declaring Interfaces

- Using @Query

 - Using Advanced LIKE Expressions

 - Native Queries

 - Query Introspection and Rewriting

 - Applying a QueryRewriter

- Using Sort

 - JpaSort.unsafe(...) limitations

- Scrolling Large Query Results

- Using Named Parameters

- Templated Queries and Expressions

- Other Methods

- Modifying Queries

 - Derived Delete Queries

- Applying Query Hints

 - Adding Comments to Queries

- Configuring Fetch- and LoadGraphs

- Scrolling

 - Scrolling using Offset

 - Scrolling using Keyset-Filtering

This section describes the various ways to create a query with Spring Data JPA.

Query Lookup Strategies

The JPA module supports defining a query manually as a String or having it being derived from the method name.

Derived queries with the predicates `IsStartingWith`, `StartingWith`, `StartsWith`, `IsEndingWith`, `EndingWith`, `EndsWith`, `IsNotContaining`, `NotContaining`, `NotContains`, `IsContaining`, `Containing`, `Contains` the respective arguments for these queries will get sanitized. This means if the arguments actually contain characters recognized by `LIKE` as wildcards these will get escaped so they match only as literals. The escape character used can be configured by setting the `escapeCharacter` of the `@EnableJpaRepositories` annotation. Compare with Using Value Expressions.

Declared Queries

Although getting a query derived from the method name is quite convenient, one might face the situation in which either the method name parser does not support the keyword one wants to use or the method name would get unnecessarily ugly. So you can either use JPA named queries through a naming convention (see Using JPA Named Queries for more information) or rather annotate your query method with `@Query` (see Using `@Query` for details).

Query Creation

Generally, the query creation mechanism for JPA works as described in [Query Methods](#). The following example shows what a JPA query method translates into:

Example 1. Query creation from method names

```
public interface UserRepository extends Repository<User, Long> {  
  
    List<User> findByEmailAddressAndLastname(String emailAddress, String lastname);  
}
```

JAVA

We create a query using JPQL translating into the following query: `select u from User u where u.emailAddress = ?1 and u.lastname = ?2`. Spring Data JPA does a property check and traverses nested properties, as described in [Property Expressions](#).

The following table describes the keywords supported for JPA and what a method containing that keyword translates to:

Table 1. Supported keywords inside method names

Keyword	Sample	JPQL snippet
---------	--------	--------------

Keyword	Sample	JPQL snippet
Distinct	findDistinctByLastnameAndFirstname	select distinct ... where x.lastname = ?1 and x.firstname = ?2
And	findByLastnameAndFirstname	... where x.lastname = ?1 and x.firstname = ?2
Or	findByLastnameOrFirstname	... where x.lastname = ?1 or x.firstname = ? 2
Is, Equals	findByFirstname, findByFirstnameIs, findByFirstnameEquals	... where x.firstname = ?1 (or ... where x.firstname IS NULL if the argument is null)
Between	findByStartDateBetween	... where x.startDate between ?1 and ?2
LessThan	findByAgeLessThan	... where x.age < ?1
LessThanEqual	findByAgeLessThanEqual	... where x.age <= ?1
GreaterThan	findByAgeGreaterThan	... where x.age > ?1
GreaterThanEqual	findByAgeGreaterThanEqual	... where x.age >= ?1
After	findByStartDateAfter	... where x.startDate > ?1
Before	findByStartDateBefore	... where x.startDate < ?1
IsNull, Null	findByAge(Is)Null	... where x.age is null
IsNotNull, NotNull	findByAge(Is)NotNull	... where x.age is not null
Like	findByFirstnameLike	... where x.firstname like ?1
NotLike	findByFirstnameNotLike	... where x.firstname not like ?1

Keyword	Sample	JPQL snippet
StartingWith	findByFirstnameStartingWith	... where x.firstname like ?1 (parameter bound with appended %)
EndingWith	findByFirstnameEndingWith	... where x.firstname like ?1 (parameter bound with prepended %)
Containing	findByFirstnameContaining	... where x.firstname like ?1 (parameter bound wrapped in %)
OrderBy	findByAgeOrderByLastNameDesc	... where x.age = ?1 order by x.lastname desc
Not	findByLastNameNot	... where x.lastname <> ?1 (or ... where x.lastname IS NOT NULL if the argument is null)
In	findByAgeIn(Collection<Age> ages)	... where x.age in ?1
NotIn	findByAgeNotIn(Collection<Age> ages)	... where x.age not in ?1
True	findByActiveTrue()	... where x.active = true
False	findByActiveFalse()	... where x.active = false
IgnoreCase	findByFirstnameIgnoreCase	... where UPPER(x.firstname) = UPPER(?1)

NOTE

In and NotIn also take any subclass of `Collection` as a parameter as well as arrays or varargs. For other syntactical versions of the same logical operator, check [Repository query keywords](#).

WARNING

WARNING

`DISTINCT` can be tricky and not always producing the results you expect. For example, `select distinct u from User u` will produce a complete different result than `select distinct u.lastname from User u`. In the first case, since you are including `User.id`, nothing will be duplicated, hence you'll get the whole table, and it would be of `User` objects.

However, that latter query would narrow the focus to just `User.lastname` and find all unique last names for that table. This would also yield a `List<String>` result set instead of a `List<User>` result set.

`countDistinctByLastname(String lastname)` can also produce unexpected results. Spring Data JPA will derive `select count(distinct u.id) from User u where u.lastname = ?1`. Again, since `u.id` won't hit any duplicates, this query will count up all the users that had the binding last name. Which would be the same as `countByLastname(String lastname)`!

What is the point of this query anyway? To find the number of people with a given last name? To find the number of *distinct* people with that binding last name? To find the number of *distinct last names*? (That last one is an entirely different query!) Using `distinct` sometimes requires writing the query by hand and using `@Query` to best capture the information you seek, since you also may be needing a projection to capture the result set.

Annotation-based Configuration

Annotation-based configuration has the advantage of not needing another configuration file to be edited, lowering maintenance effort. You pay for that benefit by the need to recompile your domain class for every new query declaration.

Example 2. Annotation-based named query configuration

```
@Entity
@NamedQuery(name = "User.findByEmailAddress",
    query = "select u from User u where u.emailAddress = ?1")
public class User {

}
```

JAVA

Using JPA Named Queries

NOTE

The examples use the `<named-query />` element and `@NamedQuery` annotation. The queries for these configuration elements have to be defined in the JPA query language. Of course, you can use `<named-native-query />` or `@NamedNativeQuery` too. These elements let you define the query in native SQL by losing the database platform independence.

XML Named Query Definition

To use XML configuration, add the necessary `<named-query />` element to the `orm.xml` JPA configuration file located in the `META-INF` folder of your classpath. Automatic invocation of named queries is enabled by using some defined naming convention. For more details, see below.

Example 3. XML named query configuration

```
<named-query name="User.findByLastname">
  <query>select u from User u where u.lastname = ?1</query>
</named-query>
```

XML

The query has a special name that is used to resolve it at runtime.

Declaring Interfaces

To allow these named queries, specify the `UserRepository` as follows:

Example 4. Query method declaration in UserRepository

```
public interface UserRepository extends JpaRepository<User, Long> {

    List<User> findByLastname(String lastname);

    User findByEmailAddress(String emailAddress);
}
```

JAVA

Spring Data tries to resolve a call to these methods to a named query, starting with the simple name of the configured domain class, followed by the method name separated by a dot. So the preceding example would use the named queries defined earlier instead of trying to create a query from the method name.

Using @Query

Using named queries to declare queries for entities is a valid approach and works fine for a small number of queries. As the queries themselves are tied to the Java method that runs them, you can actually bind them directly by using the Spring Data JPA `@Query` annotation rather than annotating them to the domain class. This frees the domain class from persistence specific information and co-locates the query to the repository interface.

Queries annotated to the query method take precedence over queries defined using `@NamedQuery` or named queries declared in `orm.xml`.

The following example shows a query created with the `@Query` annotation:

Example 5. Declare query at the query method using `@Query`

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    @Query("select u from User u where u.emailAddress = ?1")  
    User findByEmailAddress(String emailAddress);  
}
```

JAVA

Using Advanced LIKE Expressions

The query running mechanism for manually defined queries created with `@Query` allows the definition of advanced `LIKE` expressions inside the query definition, as shown in the following example:

Example 6. Advanced Like expressions in `@Query`

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    @Query("select u from User u where u.firstname like %?1")  
    List<User> findByFirstnameEndsWith(String firstname);  
}
```

JAVA

In the preceding example, the `LIKE` delimiter character (`%`) is recognized, and the query is transformed into a valid JPQL query (removing the `%`). Upon running the query, the parameter passed to the method call gets augmented with the previously recognized `LIKE` pattern.

Native Queries

Using the `@NativeQuery` annotation allows running native queries, as shown in the following example:

Example 7. Declare a native query at the query method using `@NativeQuery`

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    @NativeQuery(value = "SELECT * FROM USERS WHERE EMAIL_ADDRESS = ?1")  
    User findByEmailAddress(String emailAddress);  
}
```

JAVA

NOTE

The `@NativeQuery` annotation is mostly a composed annotation for `@Query(nativeQuery=true)` but it also provides additional attributes such as `sqlResultSetMapping` to leverage JPA's `@SqlResultSetMapping(...)`.

NOTE

Spring Data can rewrite simple queries for pagination and sorting. More complex queries require either [JSqlParser](#) to be on the class path or a `countQuery` declared in your code. See the example below for more details.

Example 8. Declare native count queries for pagination at the query method by using `@NativeQuery`

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    @NativeQuery(value = "SELECT * FROM USERS WHERE LASTNAME = ?1",  
        countQuery = "SELECT count(*) FROM USERS WHERE LASTNAME = ?1")  
    Page<User> findByLastname(String lastname, Pageable pageable);  
}
```

JAVA

A similar approach also works with named native queries, by adding the `.count` suffix to a copy of your query. You probably need to register a result set mapping for your count query, though.

Next to obtaining mapped results, native queries allow you to read the raw `Tuple` from the database by choosing a `Map` container as the method's return type. The resulting map contains key/value pairs representing the actual database column name and the value.

Example 9. Native query returning raw column name/value pairs

```
interface UserRepository extends JpaRepository<User, Long> {  
  
    @NativeQuery("SELECT * FROM USERS WHERE EMAIL_ADDRESS = ?1")  
    Map<String, Object> findRawMapByEmail(String emailAddress);  
  
    @NativeQuery("SELECT * FROM USERS WHERE LASTNAME = ?1")  
    List<Map<String, Object>> findRawMapByLastname(String lastname);  
}
```

JAVA

- 1 Single `Map` result backed by a `Tuple`.
- 2 Multiple `Map` results backed by `Tuple`s.

NOTE

String-based Tuple Queries are only supported by Hibernate. Eclipselink supports only Criteria-based Tuple Queries.

Query Introspection and Rewriting

Spring Data JPA provides a wide range of functionality that can be used to run various flavors of queries. Specifically, given a declared query, Spring Data JPA can:

- Introspect a query for its projection and run a tuple query for interface projections
- Use DTO projections if the query uses constructor expressions and rewrite the projection when the query declares the entity alias or just a multi-select of expressions
- Apply dynamic sorting
- Derive a `COUNT` query

For this purpose, we ship with Query Parsers specific to HQL (Hibernate) and EQL (EclipseLink) dialects as these dialects are well-defined. SQL on the other hand allows for quite some variance across dialects. Because of this, there is no way Spring Data will ever be able to support all levels of query complexity. We are not general purpose SQL parser library but one to increase developer productivity through making query execution simpler. Our built-in SQL query enhancer supports only simple queries for introspection `COUNT` query derivation. A more complex query will require either the usage of [JSqlParser](#) or that you provide a `COUNT` query through `@Query(countQuery=...)`. If `JSqlParser` is on the class path, Spring Data JPA will use it for native queries.

For a fine-grained control over selection, you can configure [QueryEnhancerSelector](#) using `@EnableJpaRepositories`:

Example 10. Spring Data JPA repositories using JavaConfig

```
@Configuration
@EnableJpaRepositories(queryEnhancerSelector = MyQueryEnhancerSelector.class)
class ApplicationConfig {
    // ...
}
```

JAVA

`QueryEnhancerSelector` is a strategy interface intended to select a [QueryEnhancer](#) based on a specific query. You can also provide your own `QueryEnhancer` implementation if you want.

Applying a QueryRewriter

Sometimes, no matter how many features you try to apply, it seems impossible to get Spring Data JPA to apply every thing you'd like to a query before it is sent to the `EntityManager`.

You have the ability to get your hands on the query, right before it's sent to the `EntityManager` and "rewrite" it. That is, you can make any alterations at the last moment. Query rewriting applies to the actual query and, when applicable, to count queries. Count queries are optimized and therefore, either not necessary or a count is obtained through other means, such as derived from a Hibernate `SelectionQuery` if there is an enclosing transaction.

Example 11. Declare a `QueryRewriter` using `@Query` and `@NativeQuery`

```
public interface MyRepository extends JpaRepository<User, Long> {  
  
    @NativeQuery(value = "select original_user_alias.* from SD_USER  
original_user_alias",  
        queryRewriter = MyQueryRewriter.class)  
    List<User> findByNativeQuery(String param);  
  
    @Query(value = "select original_user_alias from User original_user_alias",  
        queryRewriter = MyQueryRewriter.class)  
    List<User> findByNonNativeQuery(String param);  
}
```

This example shows both a native (pure SQL) rewriter as well as a JPQL query, both leveraging the same `QueryRewriter`. In this scenario, Spring Data JPA will look for a bean registered in the application context of the corresponding type.

You can write a query rewriter like this:

Example 12. Example `QueryRewriter`

```
public class MyQueryRewriter implements QueryRewriter {  
  
    @Override  
    public String rewrite(String query, Sort sort) {  
        return query.replaceAll("original_user_alias", "rewritten_user_alias");  
    }  
}
```

You have to ensure your `QueryRewriter` is registered in the application context, whether it's by applying one of Spring Framework's `@Component`-based annotations, or having it as part of a `@Bean` method inside an `@Configuration` class.

Another option is to have the repository itself implement the interface.

Example 13. Repository that provides the QueryRewriter

```
public interface MyRepository extends JpaRepository<User, Long>, QueryRewriter {  
  
    @Query(value = "select original_user_alias.* from SD_USER original_user_alias",  
            nativeQuery = true,  
            queryRewriter = MyRepository.class)  
    List<User> findByNativeQuery(String param);  
  
    @Query(value = "select original_user_alias from User original_user_alias",  
            queryRewriter = MyRepository.class)  
    List<User> findByNonNativeQuery(String param);  
  
    @Override  
    default String rewrite(String query, Sort sort) {  
        return query.replaceAll("original_user_alias", "rewritten_user_alias");  
    }  
}
```

Depending on what you're doing with your `QueryRewriter`, it may be advisable to have more than one, each registered with the application context.

NOTE

In a CDI-based environment, Spring Data JPA will search the `BeanManager` for instances of your implementation of `QueryRewriter`.

Using Sort

Sorting can be done by either providing a `PageRequest` or by using `Sort` directly. The properties actually used within the `Order` instances of `Sort` need to match your domain model, which means they need to resolve to either a property or an alias used within the query. The JPQL defines this as a state field path expression.

NOTE

Using any non-referenceable path expression leads to an `Exception`.

However, using `Sort` together with `@Query` lets you sneak in non-path-checked `Order` instances containing functions within the `ORDER BY` clause. This is possible because the `Order` is appended to the given query string. By default, Spring Data JPA rejects any `Order` instance containing function calls, but you can use `JpaSort.unsafe` to add potentially unsafe ordering.

The following example uses `Sort` and `JpaSort`, including an `unsafe` option on `JpaSort`:

Example 14. Using `Sort` and `JpaSort`

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    @Query("select u from User u where u.lastname like ?1%")  
    List<User> findByAndSort(String lastname, Sort sort);  
  
    @Query("select u.id, LENGTH(u.firstname) as fn_len from User u where u.lastname like  
?1%")  
    List<Object[]> findByAsArrayAndSort(String lastname, Sort sort);  
}  
  
repo.findByAndSort("lannister", Sort.by("firstname"));  
repo.findByAndSort("stark", Sort.by("LENGTH(firstname)"));  
repo.findByAndSort("targaryen", JpaSort.unsafe("LENGTH(firstname)"));  
repo.findByAsArrayAndSort("bolton", Sort.by("fn_len"));
```

JAVA

- 1 Valid `Sort` expression pointing to property in domain model.
- 2 Invalid `Sort` containing function call. Throws Exception.
- 3 Valid `Sort` containing explicitly *unsafe* Order .
- 4 Valid `Sort` expression pointing to aliased function.

JpaSort.unsafe(...) limitations

`JpaSort.unsafe(...)` operates in two modes:

- When used with derived Queries or String-based Queries, the order string is appended to the query.
- When used with Query by Example or Specifications (that use `CriteriaQuery`), order expressions are parsed and added to the `CriteriaQuery` as expressions. Query expressions can contain function calls, various clauses (such as `CASE WHEN`, arithmetic expressions) or property paths. Order translation does not support subquery expressions, `TREAT` and `CAST`.

Scrolling Large Query Results

When working with large data sets, scrolling can help to process those results efficiently without loading all results into memory.

You have multiple options to consume large query results:

1. [Paging](#). You have learned in the previous chapter about `Pageable` and `PageRequest`.
2. Offset-based scrolling. This is a lighter variant than paging because it does not require the total result count.
3. Keyset-based scrolling. This method avoids [the shortcomings of offset-based result retrieval by leveraging database indexes](#).

Read more on [which method to use best](#) for your particular arrangement.

You can use the Scroll API with query methods, [Query-by-Example](#), and [Querydsl](#).

| NOTE

Scrolling with String-based query methods is not yet supported. Scrolling is also not supported using stored `@Procedure` query methods.

Using Named Parameters

By default, Spring Data JPA uses position-based parameter binding, as described in all the preceding examples. This makes query methods a little error-prone when refactoring regarding the parameter position. To solve this issue, you can use `@Param` annotation to give a method parameter a concrete name and bind the name in the query, as shown in the following example:

Example 15. Using named parameters

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    @Query("select u from User u where u.firstname = :firstname or u.lastname =  
:lastname")  
    User findByLastnameOrFirstname(@Param("lastname") String lastname,  
                                   @Param("firstname") String firstname);  
}
```

JAVA

| NOTE

The method parameters are switched according to their order in the defined query.

| NOTE

As of version 4, Spring fully supports Java 8's parameter name discovery based on the `-parameters` compiler flag. By using this flag in your build as an alternative to debug information, you can omit the `@Param` annotation for named parameters.

Templated Queries and Expressions

We support the usage of restricted expressions in manually defined queries that are defined with `@Query`. Upon the query being run, these expressions are evaluated against a predefined set of variables.

NOTE

If you are not familiar with Value Expressions, please refer to [Value Expressions Fundamentals](#) to learn about SpEL Expressions and Property Placeholders.

Spring Data JPA supports a template variable called `entityName`. Its usage is `select x from #{entityName} x`. It inserts the `entityName` of the domain type associated with the given repository. The `entityName` is resolved as follows: * If the domain type has set the name property on the `@Entity` annotation, it is used. * Otherwise, the simple class-name of the domain type is used.

The following example demonstrates one use case for the `#{entityName}` expression in a query string where you want to define a repository interface with a query method and a manually defined query:

Example 16. Using SpEL expressions in repository query methods: entityName

```
@Entity
public class User {

    @Id
    @GeneratedValue
    Long id;

    String lastname;
}

public interface UserRepository extends JpaRepository<User, Long> {

    @Query("select u from #{entityName} u where u.lastname = ?1")
    List<User> findByLastname(String lastname);
}
```

JAVA

To avoid stating the actual entity name in the query string of a `@Query` annotation, you can use the `#{entityName}` variable.

NOTE

The `entityName` can be customized by using the `@Entity` annotation. Customizations in `orm.xml` are not supported for the SpEL expressions.

Of course, you could have just used `User` in the query declaration directly, but that would require you to change the query as well. The reference to `#entityName` picks up potential future remappings of the `User` class to a different entity name (for example, by using `@Entity(name = "MyUser")`).

Another use case for the `#{#entityName}` expression in a query string is if you want to define a generic repository interface with specialized repository interfaces for a concrete domain type. To not repeat the definition of custom query methods on the concrete interfaces, you can use the entity name expression in the query string of the `@Query` annotation in the generic repository interface, as shown in the following example:

Example 17. Using SpEL expressions in Repository Query Methods: entityName with Inheritance

```
JAVA
@MappedSuperclass
public abstract class AbstractMappedType {
    ...
    String attribute;
}

@Entity
public class ConcreteType extends AbstractMappedType { ... }

@NoRepositoryBean
public interface MappedTypeRepository<T extends AbstractMappedType>
    extends Repository<T, Long> {

    @Query("select t from #{#entityName} t where t.attribute = ?1")
    List<T> findAllByAttribute(String attribute);
}

public interface ConcreteRepository
    extends MappedTypeRepository<ConcreteType> { ... }
```

In the preceding example, the `MappedTypeRepository` interface is the common parent interface for a few domain types extending `AbstractMappedType`. It also defines the generic `findAllByAttribute(...)` method, which can be used on instances of the specialized repository interfaces. If you now invoke `findAllByAttribute(...)` on `ConcreteRepository`, the query becomes `select t from ConcreteType t where t.attribute = ?1`.

You can also use Expressions to control arguments may also be used to control method arguments. In these expressions the entity name is not available, but the arguments are. They can be accessed by name or index as demonstrated in the following example.

Example 18. Using Value Expressions in Repository Query Methods: Accessing Arguments

```
@Query("select u from User u where u.firstname = ?1 and u.firstname=?#{[0]} and  
u.emailAddress = ?#{principal.emailAddress}")  
List<User> findByFirstnameAndCurrentUserWithCustomQuery(String firstname);
```

JAVA

For `like`-conditions one often wants to append `%` to the beginning or the end of a String valued parameter. This can be done by appending or prefixing a bind parameter marker or a SpEL expression with `%`. Again the following example demonstrates this.

Example 19. Using Value Expressions in Repository Query Methods: Wildcard shortcut

```
@Query("select u from User u where u.lastname like %:#{[0]}% and u.lastname like  
%:lastname%")  
List<User> findByLastnameWithSpelExpression(@Param("lastname") String lastname);
```

JAVA

When using `like`-conditions with values that are coming from a not secure source the values should be sanitized so they can't contain any wildcards and thereby allow attackers to select more data than they should be able to. For this purpose the `escape(String)` method is made available in the SpEL context. It prefixes all instances of `_` and `%` in the first argument with the single character from the second argument. In combination with the `escape` clause of the `like` expression available in JPQL and standard SQL this allows easy cleaning of bind parameters.

Example 20. Using Value Expressions in Repository Query Methods: Sanitizing Input Values

```
@Query("select u from User u where u.firstname like ?#{escape([0])}% escape ?#{  
escapeCharacter()}")  
List<User> findContainingEscaped(String namePart);
```

JAVA

Given this method declaration in a repository interface `findContainingEscaped("Peter_")` will find `Peter_Parker` but not `Peter Parker`. The escape character used can be configured by setting the `escapeCharacter` of the `@EnableJpaRepositories` annotation. Note that the method `escape(String)` available in the SpEL context will only escape the SQL and JPQL standard wildcards `_` and `%`. If the underlying database or the JPA implementation supports additional wildcards these will not get escaped.

Example 21. Using Value Expressions in Repository Query Methods: Configuration Properties

```
@Query("select u from User u where u.applicationName = ?  
${spring.application.name:unknown}")
```

JAVA

```
List<User> findContainingEscaped(String namePart);
```

You can refer in your query methods also to configuration property names including fallbacks if you wish to resolve a property from `Environment` during runtime. The property is being evaluated upon query execution. Typically, property placeholders resolve to String-like values.

Other Methods

Spring Data JPA offers many ways to build queries. But sometimes, your query may simply be too complicated for the techniques offered. In that situation, consider:

- If you haven't already, simply write the query yourself using [@Query](#).
- If that doesn't fit your needs, consider implementing a [custom implementation](#). This lets you register a method in your repository while leaving the implementation completely up to you. This gives you the ability to:
 - Talk directly to the `EntityManager` (writing pure HQL/JPQL/EQL/native SQL or using the **Criteria API**)
 - Leverage Spring Framework's `JdbcTemplate` (native SQL)
 - Use another 3rd-party database toolkit.
- Another option is putting your query inside the database and then using either Spring Data JPA's [@StoredProcedure](#) annotation or if it's a database function using the [@Query](#) annotation and invoking it with a `CALL`.

These tactics may be most effective when you need maximum control of your query, while still letting Spring Data JPA provide resource management.

Modifying Queries

All the previous sections describe how to declare queries to access a given entity or collection of entities. You can add custom modifying behavior by using the custom method facilities described in [Custom Implementations for Spring Data Repositories](#). As this approach is feasible for comprehensive custom functionality, you can modify queries that only need parameter binding by annotating the query method with `@Modifying`, as shown in the following example:

Example 22. Declaring manipulating queries

```
@Modifying
@Query("update User u set u.firstname = ?1 where u.lastname = ?2")
```

JAVA


```
int setFixedFirstnameFor(String firstname, String lastname);
```

Doing so triggers the query annotated to the method as an updating query instead of a selecting one. As the `EntityManager` might contain outdated entities after the execution of the modifying query, we do not automatically clear it (see the [JavaDoc](#) of `EntityManager.clear()` for details), since this effectively drops all non-flushed changes still pending in the `EntityManager`. If you wish the `EntityManager` to be cleared automatically, you can set the `@Modifying` annotation's `clearAutomatically` attribute to `true`.

The `@Modifying` annotation is only relevant in combination with the `@Query` annotation. Derived query methods or custom methods do not require this annotation.

Derived Delete Queries

Spring Data JPA also supports derived delete queries that let you avoid having to declare the JPQL query explicitly, as shown in the following example:

Example 23. Using a derived delete query

```
interface UserRepository extends Repository<User, Long> {  
  
    void deleteByRoleId(long roleId);  
  
    @Modifying  
    @Query("delete from User u where u.role.id = ?1")  
    void deleteInBulkByRoleId(long roleId);  
}
```

JAVA

Although the `deleteByRoleId(...)` method looks like it basically produces the same result as the `deleteInBulkByRoleId(...)`, there is an important difference between the two method declarations in terms of the way they are run. As the name suggests, the latter method issues a single JPQL query (the one defined in the annotation) against the database. This means even currently loaded instances of `User` do not see lifecycle callbacks invoked.

To make sure lifecycle queries are actually invoked, an invocation of `deleteByRoleId(...)` runs a query and then deletes the returned instances one by one, so that the persistence provider can actually invoke `@PreRemove` callbacks on those entities.

In fact, a derived delete query is a shortcut for running the query and then calling `CrudRepository.delete(Iterable<User> users)` on the result and keeping behavior in sync with the

implementations of other `delete(...)` methods in `CrudRepository`.

NOTE

When deleting a lot of objects you will need to consider the performance implications to ensure sufficient memory availability. All resulting objects are loaded into memory before being deleted and are held in the session until flushing or completing the transaction.

Applying Query Hints

To apply JPA query hints to the queries declared in your repository interface, you can use the `@QueryHints` annotation. It takes an array of JPA `@QueryHint` annotations plus a boolean flag to potentially disable the hints applied to the additional count query triggered when applying pagination, as shown in the following example:

Example 24. Using QueryHints with a repository method

```
public interface UserRepository extends Repository<User, Long> {  
  
    @QueryHints(value = { @QueryHint(name = "name", value = "value")},  
                  forCounting = false)  
    Page<User> findByLastname(String lastname, Pageable pageable);  
}
```

JAVA

The preceding declaration would apply the configured `@QueryHint` for that actual query but omit applying it to the count query triggered to calculate the total number of pages.

Adding Comments to Queries

Sometimes, you need to debug a query based upon database performance. The query your database administrator shows you may look VERY different than what you wrote using `@Query`, or it may look nothing like what you presume Spring Data JPA has generated regarding a custom finder or if you used query by example.

To make this process easier, you can insert custom comments into almost any JPA operation, whether it's a query or other operation by applying the `@Meta` annotation.

Example 25. Apply @Meta annotation to repository operations

```
public interface RoleRepository extends JpaRepository<Role, Integer> {  
  
    @Meta(comment = "find roles by name")
```

JAVA

```

List<Role> findByName(String name);

@Override
@Meta(comment = "find roles using QBE")
<S extends Role> List<S> findAll(Example<S> example);

@Meta(comment = "count roles for a given name")
long countByName(String name);

@Override
@Meta(comment = "exists based on QBE")
<S extends Role> boolean exists(Example<S> example);
}

```

This sample repository has a mixture of custom finders as well as overriding the inherited operations from `JpaRepository`. Either way, the `@Meta` annotation lets you add a `comment` that will be inserted into queries before they are sent to the database.

It's also important to note that this feature isn't confined solely to queries. It extends to the `count` and `exists` operations. And while not shown, it also extends to certain `delete` operations.

IMPORTANT

While we have attempted to apply this feature everywhere possible, some operations of the underlying `EntityManager` don't support comments. For example, `entityManager.createQuery()` is clearly documented as supporting comments, but `entityManager.find()` operations do not.

Neither JPQL logging nor SQL logging is a standard in JPA, so each provider requires custom configuration, as shown the sections below.

Activating Hibernate comments

To activate query comments in Hibernate, you must set `hibernate.use_sql_comments` to `true`.

If you are using Java-based configuration settings, this can be done like this:

Example 26. Java-based JPA configuration

```

@Bean
public Properties jpaProperties() {

    Properties properties = new Properties();
    properties.setProperty("hibernate.use_sql_comments", "true");
    return properties;
}

```

JAVA

If you have a `persistence.xml` file, you can apply it there:

Example 27. persistence.xml-based configuration

```
<persistence-unit name="my-persistence-unit">
    ...registered classes...

    <properties>
        <property name="hibernate.use_sql_comments" value="true" />
    </properties>
</persistence-unit>
```

XML

Finally, if you are using Spring Boot, then you can set it up inside your `application.properties` file:

Example 28. Spring Boot property-based configuration

```
spring.jpa.properties.hibernate.use_sql_comments=true
```

Activating EclipseLink comments

To activate query comments in EclipseLink, you must set `eclipselink.logging.level.sql` to `FINE`.

If you are using Java-based configuration settings, this can be done like this:

Example 29. Java-based JPA configuration

```
@Bean
public Properties jpaProperties() {

    Properties properties = new Properties();
    properties.setProperty("eclipselink.logging.level.sql", "FINE");
    return properties;
}
```

JAVA

If you have a `persistence.xml` file, you can apply it there:

Example 30. persistence.xml-based configuration

```
<persistence-unit name="my-persistence-unit">
```

XML

```
...registered classes...

<properties>
  <property name="eclipselink.logging.level.sql" value="FINE" />
</properties>
</persistence-unit>
```

Finally, if you are using Spring Boot, then you can set it up inside your `application.properties` file:

Example 31. Spring Boot property-based configuration

```
spring.jpa.properties.eclipselink.logging.level.sql=FINE
```

Configuring Fetch- and LoadGraphs

The JPA 2.1 specification introduced support for specifying Fetch- and LoadGraphs that we also support with the `@EntityGraph` annotation, which lets you reference a `@NamedEntityGraph` definition. You can use that annotation on an entity to configure the fetch plan of the resulting query. The type (Fetch or Load) of the fetching can be configured by using the `type` attribute on the `@EntityGraph` annotation. See the JPA 2.1 Spec 3.7.4 for further reference.

The following example shows how to define a named entity graph on an entity:

Example 32. Defining a named entity graph on an entity.

```
@Entity
@NamedEntityGraph(name = "GroupInfo.detail",
    attributeNodes = @NamedAttributeNode("members"))
public class GroupInfo {

    // default fetch mode is lazy.
    @ManyToMany
    List<GroupMember> members = new ArrayList<GroupMember>();

    ...
}
```

JAVA

The following example shows how to reference a named entity graph on a repository query method:

Example 33. Referencing a named entity graph definition on a repository query method.

```
public interface GroupRepository extends CrudRepository<GroupInfo, String> {

    @EntityGraph(value = "GroupInfo.detail", type = EntityGraphType.LOAD)
    GroupInfo getByName(String name);

}
```

JAVA

It is also possible to define ad hoc entity graphs by using `@EntityGraph`. The provided `attributePaths` are translated into the according `EntityGraph` without needing to explicitly add `@NamedEntityGraph` to your domain types, as shown in the following example:

Example 34. Using ad-hoc entity graph definitions on a repository query method

```
public interface GroupRepository extends CrudRepository<GroupInfo, String> {

    @EntityGraph(attributePaths = { "members" })
    GroupInfo getByName(String name);

}
```

JAVA

Scrolling

Scrolling is a more fine-grained approach to iterate through larger results set chunks. Scrolling consists of a stable sort, a scroll type (Offset- or Keyset-based scrolling) and result limiting. You can define simple sorting expressions by using property names and define static result limiting using the [Top or First keyword](#) through query derivation. You can concatenate expressions to collect multiple criteria into one expression.

Scroll queries return a `Window<T>` that allows obtaining the element's scroll position to fetch the next `Window<T>` until your application has consumed the entire query result. Similar to consuming a Java `Iterator<List<...>>` by obtaining the next batch of results, query result scrolling lets you access a `ScrollPosition` through `Window.positionAt(...)`, as in the following example:

```
Window<User> users = repository.findFirst10ByLastnameOrderByFirstname("Doe",
    ScrollPosition.offset());
do {

    for (User u : users) {
        // consume the user
    }

}
```

JAVA

```

    if (users.isLast() || users.isEmpty()) {
        break;
    }

    // obtain the next Scroll
    users = repository.findFirst10ByLastnameOrderByFirstname("Doe",
users.positionAt(users.size() - 1));
} while (!users.isEmpty());

```

| NOTE

The `ScrollPosition` identifies the exact position of an element with the entire query result. Query execution treats the position parameter *exclusive*, results will start *after* the given position. `ScrollPosition#offset()` and `ScrollPosition#keyset()` as special incarnations of a `ScrollPosition` indicating the start of a scroll operation.

| NOTE

The above example shows static sorting and limiting. You can define query methods alternatively that accept a `Sort` object define a more complex sorting order or sorting on a per-request basis. In a similar way, providing a `Limit` object allows you to define a dynamic limit on a per-request basis instead of applying a static limitation. Read more on dynamic sorting and limiting in the [Query Methods Details](#).

Scrolling through consuming `Window` instances requires quite a few conditionals to reach optimum database round-trips and can quickly become a repetitive task that can be simplified using `WindowIterator`.

`WindowIterator` provides a utility to simplify scrolling across `Window`s by removing the need to check for the presence of a next `Window` and applying the `ScrollPosition`.

```

WindowIterator<User> users = WindowIterator.of(position ->
repository.findFirst10ByLastnameOrderByFirstname("Doe", position))
    .startingAt(ScrollPosition.offset());

while (users.hasNext()) {
    User u = users.next();
    // consume the user
}

```

JAVA

Scrolling using Offset

Offset scrolling uses similar to pagination, an `Offset` counter to skip a number of results and let the data source only return results beginning at the given `Offset`. This simple mechanism avoids large results be-

ing sent to the client application. However, most databases require materializing the full query result before your server can return the results.

Example 35. Using OffsetScrollPosition with Repository Query Methods

```
interface UserRepository extends Repository<User, Long> {  
  
    Window<User> findFirst10ByLastnameOrderByFirstname(String lastname,  
    OffsetScrollPosition position);  
}  
  
WindowIterator<User> users = WindowIterator.of(position ->  
    repository.findFirst10ByLastnameOrderByFirstname("Doe", position))  
    .startingAt(OffsetScrollPosition.initial()); 1
```

JAVA

1 Start with no offset to include the element at position 0.

CAUTION

There is a difference between `ScrollPosition.offset()` and `ScrollPosition.offset(0L)`. The former indicates the start of scroll operation, pointing to no specific offset whereas the latter identifies the first element (at position 0) of the result. Given the *exclusive* nature of scrolling, using `ScrollPosition.offset(0)` skips the first element and translate to an offset of 1.

Scrolling using Keyset-Filtering

Offset-based requires most databases require materializing the entire result before your server can return the results. So while the client only sees the portion of the requested results, your server needs to build the full result, which causes additional load.

Keyset-Filtering approaches result subset retrieval by leveraging built-in capabilities of your database aiming to reduce the computation and I/O requirements for individual queries. This approach maintains a set of keys to resume scrolling by passing keys into the query, effectively amending your filter criteria.

The core idea of Keyset-Filtering is to start retrieving results using a stable sorting order. Once you want to scroll to the next chunk, you obtain a `ScrollPosition` that is used to reconstruct the position within the sorted result. The `ScrollPosition` captures the keyset of the last entity within the current `Window`. To run the query, reconstruction rewrites the criteria clause to include all sort fields and the primary key so that the database can leverage potential indexes to run the query. The database needs only constructing a much smaller result from the given keyset position without the need to fully materialize a large result and then skipping results until reaching a particular offset.

WARNING

Keyset-Filtering requires the keyset properties (those used for sorting) to be non-nullable. This limitation applies due to the store specific `null` value handling of comparison operators as well as the need to run queries against an indexed source. Keyset-Filtering on nullable properties will lead to unexpected results.

Using `KeysetScrollPosition` with Repository Query Methods

JAVA

```
interface UserRepository extends Repository<User, Long> {

    Window<User> findFirst10ByLastnameOrderByFirstname(String lastname, KeysetScrollPosition
position);
}

WindowIterator<User> users = WindowIterator.of(position ->
repository.findFirst10ByLastnameOrderByFirstname("Doe", position))
    .startingAt(ScrollPosition.keyset()); 1
```

1 Start at the very beginning and do not apply additional filtering.

Keyset-Filtering works best when your database contains an index that matches the sort fields, hence a static sort works well. Scroll queries applying Keyset-Filtering require to the properties used in the sort order to be returned by the query, and these must be mapped in the returned entity.

You can use interface and DTO projections, however make sure to include all properties that you've sorted by to avoid keyset extraction failures.

When specifying your `Sort` order, it is sufficient to include sort properties relevant to your query; You do not need to ensure unique query results if you do not want to. The keyset query mechanism amends your sort order by including the primary key (or any remainder of composite primary keys) to ensure each query result is unique.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or

registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Value Expressions Fundamentals

Value Expressions Fundamentals

Scope

Expression Syntax

Parsing and Evaluation

SpEL Expressions

Extending the Evaluation Context

Property Placeholders

Value Expressions are a combination of [Spring Expression Language \(SpEL\)](#) and [Property Placeholder Resolution](#). They combine powerful evaluation of programmatic expressions with the simplicity to resort to property-placeholder resolution to obtain values from the `Environment` such as configuration properties.

Expressions are expected to be defined by a trusted input such as an annotation value and not to be determined from user input.

Scope

Value Expressions are used in contexts across annotations. Spring Data offers Value Expression evaluation in two main contexts:

- **Mapping Model Annotations:** such as `@Document`, `@Field`, `@Value` and other annotations in Spring Data modules that ship with their own mapping models respective Entity Readers such as MongoDB, Elasticsearch, Cassandra, Neo4j. Modules that build on libraries providing their own mapping models (JPA, LDAP) do not support Value Expressions in mapping annotations.

The following code demonstrates how to use expressions in the context of mapping model annotations.

Example 1. @Document Annotation Usage

```
@Document("orders-#{tenantService.getOrderCollection()}-${tenant-config.suffix}")  
class Order {
```

JAVA

```
// ...  
}
```

- **Repository Query Methods:** primarily through `@Query` .

The following code demonstrates how to use expressions in the context of repository query methods.

Example 2. @Query Annotation Usage

```
class OrderRepository extends Repository<Order, String> {  
  
    @Query("select u from User u where u.tenant = ?#{spring.application.name:unknown}  
and u.firstname like ?#{escape([0])}% escape ?#{escapeCharacter()}")  
    List<Order> findContainingEscaped(String namePart);  
}
```

JAVA

NOTE

Consult your module's documentation to determine the actual parameter by-name/by-index binding syntax. Typically, expressions are prefixed with `#{...}` / `:${...}` or `?#{...}` / `?${...}` .

Expression Syntax

Value Expressions can be defined from a sole SpEL Expression, a Property Placeholder or a composite expression mixing various expressions including literals.

Example 3. Expression Examples

<code>#{tenantService.getOrderCollection()}</code>	1
<code>#{(1+1) + '-hello-world'}</code>	2
<code>\${tenant-config.suffix}</code>	3
<code>orders-\${tenant-config.suffix}</code>	4
<code>#{tenantService.getOrderCollection()}-\${tenant-config.suffix}</code>	5

- 1 Value Expression using a single SpEL Expression.
- 2 Value Expression using a static SpEL Expression evaluating to `2-hello-world` .
- 3 Value Expression using a single Property Placeholder.
- 4 Composite expression comprised of the literal `orders-` and the Property Placeholder `${tenant-config.suffix}` .
- 5 Composite expression using SpEL, Property Placeholders and literals.

NOTE

NOTE

Using value expressions introduces a lot of flexibility to your code. Doing so requires evaluation of the expression on each usage and, therefore, value expression evaluation has an impact on the performance profile.

[Spring Expression Language \(SpEL\)](#) and [Property Placeholder Resolution](#) explain the syntax and capabilities of SpEL and Property Placeholders in detail.

Parsing and Evaluation

Value Expressions are parsed by the `ValueExpressionParser` API. Instances of `ValueExpression` are thread-safe and can be cached for later use to avoid repeated parsing.

The following example shows the Value Expression API usage:

Parsing and Evaluation

Java

Kotlin

JAVA

```
ValueParserConfiguration configuration = SpelExpressionParser::new;
ValueEvaluationContext context = ValueEvaluationContext.of(environment,
evaluationContext);

ValueExpressionParser parser = ValueExpressionParser.create(configuration);
ValueExpression expression = parser.parse("Hello, World");
Object result = expression.evaluate(context);
```

SpEL Expressions

[SpEL Expressions](#) follow the Template style where the expression is expected to be enclosed within the `# {...}` format. Expressions are evaluated using an `EvaluationContext` that is provided by `EvaluationContextProvider`. The context itself is a powerful `StandardEvaluationContext` allowing a wide range of operations, access to static types and context extensions.

NOTE

Make sure to parse and evaluate only expressions from trusted sources such as annotations. Accepting user-provided expressions can create an entry path to exploit the application context and your system resulting in a potential security vulnerability.

Extending the Evaluation Context

`EvaluationContextProvider` and its reactive variant `ReactiveEvaluationContextProvider` provide access to an `EvaluationContext`. `ExtensionAwareEvaluationContextProvider` and its reactive vari-

ant `ReactiveExtensionAwareEvaluationContextProvider` are default implementations that determine context extensions from an application context, specifically `ListableBeanFactory`.

Extensions implement either `EvaluationContextExtension` or `ReactiveEvaluationContextExtension` to provide extension support to hydrate `EvaluationContext`. That are a root object, properties and functions (top-level methods).

The following example shows a context extension that provides a root object, properties, functions and an aliased function.

Implementing a `EvaluationContextExtension`

Java

Kotlin

JAVA

```
@Component
public class MyExtension implements EvaluationContextExtension {

    @Override
    public String getExtensionId() {
        return "my-extension";
    }

    @Override
    public Object getRootObject() {
        return new CustomExtensionRootObject();
    }

    @Override
    public Map<String, Object> getProperties() {

        Map<String, Object> properties = new HashMap<>();

        properties.put("key", "Hello");

        return properties;
    }

    @Override
    public Map<String, Function> getFunctions() {

        Map<String, Function> functions = new HashMap<>();

        try {
            functions.put("aliasedMethod", new
Function(getClass().getMethod("extensionMethod")));
            return functions;
        } catch (Exception o_0) {
```

```

        throw new RuntimeException(o_0);
    }
}

public static String extensionMethod() {
    return "Hello World";
}

public static int add(int i1, int i2) {
    return i1 + i2;
}
}

public class CustomExtensionRootObject {

    public boolean rootObjectInstanceMethod() {
        return true;
    }

}

```

Once the above shown extension is registered, you can use its exported methods, properties and root object to evaluate SpEL expressions:

Example 4. Expression Evaluation Examples

<code>#{add(1, 2)}</code>	1
<code>#{extensionMethod()}</code>	2
<code>#{aliasedMethod()}</code>	3
<code>#{key}</code>	4
<code>#{rootObjectInstanceMethod()}</code>	5

- 1 Invoke the method `add` declared by `MyExtension` resulting in `3` as the method adds both numeric parameters and returns the sum.
- 2 Invoke the method `extensionMethod` declared by `MyExtension` resulting in `Hello World`.
- 3 Invoke the method `aliasedMethod`. The method is exposed as function and redirects into the method `extensionMethod` declared by `MyExtension` resulting in `Hello World`.
- 4 Evaluate the `key` property resulting in `Hello`.
- 5 Invoke the method `rootObjectInstanceMethod` on the root object instance `CustomExtensionRootObject`.

You can find real-life context extensions at [SecurityEvaluationContextExtension](#).

Property Placeholders

Property placeholders following the form `${...}` refer to properties provided typically by a `PropertySource` through `Environment`. Properties are useful to resolve against system properties, application configuration files, environment configuration or property sources contributed by secret management systems. You can find more details on the property placeholders in [Spring Framework's documentation on @Value usage](#).



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Projections

Projections

Introduction

Interface-based Projections

Class-based Projections (DTOs)

Using Projections with JPA

Derived queries

String-based queries

Introduction

Spring Data query methods usually return one or multiple instances of the aggregate root managed by the repository. However, it might sometimes be desirable to create projections based on certain attributes of those types. Spring Data allows modeling dedicated return types, to more selectively retrieve partial views of the managed aggregates.

Imagine a repository and aggregate root type such as the following example:

A sample aggregate and repository

```
class Person {  
  
    @Id UUID id;  
    String firstname, lastname;  
    Address address;  
  
    static class Address {  
        String zipCode, city, street;  
    }  
}  
  
interface PersonRepository extends Repository<Person, UUID> {  
  
    Collection<Person> findByLastname(String lastname);  
}
```

JAVA

Now imagine that we want to retrieve the person's name attributes only. What means does Spring Data offer to achieve this? The rest of this chapter answers that question.

| NOTE

Projection types are types residing outside the entity's type hierarchy. Superclasses and interfaces implemented by the entity are inside the type hierarchy hence returning a supertype (or implemented interface) returns an instance of the fully materialized entity.

Interface-based Projections

The easiest way to limit the result of the queries to only the name attributes is by declaring an interface that exposes accessor methods for the properties to be read, as shown in the following example:

A projection interface to retrieve a subset of attributes

```
interface NamesOnly {  
  
    String getFirstname();  
    String getLastname();  
}
```

JAVA

The important bit here is that the properties defined here exactly match properties in the aggregate root. Doing so lets a query method be added as follows:

A repository using an interface based projection with a query method

```
interface PersonRepository extends Repository<Person, UUID> {  
  
    Collection<NamesOnly> findByLastname(String lastname);  
}
```

JAVA

The query execution engine creates proxy instances of that interface at runtime for each element returned and forwards calls to the exposed methods to the target object.

| NOTE

Declaring a method in your `Repository` that overrides a base method (e.g. declared in `CrudRepository`, a store-specific repository interface, or the `Simple...Repository`) results in a call to the base method regardless of the declared return type. Make sure to use a compatible return type as base methods cannot be used for projections. Some store modules support `@Query` annotations to turn an overridden base method into a query method that then can be used to return projections.

Projections can be used recursively. If you want to include some of the `Address` information as well, create a projection interface for that and return that interface from the declaration of `getAddress()`, as shown in the following example:

A projection interface to retrieve a subset of attributes

JAVA

```
interface PersonSummary {

    String getFirstname();
    String getLastName();
    AddressSummary getAddress();

    interface AddressSummary {
        String getCity();
    }
}
```

On method invocation, the `address` property of the target instance is obtained and wrapped into a projecting proxy in turn.

Closed Projections

A projection interface whose accessor methods all match properties of the target aggregate is considered to be a closed projection. The following example (which we used earlier in this chapter, too) is a closed projection:

A closed projection

JAVA

```
interface NamesOnly {

    String getFirstname();
    String getLastName();
}
```

If you use a closed projection, Spring Data can optimize the query execution, because we know about all the attributes that are needed to back the projection proxy. For more details on that, see the module-specific part of the reference documentation.

Open Projections

Accessor methods in projection interfaces can also be used to compute new values by using the `@Value` annotation, as shown in the following example:

An Open Projection

JAVA

```
interface NamesOnly {

    @Value("#{target.firstname + ' ' + target.lastname}")
    String getFullName();
}
```

```
...  
}
```

The aggregate root backing the projection is available in the `target` variable. A projection interface using `@Value` is an open projection. Spring Data cannot apply query execution optimizations in this case, because the SpEL expression could use any attribute of the aggregate root.

The expressions used in `@Value` should not be too complex — you want to avoid programming in String variables. For very simple expressions, one option might be to resort to default methods (introduced in Java 8), as shown in the following example:

A projection interface using a default method for custom logic

```
interface NamesOnly {  
  
    String getFirstname();  
    String getLastName();  
  
    default String getFullName() {  
        return getFirstname().concat(" ").concat(getLastName());  
    }  
}
```

JAVA

This approach requires you to be able to implement logic purely based on the other accessor methods exposed on the projection interface. A second, more flexible, option is to implement the custom logic in a Spring bean and then invoke that from the SpEL expression, as shown in the following example:

Sample Person object

```
@Component  
class MyBean {  
  
    String getFullName(Person person) {  
        ...  
    }  
}  
  
interface NamesOnly {  
  
    @Value("#{@myBean.getFullName(target)}")  
    String getFullName();  
  
    ...  
}
```

JAVA

Notice how the SpEL expression refers to `myBean` and invokes the `getFullName(...)` method and forwards the projection target as a method parameter. Methods backed by SpEL expression evaluation can also use method parameters, which can then be referred to from the expression. The method parameters are available through an `Object` array named `args`. The following example shows how to get a method parameter from the `args` array:

Sample Person object

```
interface NamesOnly {

    @Value("#{args[0] + ' ' + target.firstname + '!'}")
    String getSalutation(String prefix);
}
```

JAVA

Again, for more complex expressions, you should use a Spring bean and let the expression invoke a method, as described earlier.

Nullable Wrappers

Getters in projection interfaces can make use of nullable wrappers for improved null-safety. Currently supported wrapper types are:

- `java.util.Optional`
- `com.google.common.base.Optional`
- `scala.Option`
- `io.vavr.control.Option`

A projection interface using nullable wrappers

```
interface NamesOnly {

    Optional<String> getFirstname();
}
```

JAVA

If the underlying projection value is not `null`, then values are returned using the present-representation of the wrapper type. In case the backing value is `null`, then the getter method returns the empty representation of the used wrapper type.

Class-based Projections (DTOs)

Another way of defining projections is by using value type DTOs (Data Transfer Objects) that hold properties for the fields that are supposed to be retrieved. These DTO types can be used in exactly the same

way projection interfaces are used, except that no proxying happens and no nested projections can be applied.

If the store optimizes the query execution by limiting the fields to be loaded, the fields to be loaded are determined from the parameter names of the constructor that is exposed.

The following example shows a projecting DTO:

A projecting DTO

```
record NamesOnly(String firstname, String lastname) {  
}
```

JAVA

Java Records are ideal to define DTO types since they adhere to value semantics: All fields are `private final` and `equals(...)` / `hashCode()` / `toString()` methods are created automatically. Alternatively, you can use any class that defines the properties you want to project.

Dynamic Projections

So far, we have used the projection type as the return type or element type of a collection. However, you might want to select the type to be used at invocation time (which makes it dynamic). To apply dynamic projections, use a query method such as the one shown in the following example:

A repository using a dynamic projection parameter

```
interface PersonRepository extends Repository<Person, UUID> {  
  
    <T> Collection<T> findByLastname(String lastname, Class<T> type);  
}
```

JAVA

This way, the method can be used to obtain the aggregates as is or with a projection applied, as shown in the following example:

Using a repository with dynamic projections

```
void someMethod(PersonRepository people) {  
  
    Collection<Person> aggregates =  
        people.findByLastname("Matthews", Person.class);  
  
    Collection<NamesOnly> aggregates =  
        people.findByLastname("Matthews", NamesOnly.class);  
}
```

JAVA

NOTE

Query parameters of type `Class` are inspected whether they qualify as dynamic projection parameter. If the actual return type of the query equals the generic parameter type of the `Class` parameter, then the matching `Class` parameter is not available for usage within the query or SpEL expressions. If you want to use a `Class` parameter as query argument then make sure to use a different generic parameter, for example `Class<?>`.

NOTE

When using Class-based projection, types must declare a single constructor so that Spring Data can determine their input properties. If your class defines more than one constructor, then you cannot use the type without further hints for DTO projections. In such a case annotate the desired constructor with `@PersistenceCreator` as outlined below so that Spring Data can determine which properties to select:

```
public class NamesOnly {

    private final String firstname;
    private final String lastname;

    protected NamesOnly() { }

    @PersistenceCreator
    public NamesOnly(String firstname, String lastname) {
        this.firstname = firstname;
        this.lastname = lastname;
    }

    // ...
}
```

Using Projections with JPA

You can use Projections with JPA in several ways. Depending on the technique and query type, you need to apply specific considerations.

Spring Data JPA uses generally `Tuple` queries to construct interface proxies for Interface-based Projections.

Derived queries

Query derivation supports both, class-based and interface projections by introspecting the returned type. Class-based projections use JPA's instantiation mechanism (constructor expressions) to create the projection instance.

Projections limit the selection to top-level properties of the target entity. Any nested properties resolving to joins select the entire nested property causing the full join to materialize.

String-based queries

Support for string-based queries covers both JPQL queries (`@Query`) and native queries (`@NativeQuery`).

JPQL Queries

JPA's mechanism to return Class-based projections using JPQL is **constructor expressions**. Therefore, your query must define a constructor expression such as `SELECT new com.example.NamesOnly(u.firstname, u.lastname) from User u`. (Note the usage of a FQDN for the DTO type!) This JPQL expression can be used in `@Query` annotations as well where you define any named queries. As a workaround you may use named queries with `ResultSetMapping` or the Hibernate-specific [ResultListTransformer](#).

Spring Data JPA can aid with rewriting your query to a constructor expression if your query selects the primary entity or a list of select items.

DTO Projection JPQL Query Rewriting

JPQL queries allow selection of the root object, individual properties, and DTO objects through constructor expressions. Using a constructor expression can quickly add a lot of text to a query and make it difficult to read the actual query. Spring Data JPA can support you with your JPQL queries by introducing constructor expressions for your convenience.

Consider the following queries:

Example 1. Projection Queries

```
interface UserRepository extends Repository<User, Long> {  
  
    @Query("SELECT u FROM USER u WHERE u.lastname = :lastname")  
    List<UserDto> findByLastname(String lastname);  
  
    @Query("SELECT u.firstname, u.lastname FROM USER u WHERE u.lastname = :lastname")  
    List<UserDto> findMultipleColumnsByLastname(String lastname);  
}  
  
record UserDto(String firstname, String lastname){}
```

- 1 Selection of the top-level entity. This query gets rewritten to `SELECT new UserDto(u.firstname, u.lastname) FROM USER u WHERE u.lastname = :lastname`.
- 2 Multi-select of `firstname` and `lastname` properties. This query gets rewritten to `SELECT new UserDto(u.firstname, u.lastname) FROM USER u WHERE u.lastname = :lastname`.

WARNING

JPQL constructor expressions must not contain aliases for selected columns and query rewriting will not remove them for you. While `SELECT u as user, count(u.roles) as roleCount FROM USER u ...` is a valid query for interface-based projections that rely on column names from the returned `Tuple`, the same construct is invalid when requesting a DTO where it needs to be `SELECT u, count(u.roles) FROM USER u ...`.

Some persistence providers may be lenient about this, others not.

Repository query methods that return a DTO projection type (a Java type outside the domain type hierarchy) are subject for query rewriting. If an `@Query`-annotated query already uses constructor expressions, then Spring Data backs off and doesn't apply DTO constructor expression rewriting.

Make sure that your DTO types provide an all-args constructor for the projection, otherwise the query will fail.

Native Queries

When using Class-based projections, their usage requires slightly more consideration depending on your :

- If properties of the result type map directly to the result (the order of columns and their types match the constructor arguments), then you can declare the query result type as the DTO type without further hints (or use the DTO class through dynamic projections).
- If the properties do not match or require transformation, use `@SqlResultSetMapping` through JPA's annotations map the result set to the DTO and provide the result mapping name through `@NativeQuery(resultSetMapping = "...")`.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Stored Procedures

The JPA 2.1 specification introduced support for calling stored procedures by using the JPA criteria query API. We introduced the `@Procedure` annotation for declaring stored procedure metadata on a repository method.

The examples to follow use the following stored procedure:

Example 1. The definition of the `plus1inout` procedure in HSQL DB.

```
;/
DROP procedure IF EXISTS plus1inout
;/
CREATE procedure plus1inout (IN arg int, OUT res int)
BEGIN ATOMIC
    set res = arg + 1;
END
;/
```

SQL

Metadata for stored procedures can be configured by using the `NamedStoredProcedureQuery` annotation on an entity type.

Example 2. `StoredProcedure` metadata definitions on an entity.

```
@Entity
@NamedStoredProcedureQuery(name = "User.plus1", procedureName = "plus1inout",
parameters = {
    @StoredProcedureParameter(mode = ParameterMode.IN, name = "arg", type =
Integer.class),
    @StoredProcedureParameter(mode = ParameterMode.OUT, name = "res", type =
Integer.class) })
public class User {}
```

JAVA

Note that `@NamedStoredProcedureQuery` has two different names for the stored procedure. `name` is the name JPA uses. `procedureName` is the name the stored procedure has in the database.

You can reference stored procedures from a repository method in multiple ways. The stored procedure to be called can either be defined directly by using the `value` or `procedureName` attribute of the `@Procedure` annotation. This refers directly to the stored procedure in the database and ignores any configuration via `@NamedStoredProcedureQuery`.

Alternatively you may specify the `@NamedStoredProcedureQuery.name` attribute as the `@Procedure.name` attribute. If neither `value`, `procedureName` nor `name` is configured, the name of the repository method is used as the `name` attribute.

The following example shows how to reference an explicitly mapped procedure:

Example 3. Referencing explicitly mapped procedure with name "plus1inout" in database.

```
@Procedure("plus1inout")
Integer explicitlyNamedPlus1inout(Integer arg);
```

JAVA

The following example is equivalent to the previous one but uses the `procedureName` alias:

Example 4. Referencing implicitly mapped procedure with name "plus1inout" in database via procedureName alias.

```
@Procedure(procedureName = "plus1inout")
Integer callPlus1InOut(Integer arg);
```

JAVA

The following is again equivalent to the previous two but using the method name instead of an explicit annotation attribute.

Example 5. Referencing implicitly mapped named stored procedure "User.plus1" in EntityManager by using the method name.

```
@Procedure
Integer plus1inout(@Param("arg") Integer arg);
```

JAVA

The following example shows how to reference a stored procedure by referencing the `@NamedStoredProcedureQuery.name` attribute.

Example 6. Referencing explicitly mapped named stored procedure "User.plus1IO" in EntityManager.

```
@Procedure(name = "User.plus1IO")
Integer entityAnnotatedCustomNamedProcedurePlus1IO(@Param("arg") Integer arg);
```

JAVA

If the stored procedure getting called has a single out parameter that parameter may be returned as the return value of the method. If there are multiple out parameters specified in a `@NamedStoredProcedureQuery` annotation those can be returned as a `Map` with the key being the parameter name given in the `@NamedStoredProcedureQuery` annotation.

NOTE

Note that if the stored procedure returns a `ResultSet` then any `OUT` parameters are omitted as Java can only return a single method return value unless the method declares a `Map` return type.

The following example shows how to obtain multiple `OUT` parameters if the stored procedure has multiple `OUT` parameters and is registered as `@NamedStoredProcedureQuery`.

`@NamedStoredProcedureQuery` registration is required to provide parameter metadata.

Example 7. StoredProcedure metadata definitions on an entity.

```
@Entity
@NamedStoredProcedureQuery(name = "User.multiple_out_parameters", procedureName =
"multiple_out_parameters", parameters = {
    @StoredProcedureParameter(mode = ParameterMode.IN, name = "arg", type =
Integer.class),
    @StoredProcedureParameter(mode = ParameterMode.REF_CURSOR, name = "some_cursor", type
= void.class),
    @StoredProcedureParameter(mode = ParameterMode.OUT, name = "res", type =
Integer.class) })
public class User {}
```

JAVA

Example 8. Returning multiple OUT parameters

```
@Procedure(name = "User.multiple_out_parameters")
Map<String, Object> returnsMultipleOutParameters(@Param("arg") Integer arg);
```

JAVA



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the

United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Specifications

Specifications

PredicateSpecification

Specification, UpdateSpecification, DeleteSpecification

Fluent API

JPA's Criteria API lets you build queries programmatically. Spring Data JPA Specification provides a small, focused API to express predicates over entities and reuse them across repositories. Based on the concept of a specification from Eric Evans' book "Domain Driven Design", specifications follow the same semantics providing an API to define criteria using JPA. To support specifications, you can extend your repository interface with the `JpaSpecificationExecutor` interface, as follows:

```
public interface CustomerRepository extends CrudRepository<Customer, Long>,
    JpaSpecificationExecutor<Customer> {
}
```

JAVA

A specification is a predicate over an entity expressed with the Criteria API. Spring Data JPA offers two entry points:

- [PredicateSpecification](#): A flexible, query-type-agnostic interface introduced with Spring Data JPA 4.0.
- [Specification](#) (and [UpdateSpecification](#), [DeleteSpecification](#)): Query-bound variants.

PredicateSpecification

The `PredicateSpecification` interface is defined with a minimal set of dependencies allowing broad functional composition:

```
public interface PredicateSpecification<T> {
    Predicate toPredicate(From<?, T> from, CriteriaBuilder builder);
}
```

JAVA

Specifications can easily be used to build an extensible set of predicates and used with `JpaRepository` removing the need to declare a query (method) for every needed combination as shown in the following example:

Example 1. Specifications for a Customer

```
class CustomerSpecs {

    static PredicateSpecification<Customer> isLongTermCustomer() {
        return (from, builder) -> {
            LocalDate date = LocalDate.now().minusYears(2);
            return builder.lessThan(from.get(Customer_.createdAt), date);
        };
    }

    static PredicateSpecification<Customer> hasSalesOfMoreThan(MonetaryAmount value) {
        return (from, builder) -> {
            // build predicate for sales > value
        };
    }
}
```

JAVA

The `Customer_` type is a metamodel type generated using the JPA Metamodel generator (see the [Hibernate implementation's documentation for an example](#)). So the expression, `Customer_.createdAt`, assumes the `Customer` has a `createdAt` attribute of type `Date`. Besides that, we have expressed some criteria on a business requirement abstraction level and created executable `Specifications`.

Use a specification directly with a repository:

```
List<Customer> customers = customerRepository.findAll(isLongTermCustomer());
```

JAVA

Specifications become most valuable when composed:

```
MonetaryAmount amount = new MonetaryAmount(200.0, Currencies.DOLLAR);
List<Customer> customers = customerRepository.findAll(
    isLongTermCustomer().or(hasSalesOfMoreThan(amount))
);
```

JAVA

Specification, UpdateSpecification, DeleteSpecification

The [Specification](#) interface has been available for a much longer time and is tied a particular query type (select, update, delete) as per Criteria API restrictions. The three specification interfaces are defined as follows:

Specification	UpdateSpecification	DeleteSpecification
---------------	---------------------	---------------------

```
public interface Specification<T> {
    Predicate toPredicate(Root<T> root, CriteriaQuery<?> query,
        CriteriaBuilder builder);
}
```

JAVA

Specification objects can be constructed either directly or by reusing PredicateSpecification instances, as shown in the following example:

Example 2. Specifications for a Customer

```
public class CustomerSpecs {

    public static UpdateSpecification<Customer>
updateLastnameByFirstnameAndLastname(String newLastName, String currentFirstname,
String currentLastname) {
        return UpdateSpecification<User> updateLastname = UpdateSpecification.<User>
update((root, update, criteriaBuilder) -> {
            update.set("lastname", newLastName);
        }).where(hasFirstname(currentFirstname).and(hasLastname(currentLastname)));
    }

    public static PredicateSpecification<Customer> hasFirstname(String firstname) {
        return (root, builder) -> {
            return builder.equal(from.get("firstname"), value);
        };
    }

    public static PredicateSpecification<Customer> hasLastname(String lastname) {
        return (root, builder) -> {
            // build query here
        };
    }
}
```

JAVA

Fluent API

`JpaSpecificationExecutor` defines fluent query methods for flexible execution of queries based on `Specification` instances:

1. For `PredicateSpecification`: `findBy(PredicateSpecification<T> spec, Function<? super SpecificationFluentQuery<S>, R> queryFunction)`
2. For `Specification`: `findBy(Specification<T> spec, Function<? super SpecificationFluentQuery<S>, R> queryFunction)`

As with other methods, it executes a query derived from a `Specification`. However, the query function allows you to take control over aspects of query execution that you cannot dynamically control otherwise. You do so by invoking the various intermediate and terminal methods of `SpecificationFluentQuery`.

Intermediate methods

- `sortBy`: Apply an ordering for your result. Repeated method calls append each `Sort` (note that `page(Pageable)` using a sorted `Pageable` overrides any previous sort order).
- `limit`: Limit the result count.
- `as`: Specify the type to be read or projected to.
- `project`: Limit the queries properties.

Terminal methods

- `first`, `firstValue`: Return the first value. `first` returns an `Optional<T>` or `Optional.empty()` if the query did not yield any result. `firstValue` is its nullable variant without the need to use `Optional`.
- `one`, `oneValue`: Return the one value. `one` returns an `Optional<T>` or `Optional.empty()` if the query did not yield any result. `oneValue` is its nullable variant without the need to use `Optional`. Throws `IncorrectResultSizeDataAccessException` if more than one match found.
- `all`: Return all results as a `List<T>`.
- `page(Pageable)`: Return all results as a `Page<T>`.
- `slice(Pageable)`: Return all results as a `Slice<T>`.
- `scroll(ScrollPosition)`: Use scrolling (offset, keyset) to retrieve results as a `Window<T>`.
- `stream()`: Return a `Stream<T>` to process results lazily. The stream is stateful and must be closed after use.
- `count` and `exists`: Return the count of matching entities or whether any match exists.

Intermediate and terminal methods must be invoked within the query function.

Example 3. Use the fluent API to get a projected Page, ordered by Lastname

```
Page<CustomerProjection> page = repository.findBy(spec,  
    q -> q.as(CustomerProjection.class)  
        .page(PageRequest.of(0, 20, Sort.by("lastname"))))  
);
```

JAVA

Example 4. Use the fluent API to get the last of potentially many results, ordered by Lastname

```
Optional<Customer> match = repository.findBy(spec,  
    q -> q.sortBy(Sort.by("lastname").descending())  
        .first()  
);
```

JAVA



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Query by Example

Query by Example

Introduction

Usage

Example Matchers

Fluent API

Introduction

This chapter provides an introduction to Query by Example and explains how to use it.

Query by Example (QBE) is a user-friendly querying technique with a simple interface. It allows dynamic query creation and does not require you to write queries that contain field names. In fact, Query by Example does not require you to write queries by using store-specific query languages at all.

NOTE

This chapter explains the core concepts of Query by Example. The information is pulled from the Spring Data Commons module. Depending on your database, String matching support can be limited.

Usage

The Query by Example API consists of four parts:

- **Probe**: The actual example of a domain object with populated fields.
- **ExampleMatcher**: The `ExampleMatcher` carries details on how to match particular fields. It can be reused across multiple Examples.
- **Example**: An `Example` consists of the probe and the `ExampleMatcher`. It is used to create the query.
- **FetchableFluentQuery**: A `FetchableFluentQuery` offers a fluent API, that allows further customization of a query derived from an `Example`. Using the fluent API lets you specify ordering projection and result processing for your query.

Query by Example is well suited for several use cases:

- Querying your data store with a set of static or dynamic constraints.
- Frequent refactoring of the domain objects without worrying about breaking existing queries.
- Working independently of the underlying data store API.

Query by Example also has several limitations:

- No support for nested or grouped property constraints, such as `firstname = ?0` or `(firstname = ?1 and lastname = ?2)`.
- No support for matching collections or maps.
- Store-specific support on string matching. Depending on your databases, String matching can support starts/contains/ends/regex for strings.
- Exact matching for other property types.

Before getting started with Query by Example, you need to have a domain object. To get started, create an interface for your repository, as shown in the following example:

Sample Person object

```
public class Person {  
  
    @Id  
    private String id;  
    private String firstname;  
    private String lastname;  
    private Address address;  
  
    // ... getters and setters omitted  
}
```

JAVA

The preceding example shows a simple domain object. You can use it to create an `Example`. By default, fields having `null` values are ignored, and strings are matched by using the store specific defaults.

NOTE

Inclusion of properties into a Query by Example criteria is based on nullability. Properties using primitive types (`int`, `double`, ...) are always included unless the [ExampleMatcher](#) ignores the property path.

Examples can be built by either using the `of` factory method or by using [ExampleMatcher](#). `Example` is immutable. The following listing shows a simple Example:

Example 1. Simple Example

```

Person person = new Person();           1
person.setFirstname("Dave");           2

Example<Person> example = Example.of(person); 3

```

JAVA

- 1 Create a new instance of the domain object.
- 2 Set the properties to query.
- 3 Create the Example.

You can run the example queries by using repositories. To do so, let your repository interface extend `QueryByExampleExecutor<T>`. The following listing shows an excerpt from the `QueryByExampleExecutor` interface:

The QueryByExampleExecutor

```

public interface QueryByExampleExecutor<T> {

    <S extends T> S findOne(Example<S> example);

    <S extends T> Iterable<S> findAll(Example<S> example);

    // ... more functionality omitted.
}

```

JAVA

Example Matchers

Examples are not limited to default settings. You can specify your own defaults for string matching, null handling, and property-specific settings by using the `ExampleMatcher`, as shown in the following example:

Example 2. Example matcher with customized matching

```

Person person = new Person();           1
person.setFirstname("Dave");           2

ExampleMatcher matcher = ExampleMatcher.matching() 3
    .withIgnorePaths("lastname")           4
    .withIncludeNullValues()               5
    .withStringMatcher(StringMatcher.ENDING); 6

Example<Person> example = Example.of(person, matcher); 7

```

JAVA

- 1 Create a new instance of the domain object.
- 2 Set properties.
- 3 Create an `ExampleMatcher` to expect all values to match. It is usable at this stage even without further configuration.
- 4 Construct a new `ExampleMatcher` to ignore the `lastname` property path.
- 5 Construct a new `ExampleMatcher` to ignore the `lastname` property path and to include null values.
- 6 Construct a new `ExampleMatcher` to ignore the `lastname` property path, to include null values, and to perform suffix string matching.
- 7 Create a new `Example` based on the domain object and the configured `ExampleMatcher`.

By default, the `ExampleMatcher` expects all values set on the probe to match. If you want to get results matching any of the predicates defined implicitly, use `ExampleMatcher.matchingAny()`.

You can specify behavior for individual properties (such as "firstname" and "lastname" or, for nested properties, "address.city"). You can tune it with matching options and case sensitivity, as shown in the following example:

Configuring matcher options

```
ExampleMatcher matcher = ExampleMatcher.matching()
    .withMatcher("firstname", endsWith())
    .withMatcher("lastname", startsWith().ignoreCase());
}
```

JAVA

Another way to configure matcher options is to use lambdas (introduced in Java 8). This approach creates a callback that asks the implementor to modify the matcher. You need not return the matcher, because configuration options are held within the matcher instance. The following example shows a matcher that uses lambdas:

Configuring matcher options with lambdas

```
ExampleMatcher matcher = ExampleMatcher.matching()
    .withMatcher("firstname", match -> match.endsWith())
    .withMatcher("firstname", match -> match.startsWith());
}
```

JAVA

Queries created by `Example` use a merged view of the configuration. Default matching settings can be set at the `ExampleMatcher` level, while individual settings can be applied to particular property paths. Settings that are set on `ExampleMatcher` are inherited by property path settings unless they are defined explicitly. Settings on a property patch have higher precedence than default settings. The following table describes the scope of the various `ExampleMatcher` settings:

Table 1. Scope of `ExampleMatcher` settings

Setting	Scope
Null-handling	<code>ExampleMatcher</code>
String matching	<code>ExampleMatcher</code> and property path
Ignoring properties	Property path
Case sensitivity	<code>ExampleMatcher</code> and property path
Value transformation	Property path

Fluent API

`QueryByExampleExecutor` defines fluent query methods for flexible execution of queries based on `Example` instances through `findBy(Example<S> example, Function<FluentQuery.FetchableFluentQuery<S>, R> queryFunction)`.

As with other methods, it executes a query derived from a `Example`. However, the query function allows you to take control over aspects of query execution that you cannot dynamically control otherwise. You do so by invoking the various intermediate and terminal methods of `FetchableFluentQuery`.

Intermediate methods

- `sortBy`: Apply an ordering for your result. Repeated method calls append each `Sort` (note that `page(Pageable)` using a sorted `Pageable` overrides any previous sort order).
- `limit`: Limit the result count.
- `as`: Specify the type to be read or projected to.
- `project`: Limit the queries properties.

Terminal methods

- `first`, `firstValue`: Return the first value. `first` returns an `Optional<T>` or `Optional.empty()` if the query did not yield any result. `firstValue` is its nullable variant without the need to use `Optional`.
- `one`, `oneValue`: Return the one value. `one` returns an `Optional<T>` or `Optional.empty()` if the query did not yield any result. `oneValue` is its nullable variant without the need to use `Optional`. Throws `IncorrectResultSizeDataAccessException` if more than one match found.
- `all`: Return all results as a `List<T>`.

- `page(Pageable)` : Return all results as a `Page<T>` .
- `slice(Pageable)` : Return all results as a `Slice<T>` .
- `scroll(ScrollPosition)` : Use scrolling (offset, keyset) to retrieve results as a `Window<T>` .
- `stream()` : Return a `Stream<T>` to process results lazily. The stream is stateful and must be closed after use.
- `count` and `exists` : Return the count of matching entities or whether any match exists.

NOTE

Intermediate and terminal methods must be invoked within the query function.

Example 3. Use the fluent API to get a projected Page, ordered by Lastname

```
Page<CustomerProjection> page = repository.findBy(example,
    q -> q.as(CustomerProjection.class)
        .page(PageRequest.of(0, 20, Sort.by("lastname"))))
);
```

JAVA

Example 4. Use the fluent API to get the last of potentially many results, ordered by Lastname

```
Optional<Customer> match = repository.findBy(example,
    q -> q.sortBy(Sort.by("lastname").descending())
        .first())
);
```

JAVA

In Spring Data JPA, you can use Query by Example with Repositories, as shown in the following example:

Example 5. Query by Example using a Repository

```
public interface PersonRepository extends JpaRepository<Person, String> { ... }

public class PersonService {

    @Autowired PersonRepository personRepository;

    public List<Person> findPeople(Person probe) {
        return personRepository.findAll(Example.of(probe));
    }
}
```

JAVA

NOTE

NOTE

Currently, only `SingularAttribute` properties can be used for property matching.

The property specifier accepts property names (such as `firstname` and `lastname`). You can navigate by chaining properties together with dots (`address.city`). You can also tune it with matching options and case sensitivity.

The following table shows the various `StringMatcher` options that you can use and the result of using them on a field named `firstname` :

Table 2. *StringMatcher* options

Matching	Logical result
DEFAULT (case-sensitive)	<code>firstname = ?0</code>
DEFAULT (case-insensitive)	<code>LOWER(firstname) = LOWER(?0)</code>
EXACT (case-sensitive)	<code>firstname = ?0</code>
EXACT (case-insensitive)	<code>LOWER(firstname) = LOWER(?0)</code>
STARTING (case-sensitive)	<code>firstname like ?0 + '%'</code>
STARTING (case-insensitive)	<code>LOWER(firstname) like LOWER(?0) + '%'</code>
ENDING (case-sensitive)	<code>firstname like '%' + ?0</code>
ENDING (case-insensitive)	<code>LOWER(firstname) like '%' + LOWER(?0)</code>
CONTAINING (case-sensitive)	<code>firstname like '%' + ?0 + '%'</code>
CONTAINING (case-insensitive)	<code>LOWER(firstname) like '%' + LOWER(?0) + '%'</code>

NOTE

Regex-matching is not supported by JPA.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States

and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Vector Search

Vector Search

Vector Model

Vector

Search Results

Score, Similarity, and Scoring Functions

Vector Search Methods

Derived Search Methods

Annotated Search Methods

Sorting

With the rise of Generative AI, Vector databases have gained strong traction in the world of databases. These databases enable efficient storage and querying of high-dimensional vectors, making them well-suited for tasks such as semantic search, recommendation systems, and natural language understanding.

Vector search is a technique that retrieves semantically similar data by comparing vector representations (also known as embeddings) rather than relying on traditional exact-match queries. This approach enables intelligent, context-aware applications that go beyond keyword-based retrieval.

In the context of Spring Data, vector search opens new possibilities for building intelligent, context-aware applications, particularly in domains like natural language processing, recommendation systems, and generative AI. By modelling vector-based querying using familiar repository abstractions, Spring Data allows developers to seamlessly integrate similarity-based vector-capable databases with the simplicity and consistency of the Spring Data programming model.

To use Hibernate Vector Search, you need to add the following dependencies to your project.

The following example shows how to set up dependencies in Maven and Gradle:

Maven

Gradle

```
<dependencies>
  <dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-vector</artifactId>
```

XML

```
<version>${hibernate.version}</version>
</dependency>
</dependencies>
```

NOTE

While you can use `Vector` as type for queries, you cannot use it in your domain model as Hibernate requires float or double arrays as vector types.

Vector Model

To support vector search in a type-safe and idiomatic way, Spring Data introduces the following core abstractions:

- `Vector`
- `SearchResults<T>` and `SearchResult<T>`
- `Score`, `Similarity` and Scoring Functions

Vector

The `Vector` type represents an n-dimensional numerical embedding, typically produced by embedding models. In Spring Data, it is defined as a lightweight wrapper around an array of floating-point numbers, ensuring immutability and consistency. This type can be used as an input for search queries or as a property on a domain entity to store the associated vector representation.

```
Vector vector = Vector.of(0.23f, 0.11f, 0.77f);
```

JAVA

Using `Vector` in your domain model removes the need to work with raw arrays or lists of numbers, providing a more type-safe and expressive way to handle vector data. This abstraction also allows for easy integration with various vector databases and libraries. It also allows for implementing vendor-specific optimizations such as binary or quantized vectors that do not map to a standard floating point (`float` and `double` as of [IEEE 754](#)) representation. A domain object can have a vector property, which can be used for similarity searches. Consider the following example:

```
class Comment {

    @Id String id;
    String country;
    String comment;
```

JAVA

```

@Column(name = "the_embedding")
@JdbcTypeCode(SqlTypes.VECTOR)
@Array(length = 5)
Vector embedding;

// getters, setters, ...
}

```

NOTE

Associating a vector with a domain object results in the vector being loaded and stored as part of the entity life-cycle, which may introduce additional overhead on retrieval and persistence operations.

Search Results

The `SearchResult<T>` type encapsulates the results of a vector similarity query. It includes both the matched domain object and a relevance score that indicates how closely it matches the query vector. This abstraction provides a structured way to handle result ranking and enables developers to easily work with both the data and its contextual relevance.

Example 1. Using `SearchResult<T>` in a Repository Search Method

```

interface CommentRepository extends Repository<Comment, String> {

    SearchResults<Comment> searchByCountryAndEmbeddingNear(String country, Vector vector,
        Score distance,
        Limit limit);

    @Query("""
        SELECT c, cosine_distance(c.embedding, :embedding) as distance FROM Comment c
        WHERE c.country = ?1
        AND cosine_distance(c.embedding, :embedding) <= :distance
        ORDER BY distance asc""")
    SearchResults<Comment> searchAnnotatedByCountryAndEmbeddingWithin(String country,
        Vector embedding,
        Score distance);
}

SearchResults<Comment> results = repository.searchByCountryAndEmbeddingNear("en",
    Vector.of(...), Score.of(0.9), Limit.of(10));

```

JAVA

In this example, the `searchByCountryAndEmbeddingNear` method returns a `SearchResults<Comment>` object, which contains a list of `SearchResult<Comment>` instances. Each result includes the matched

`Comment` entity and its relevance score.

Relevance score is a numerical value that indicates how closely the matched vector aligns with the query vector. Depending on whether a score represents distance or similarity a higher score can mean a closer match or a more distant one.

The scoring function used to calculate this score can vary based on the underlying database, index or input parameters.

Score, Similarity, and Scoring Functions

The `Score` type holds a numerical value indicating the relevance of a search result. It can be used to rank results based on their similarity to the query vector. The `Score` type is typically a floating-point number, and its interpretation (higher is better or lower is better) depends on the specific similarity function used. Scores are a by-product of vector search and are not required for a successful search operation. Score values are not part of a domain model and therefore represented best as out-of-band data.

Generally, a `Score` is computed by a `ScoringFunction`. The actual scoring function used to calculate this score can depends on the underlying database and can be obtained from a search index or input parameters.

Spring Data support declares constants for commonly used functions such as:

Euclidean Distance

Calculates the straight-line distance in n-dimensional space involving the square root of the sum of squared differences.

Cosine Similarity

Measures the angle between two vectors by calculating the Dot product first and then normalizing its result by dividing by the product of their lengths.

Dot Product

Computes the sum of element-wise multiplications.

The choice of similarity function can impact both the performance and semantics of the search and is often determined by the underlying database or index being used. Spring Data adopts to the database's native scoring function capabilities and whether the score can be used to limit results.

Hibernate translates distance function calls to native database functions for PGvector and Oracle. Their result is typically a distance. When using `Similarity` instead of `Score`, Spring Data normalizes dis-

tance scores into a similarity score between 0 and 1. The higher the score, the more similar the two vectors are.

Example 2. Using Score and Similarity in a Repository Search Methods

```
interface CommentRepository extends Repository<Comment, String> {  
  
    SearchResults<Comment> searchByEmbeddingNear(Vector vector, ScoringFunction  
function);  
  
    SearchResults<Comment> searchByEmbeddingNear(Vector vector, Score score);  
  
    SearchResults<Comment> searchByEmbeddingNear(Vector vector, Similarity similarity);  
  
    SearchResults<Comment> searchByEmbeddingNear(Vector vector, Range<Similarity> range);  
}  
  
repository.searchByEmbeddingNear(Vector.of(...), ScoringFunction.cosine());  
1  
  
repository.searchByEmbeddingNear(Vector.of(...), Score.of(0.9,  
ScoringFunction.cosine()));  
2  
  
repository.searchByEmbeddingNear(Vector.of(...), Similarity.of(0.9,  
ScoringFunction.cosine()));  
3  
  
repository.searchByEmbeddingNear(Vector.of(...), Similarity.between(0.5, 1,  
ScoringFunction.euclidean()));  
4
```

JAVA

- 1 Run a search and return results that are similar to the given `Vector` applying Cosine scoring.
- 2 Run a search and return results with a score of `0.9` or smaller using the Cosine distance.
- 3 Run a search and normalize the score into a similarity value. Return results with a similarity of `0.9` or greater using Cosine scoring.
- 4 Run a search and normalize the score into a similarity value. Return results with a similarity of between `0.5` and `1.0` or greater using Euclidean scoring.

NOTE

JPA requires a `ScoringFunction` to be provided when creating `Score` or `Similarity` instances to select a scoring function.

Vector Search Methods

Vector search methods are defined in repositories using the same conventions as standard Spring Data query methods. These methods return `SearchResults<T>` and require a `Vector` parameter to define the query vector. The actual implementation depends on the actual internals of the underlying data store and its capabilities around vector search.

NOTE

If you are new to Spring Data repositories, make sure to familiarize yourself with the [basics of repository definitions and query methods](#).

Generally, you have the choice of declaring a search method using two approaches:

- Query Derivation
- Declaring a String-based Query

Vector Search methods must declare a `Vector` parameter to define the query vector.

Derived Search Methods

A derived search method uses the name of the method to derive the query. Vector Search supports the following keywords to run a Vector search when declaring a search method:

Table 1. Query predicate keywords

Logical keyword	Keyword expressions
NEAR	Near , IsNear
WITHIN	Within , IsWithin

Example 3. Using Near and Within Keywords in Repository Search Methods

```
interface CommentRepository extends Repository<Comment, String> {  
  
    SearchResults<Comment> searchByEmbeddingNear(Vector vector, Score score);  
  
    SearchResults<Comment> searchByEmbeddingWithin(Vector vector, Range<Similarity>  
range);  
  
    SearchResults<Comment> searchByCountryAndEmbeddingWithin(String country, Vector  
vector, Range<Similarity> range);  
}
```

JAVA

Derived search methods can declare predicates on domain model attributes and `Vector` parameters.

Derived search methods are typically easier to read and maintain, as they rely on the method name to express the query intent. However, a derived search method requires either to declare a `Score`, `Range<Score>` or `ScoreFunction` as second argument to the `Near / Within` keyword to limit search results by their score.

Annotated Search Methods

Annotated methods provide full control over the query semantics and parameters. Unlike derived methods, they do not rely on method name conventions.

Annotated search methods must define the entire JPQL query to run a Vector Search.

Example 4. Using @Query Search Methods

```
interface CommentRepository extends Repository<Comment, String> {  
  
    @Query("""  
        SELECT c, cosine_distance(c.embedding, :embedding) as distance FROM Comment c  
        WHERE c.country = ?1  
        AND cosine_distance(c.embedding, :embedding) <= :distance  
        ORDER BY distance asc""")  
    SearchResults<Comment> searchAnnotatedByCountryAndEmbeddingWithin(String country,  
        Vector embedding,  
        Score distance);  
  
    @Query("""  
        SELECT c FROM Comment c  
        WHERE c.country = ?1  
        AND cosine_distance(c.embedding, :embedding) <= :distance  
        ORDER BY cosine_distance(c.embedding, :embedding) asc""")  
    List<Comment> findAnnotatedByCountryAndEmbeddingWithin(String country, Vector  
        embedding, Score distance);  
}
```

JAVA

Vector Search methods are not required to include a score or distance in their projection. When using annotated search methods returning `SearchResults`, the execution mechanism assumes that if a second projection column is present that this one holds the score value.

With more control over the actual query, Spring Data can make fewer assumptions about the query and its parameters. For example, `Similarity` normalization uses the native score function within the query to normalize the given similarity into a score predicate value and vice versa. If an annotated query does not define e.g. the score, then the score value in the returned `SearchResult<T>` will be zero.

Sorting

By default, search results are ordered according to their score. You can override sorting by using the `Sort` parameter:

Example 5. Using Sort in Repository Search Methods

```
interface CommentRepository extends Repository<Comment, String> {  
  
    SearchResults<Comment> searchByEmbeddingNearOrderByCountry(Vector vector, Score  
score);  
  
    SearchResults<Comment> searchByEmbeddingWithin(Vector vector, Score score, Sort  
sort);  
}
```

JAVA

Please note that custom sorting does not allow expressing the score as a sorting criteria. You can only refer to domain properties.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Transactionality

Transactionality

Transactional query methods

By default, methods inherited from `CrudRepository` inherit the transactional configuration from [SimpleJpaRepository](#). For read operations, the transaction configuration `readOnly` flag is set to `true`. All others are configured with a plain `@Transactional` so that default transaction configuration applies. Repository methods that are backed by transactional repository fragments inherit the transactional attributes from the actual fragment method.

If you need to tweak transaction configuration for one of the methods declared in a repository, redeclare the method in your repository interface, as follows:

Example 1. Custom transaction configuration for CRUD

```
public interface UserRepository extends CrudRepository<User, Long> {  
  
    @Override  
    @Transactional(timeout = 10)  
    public List<User> findAll();  
  
    // Further query method declarations  
}
```

JAVA

Doing so causes the `findAll()` method to run with a timeout of 10 seconds and without the `readOnly` flag.

Another way to alter transactional behaviour is to use a facade or service implementation that (typically) covers more than one repository. Its purpose is to define transactional boundaries for non-CRUD operations. The following example shows how to use such a facade for more than one repository:

Example 2. Using a facade to define transactions for multiple repository calls

```
@Service  
public class UserManagementImpl implements UserManagement {
```

JAVA

```

private final UserRepository userRepository;
private final RoleRepository roleRepository;

public UserManagementImpl(UserRepository userRepository,
    RoleRepository roleRepository) {
    this.userRepository = userRepository;
    this.roleRepository = roleRepository;
}

@Transactional
public void addRoleToAllUsers(String roleName) {

    Role role = roleRepository.findByName(roleName);

    for (User user : userRepository.findAll()) {
        user.addRole(role);
        userRepository.save(user);
    }
}
}

```

This example causes call to `addRoleToAllUsers(...)` to run inside a transaction (participating in an existing one or creating a new one if none are already running). The transaction configuration at the repositories is then neglected, as the outer transaction configuration determines the actual one used. Note that you must activate `<tx:annotation-driven />` or use

`@EnableTransactionManagement` explicitly to get annotation-based configuration of facades to work. This example assumes you use component scanning.

Note that the call to `save` is not strictly necessary from a JPA point of view, but should still be there in order to stay consistent to the repository abstraction offered by Spring Data.

Transactional query methods

Declared query methods (including default methods) do not get any transaction configuration applied by default. To run those methods transactionally, use `@Transactional` at the repository interface you define, as shown in the following example:

Example 3. Using `@Transactional` at query methods

```

@Transactional(readOnly = true)
interface UserRepository extends JpaRepository<User, Long> {

```

JAVA

```
List<User> findByLastname(String lastname);

@Modifying
@Transactional
@Query("delete from User u where u.active = false")
void deleteInactiveUsers();
}
```

Typically, you want the `readOnly` flag to be set to `true`, as most of the query methods only read data. In contrast to that, `deleteInactiveUsers()` makes use of the `@Modifying` annotation and overrides the transaction configuration. Thus, the method runs with the `readOnly` flag set to `false`.

NOTE

You can use transactions for read-only queries and mark them as such by setting the `readOnly` flag. Doing so does not, however, act as a check that you do not trigger a manipulating query (although some databases reject `INSERT` and `UPDATE` statements inside a read-only transaction). The `readOnly` flag is instead propagated as a hint to the underlying JDBC driver for performance optimizations. Furthermore, Spring performs some optimizations on the underlying JPA provider. For example, when used with Hibernate, the flush mode is set to `NEVER` when you configure a transaction as `readOnly`, which causes Hibernate to skip dirty checks (a noticeable improvement on large object trees).

NOTE

While examples discuss `@Transactional` usage on the repository, we generally recommend declaring transaction boundaries when starting a unit of work to ensure proper consistency and desired transaction participation.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Locking

To specify the lock mode to be used, you can use the `@Lock` annotation on query methods, as shown in the following example:

Example 1. Defining lock metadata on query methods

```
interface UserRepository extends Repository<User, Long> {  
  
    // Plain query method  
    @Lock(LockModeType.READ)  
    List<User> findByLastname(String lastname);  
}
```

JAVA

This method declaration causes the query being triggered to be equipped with a `LockModeType` of `READ`. You can also define locking for CRUD methods by redeclaring them in your repository interface and adding the `@Lock` annotation, as shown in the following example:

Example 2. Defining lock metadata on CRUD methods

```
interface UserRepository extends Repository<User, Long> {  
  
    // Redclaration of a CRUD method  
    @Lock(LockModeType.READ)  
    List<User> findAll();  
}
```

JAVA

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Auditing

Auditing

Basics

- Annotation-based Auditing Metadata

- Interface-based Auditing Metadata

- AuditorAware

- ReactiveAuditorAware

- General Auditing Configuration

Basics

Spring Data provides sophisticated support to transparently keep track of who created or changed an entity and when the change happened. To benefit from that functionality, you have to equip your entity classes with auditing metadata that can be defined either using annotations or by implementing an interface. Additionally, auditing has to be enabled either through Annotation configuration or XML configuration to register the required infrastructure components. Please refer to the store-specific section for configuration samples.

NOTE

Applications that only track creation and modification dates are not required to make their entities implement [AuditorAware](#).

Annotation-based Auditing Metadata

We provide `@CreatedBy` and `@LastModifiedBy` to capture the user who created or modified the entity as well as `@CreatedDate` and `@LastModifiedDate` to capture when the change happened.

An audited entity

```
class Customer {  
  
    @CreatedBy  
    private User user;  
  
    @CreatedDate
```

JAVA

```
private Instant createdAt;  
  
// ... further properties omitted  
}
```

As you can see, the annotations can be applied selectively, depending on which information you want to capture. The annotations, indicating to capture when changes are made, can be used on properties of type JDK8 date and time types, `long`, `Long`, and legacy Java `Date` and `Calendar`.

The time giving instance is provided by a `org.springframework.data.auditing.DateTimeProvider`. By default this is a `CurrentDateTimeProvider`. This can be changed via the `dateTimeProviderRef` attribute when enabling auditing, or a dedicated `AuditingHandler` or `DateTimeProvider` bean being present in the `ApplicationContext`.

Auditing metadata does not necessarily need to live in the root level entity but can be added to an embedded one (depending on the actual store in use), as shown in the snippet below.

Audit metadata in embedded entity

```
class Customer {  
  
    private AuditMetadata auditingMetadata;  
  
    // ... further properties omitted  
}  
  
class AuditMetadata {  
  
    @CreatedBy  
    private User user;  
  
    @CreatedDate  
    private Instant createdAt;  
  
}
```

JAVA

Interface-based Auditing Metadata

In case you do not want to use annotations to define auditing metadata, you can let your domain class implement the `Auditable` interface. It exposes setter methods for all of the auditing properties.

AuditorAware

In case you use either `@CreatedBy` or `@LastModifiedBy`, the auditing infrastructure somehow needs to become aware of the current principal. To do so, we provide an `AuditorAware<T>` SPI interface that you have to implement to tell the infrastructure who the current user or system interacting with the application is. The generic type `T` defines what type the properties annotated with `@CreatedBy` or `@LastModifiedBy` have to be.

The following example shows an implementation of the interface that uses Spring Security's `Authentication` object:

Implementation of AuditorAware based on Spring Security

```
class SpringSecurityAuditorAware implements AuditorAware<User> {  
  
    @Override  
    public Optional<User> getCurrentAuditor() {  
  
        return Optional.ofNullable(SecurityContextHolder.getContext())  
            .map(SecurityContext::getAuthentication)  
            .filter(Authentication::isAuthenticated)  
            .map(Authentication::getPrincipal)  
            .map(User.class::cast);  
    }  
}
```

JAVA

The implementation accesses the `Authentication` object provided by Spring Security and looks up the custom `UserDetails` instance that you have created in your `UserDetailsService` implementation. We assume here that you are exposing the domain user through the `UserDetails` implementation but that, based on the `Authentication` found, you could also look it up from anywhere.

ReactiveAuditorAware

When using reactive infrastructure you might want to make use of contextual information to provide `@CreatedBy` or `@LastModifiedBy` information. We provide an `ReactiveAuditorAware<T>` SPI interface that you have to implement to tell the infrastructure who the current user or system interacting with the application is. The generic type `T` defines what type the properties annotated with `@CreatedBy` or `@LastModifiedBy` have to be.

The following example shows an implementation of the interface that uses reactive Spring Security's `Authentication` object:

Implementation of ReactiveAuditorAware based on Spring Security

```

class SpringSecurityAuditorAware implements ReactiveAuditorAware<User> {

    @Override
    public Mono<User> getCurrentAuditor() {

        return ReactiveSecurityContextHolder.getContext()
            .mapNotNull(SecurityContext::getAuthentication)
            .filter(Authentication::isAuthenticated)
            .mapNotNull(Authentication::getPrincipal)
            .map(User.class::cast);
    }
}

```

The implementation accesses the `Authentication` object provided by Spring Security and looks up the custom `UserDetails` instance that you have created in your `UserDetailsService` implementation. We assume here that you are exposing the domain user through the `UserDetails` implementation but that, based on the `Authentication` found, you could also look it up from anywhere.

There is also a convenience base class, `AbstractAuditable`, which you can extend to avoid the need to manually implement the interface methods. Doing so increases the coupling of your domain classes to Spring Data, which might be something you want to avoid. Usually, the annotation-based way of defining auditing metadata is preferred as it is less invasive and more flexible.

General Auditing Configuration

Spring Data JPA ships with an entity listener that can be used to trigger the capturing of auditing information. First, you must register the `AuditingEntityListener` to be used for all entities in your persistence contexts inside your `orm.xml` file, as shown in the following example:

Example 1. Auditing configuration orm.xml

```

<persistence-unit-metadata>
  <persistence-unit-defaults>
    <entity-listeners>
      <entity-listener class="...data.jpa.domain.support.AuditingEntityListener" />
    </entity-listeners>
  </persistence-unit-defaults>
</persistence-unit-metadata>

```

You can also enable the `AuditingEntityListener` on a per-entity basis by using the `@EntityListeners` annotation, as follows:

JAVA

```
@Entity
@EntityListeners(AuditingEntityListener.class)
public class MyEntity {

}
```

NOTE

The auditing feature requires `spring-aspects.jar` to be on the classpath.

With `orm.xml` suitably modified and `spring-aspects.jar` on the classpath, activating auditing functionality is a matter of adding the Spring Data JPA `auditing` namespace element to your configuration, as follows:

Example 2. Activating auditing using XML configuration

XML

```
<jpa:auditing auditor-aware-ref="yourAuditorAwareBean" />
```

As of Spring Data JPA 1.5, you can enable auditing by annotating a configuration class with the `@EnableJpaAuditing` annotation. You must still modify the `orm.xml` file and have `spring-aspects.jar` on the classpath. The following example shows how to use the `@EnableJpaAuditing` annotation:

Example 3. Activating auditing with Java configuration

JAVA

```
@Configuration
@EnableJpaAuditing
class Config {

    @Bean
    public AuditorAware<AuditableUser> auditorProvider() {
        return new AuditorAwareImpl();
    }
}
```

If you expose a bean of type `AuditorAware` to the `ApplicationContext`, the auditing infrastructure automatically picks it up and uses it to determine the current user to be set on domain types. If you have multiple implementations registered in the `ApplicationContext`, you can select the one to be used by explicitly setting the `auditorAwareRef` attribute of `@EnableJpaAuditing`.

Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Merging persistence units

Merging persistence units

Classpath Scanning for @Entity Classes and JPA Mapping Files

Spring supports having multiple persistence units. Sometimes, however, you might want to modularize your application but still make sure that all these modules run inside a single persistence unit. To enable that behavior, Spring Data JPA offers a `PersistenceUnitManager` implementation that automatically merges persistence units based on their name, as shown in the following example:

Example 1. Using `MergingPersistenceUnitManager`

```
<bean class="...LocalContainerEntityManagerFactoryBean">
  <property name="persistenceUnitManager">
    <bean class="...MergingPersistenceUnitManager" />
  </property>
</bean>
```

XML

Classpath Scanning for @Entity Classes and JPA Mapping Files

A plain JPA setup requires all annotation-mapped entity classes to be listed in `orm.xml`. The same applies to XML mapping files. Spring Data JPA provides a

`ClasspathScanningPersistenceUnitPostProcessor` that gets a base package configured and optionally takes a mapping filename pattern. It then scans the given package for classes annotated with `@Entity` or `@MappedSuperclass`, loads the configuration files that match the filename pattern, and hands them to the JPA configuration. The post-processor must be configured as follows:

Example 2. Using `ClasspathScanningPersistenceUnitPostProcessor`

```
<bean class="...LocalContainerEntityManagerFactoryBean">
  <property name="persistenceUnitPostProcessors">
    <list>
      <bean
class="org.springframework.data.jpa.support.ClasspathScanningPersistenceUnitPostProcess
or">
```

XML

```
<constructor-arg value="com.acme.domain" />
<property name="mappingFileNamePattern" value="**/*Mapping.xml" />
</bean>
</list>
</property>
</bean>
```

NOTE

As of Spring 3.1, a package to scan can be configured on the `LocalContainerEntityManagerFactoryBean` directly to enable classpath scanning for entity classes. See the [JavaDoc](#) for details.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

CDI Integration

Instances of the repository interfaces are usually created by a container, for which Spring is the most natural choice when working with Spring Data. Spring offers sophisticated support for creating bean instances, as documented in [Creating Repository Instances](#). As of version 1.1.0, Spring Data JPA ships with a custom CDI extension that allows using the repository abstraction in CDI environments. The extension is part of the JAR. To activate it, include the Spring Data JPA JAR on your classpath.

You can now set up the infrastructure by implementing a CDI Producer for the `EntityManagerFactory` and `EntityManager`, as shown in the following example:

```
class EntityManagerFactoryProducer {  
  
    @Produces  
    @ApplicationScoped  
    public EntityManagerFactory createEntityManagerFactory() {  
        return Persistence.createEntityManagerFactory("my-persistence-unit");  
    }  
  
    public void close(@Disposes EntityManagerFactory entityManagerFactory) {  
        entityManagerFactory.close();  
    }  
  
    @Produces  
    @RequestScoped  
    public EntityManager createEntityManager(EntityManagerFactory entityManagerFactory) {  
        return entityManagerFactory.createEntityManager();  
    }  
  
    public void close(@Disposes EntityManager entityManager) {  
        entityManager.close();  
    }  
}
```

JAVA

The necessary setup can vary depending on the JavaEE environment. You may need to do nothing more than redeclare a `EntityManager` as a CDI bean, as follows:

```
class CdiConfig {

    @Produces
    @RequestScoped
    @PersistenceContext
    public EntityManager entityManager;
}
```

In the preceding example, the container has to be capable of creating JPA `EntityManager` itself. All the configuration does is re-export the JPA `EntityManager` as a CDI bean.

The Spring Data JPA CDI extension picks up all available `EntityManager` instances as CDI beans and creates a proxy for a Spring Data repository whenever a bean of a repository type is requested by the container. Thus, obtaining an instance of a Spring Data repository is a matter of declaring an `@Inject` property, as shown in the following example:

```
class RepositoryClient {

    @Inject
    PersonRepository repository;

    public void businessMethod() {
        List<Person> people = repository.findAll();
    }
}
```



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Custom Repository Implementations

Custom Repository Implementations

- Customizing Individual Repositories

 - Configuration

- Customize the Base Repository

- Customizing the Repository Factory

- Customize the Repository Factory Bean

- Using JpaContext in Custom Implementations

Spring Data provides various options to create query methods with little coding. But when those options don't fit your needs you can also provide your own custom implementation for repository methods. This section describes how to do that.

Customizing Individual Repositories

To enrich a repository with custom functionality, you must first define a fragment interface and an implementation for the custom functionality, as follows:

Interface for custom repository functionality

```
interface CustomizedUserRepository {  
    void someCustomMethod(User user);  
}
```

JAVA

Implementation of custom repository functionality

```
class CustomizedUserRepositoryImpl implements CustomizedUserRepository {  
  
    @Override  
    public void someCustomMethod(User user) {  
        // Your custom implementation  
    }  
}
```

JAVA

NOTE

The most important part of the class name that corresponds to the fragment interface is the `Impl` postfix. You can customize the store-specific postfix by setting `@Enable<StoreModule>Repositories(repositoryImplementationPostfix = ...)`.

WARNING

Historically, Spring Data custom repository implementation discovery followed a naming pattern that derived the custom implementation class name from the repository allowing effectively a single custom implementation.

A type located in the same package as the repository interface, matching *repository interface name* followed by *implementation postfix*, is considered a custom implementation and will be treated as a custom implementation. A class following that name can lead to undesired behavior.

We consider the single-custom implementation naming deprecated and recommend not using this pattern. Instead, migrate to a fragment-based programming model.

The implementation itself does not depend on Spring Data and can be a regular Spring bean. Consequently, you can use standard dependency injection behavior to inject references to other beans (such as a `JdbcTemplate`), take part in aspects, and so on.

Then you can let your repository interface extend the fragment interface, as follows:

Changes to your repository interface

```
interface UserRepository extends CrudRepository<User, Long>, CustomizedUserRepository {JAVA  
  
    // Declare query methods here  
}
```

Extending the fragment interface with your repository interface combines the CRUD and custom functionality and makes it available to clients.

Spring Data repositories are implemented by using fragments that form a repository composition. Fragments are the base repository, functional aspects (such as Querydsl), and custom interfaces along with their implementations. Each time you add an interface to your repository interface, you enhance the composition by adding a fragment. The base repository and repository aspect implementations are provided by each Spring Data module.

The following example shows custom interfaces and their implementations:

Fragments with their implementations

```
interface HumanRepository {JAVA  
    void someHumanMethod(User user);  
}
```

```

class HumanRepositoryImpl implements HumanRepository {

    @Override
    public void someHumanMethod(User user) {
        // Your custom implementation
    }
}

interface ContactRepository {

    void someContactMethod(User user);

    User anotherContactMethod(User user);
}

class ContactRepositoryImpl implements ContactRepository {

    @Override
    public void someContactMethod(User user) {
        // Your custom implementation
    }

    @Override
    public User anotherContactMethod(User user) {
        // Your custom implementation
    }
}

```

The following example shows the interface for a custom repository that extends `CrudRepository`:

Changes to your repository interface

```

interface UserRepository extends CrudRepository<User, Long>, HumanRepository,
ContactRepository {

    // Declare query methods here
}

```

JAVA

Repositories may be composed of multiple custom implementations that are imported in the order of their declaration. Custom implementations have a higher priority than the base implementation and repository aspects. This ordering lets you override base repository and aspect methods and resolves ambiguity if two fragments contribute the same method signature. Repository fragments are not limited to use in a single repository interface. Multiple repositories may use a fragment interface, letting you re-use customizations across different repositories.

The following example shows a repository fragment and its implementation:

Fragments overriding save(...)

```
interface CustomizedSave<T> {
    <S extends T> S save(S entity);
}

class CustomizedSaveImpl<T> implements CustomizedSave<T> {

    @Override
    public <S extends T> S save(S entity) {
        // Your custom implementation
    }
}
```

JAVA

The following example shows a repository that uses the preceding repository fragment:

Customized repository interfaces

```
interface UserRepository extends CrudRepository<User, Long>, CustomizedSave<User> {
}

interface PersonRepository extends CrudRepository<Person, Long>, CustomizedSave<Person> {
}
```

JAVA

Configuration

The repository infrastructure tries to autodetect custom implementation fragments by scanning for classes below the package in which it found a repository. These classes need to follow the naming convention of appending a postfix defaulting to `Impl`.

The following example shows a repository that uses the default postfix and a repository that sets a custom value for the postfix:

Example 1. Configuration example

Java

XML

```
@EnableJpaRepositories(repositoryImplementationPostfix = "MyPostfix")
class Configuration { ... }
```

JAVA

The first configuration in the preceding example tries to look up a class called `com.acme.repository.CustomizedUserRepositoryImpl` to act as a custom repository implementation. The second example tries to look up `com.acme.repository.CustomizedUserRepositoryMyPostfix`.

Resolution of Ambiguity

If multiple implementations with matching class names are found in different packages, Spring Data uses the bean names to identify which one to use.

Given the following two custom implementations for the `CustomizedUserRepository` shown earlier, the first implementation is used. Its bean name is `customizedUserRepositoryImpl`, which matches that of the fragment interface (`CustomizedUserRepository`) plus the postfix `Impl`.

Example 2. Resolution of ambiguous implementations

```
class CustomizedUserRepositoryImpl implements CustomizedUserRepository {  
  
    // Your custom implementation  
}
```

JAVA

```
@Component("specialCustomImpl")  
class CustomizedUserRepositoryImpl implements CustomizedUserRepository {  
  
    // Your custom implementation  
}
```

JAVA

If you annotate the `UserRepository` interface with `@Component("specialCustom")`, the bean name plus `Impl` then matches the one defined for the repository implementation in `com.acme.impl.two`, and it is used instead of the first one.

Manual Wiring

If your custom implementation uses annotation-based configuration and autowiring only, the preceding approach shown works well, because it is treated as any other Spring bean. If your implementation fragment bean needs special wiring, you can declare the bean and name it according to the conventions described in the preceding section. The infrastructure then refers to the manually defined bean definition by name instead of creating one itself. The following example shows how to manually wire a custom implementation:

Example 3. Manual wiring of custom implementations

Java	XML
------	------------

```
class MyClass {  
    MyClass(@Qualifier("userRepositoryImpl") UserRepository userRepository) {  
        ...  
    }  
}
```

JAVA

```
}  
}
```

Registering Fragments with `spring.factories`

As already mentioned in the Configuration section, the infrastructure only auto-detects fragments within the repository base-package. Therefore, fragments residing in another location or want to be contributed by an external archive will not be found if they do not share a common namespace. Registering fragments within `spring.factories` allows you to circumvent this restriction as explained in the following section.

Imagine you'd like to provide some custom search functionality usable across multiple repositories for your organization leveraging a text search index.

First all you need is the fragment interface. Note the generic `<T>` parameter to align the fragment with the repository domain type.

Fragment Interface

```
public interface SearchExtension<T> {  
  
    List<T> search(String text, Limit limit);  
}
```

JAVA

Let's assume the actual full-text search is available via a `SearchService` that is registered as a `Bean` within the context so you can consume it in our `SearchExtension` implementation. All you need to run the search is the collection (or index) name and an object mapper that converts the search results into actual domain objects as sketched out below.

Fragment implementation

```
class DefaultSearchExtension<T> implements SearchExtension<T> {  
  
    private final SearchService service;  
  
    DefaultSearchExtension(SearchService service) {  
        this.service = service;  
    }  
  
    @Override  
    public List<T> search(String text, Limit limit) {  
        return search(RepositoryMethodContext.getContext(), text, limit);  
    }  
}
```

JAVA


```

List<T> search(RepositoryMethodContext metadata, String text, Limit limit) {

    Class<T> domainType = metadata.getRepository().getDomainType();

    String indexName = domainType.getSimpleName().toLowerCase();
    List<String> jsonResult = service.search(indexName, text, 0, limit.max());

    return jsonResult.stream().map(...).collect(toList());
}
}

```

In the example above `RepositoryMethodContext.getContext()` is used to retrieve metadata for the actual method invocation. `RepositoryMethodContext` exposes information attached to the repository such as the domain type. In this case we use the repository domain type to identify the name of the index to be searched.

Exposing invocation metadata is costly, hence it is disabled by default. To access `RepositoryMethodContext.getContext()` you need to advise the repository factory responsible for creating the actual repository to expose method metadata.

Expose Repository Metadata

Marker Interface

Bean Post Processor

Adding the `RepositoryMetadataAccess` marker interface to the fragments implementation will trigger the infrastructure and enable metadata exposure for those repositories using the fragment.

```

class DefaultSearchExtension<T> implements SearchExtension<T>, RepositoryMetadataAccess
{
    // ...
}

```

JAVA

Having both, the fragment declaration and implementation in place you can register the extension in the `META-INF/spring.factories` file and package things up if needed.

Register the fragment in META-INF/spring.factories

```
com.acme.search.SearchExtension=com.acme.search.DefaultSearchExtension
```

PROPERTIES

Now you are ready to make use of your extension; Simply add the interface to your repository.

Using it

JAVA

```
interface MovieRepository extends CrudRepository<Movie, String>, SearchExtension<Movie> {  
  
}
```

Customize the Base Repository

The approach described in the preceding section requires customization of each repository interfaces when you want to customize the base repository behavior so that all repositories are affected. To instead change behavior for all repositories, you can create an implementation that extends the persistence technology-specific repository base class. This class then acts as a custom base class for the repository proxies, as shown in the following example:

Custom repository base class

```
class MyRepositoryImpl<T, ID> JAVA  
    extends SimpleJpaRepository<T, ID> {  
  
    private final EntityManager entityManager;  
  
    MyRepositoryImpl(JpaEntityInformation entityInformation,  
                    EntityManager entityManager) {  
        super(entityInformation, entityManager);  
  
        // Keep the EntityManager around to used from the newly introduced methods.  
        this.entityManager = entityManager;  
    }  
  
    @Override  
    @Transactional  
    public <S extends T> S save(S entity) {  
        // implementation goes here  
    }  
}
```

CAUTION

The class needs to have a constructor of the super class which the store-specific repository factory implementation uses. If the repository base class has multiple constructors, override the one taking an `EntityInformation` plus a store specific infrastructure object (such as an `EntityManager` or a template class).

The final step is to make the Spring Data infrastructure aware of the customized repository base class. In configuration, you can do so by using the `repositoryBaseClass`, as shown in the following example:

Example 4. Configuring a custom repository base class

Java

XML

```
@Configuration
@EnableJpaRepositories(repositoryBaseClass = MyRepositoryImpl.class)
class ApplicationConfiguration { ... }
```

JAVA

Customizing the Repository Factory

Customizing the repository factory through [RepositoryFactoryCustomizer](#) provides direct access to components involved with repository instance creation. This mechanism is useful when you want to adjust selected aspects of proxy creation without introducing a fully custom repository factory bean. The following example, demonstrates registering additional listeners and proxy advisors:

```
factoryBean.addRepositoryFactoryCustomizer(repositoryFactory -> {
    repositoryFactory.addInvocationListener(...);
    repositoryFactory.addQueryCreationListener(...);

    repositoryFactory.addRepositoryProxyPostProcessor((factory, repositoryInformation) ->
factory.addAdvice(...));
});
```

JAVA

A `RepositoryFactoryCustomizer` is associated with a particular repository factory bean, ideally through `BeanPostProcessor` so that only specific repositories are affected. Note that customizer beans are not applied automatically to prevent unwanted wiring that become especially relevant in multi-repository arrangements or when using multiple Spring Data modules.

Customize the Repository Factory Bean

The most powerful approach to customize repository creation is to provide a custom repository factory bean, typically a subclass of [RepositoryFactorySupport](#), [TransactionalRepositoryFactoryBeanSupport](#) or the store-specific repository factory bean.

Customizing the repository factory bean allows you to change repository creation entirely with full access to the underlying repository factory.

Note that this approach requires the most effort and is typically only needed when you want to change core aspects of repository creation. Also, you need to take in consideration repository metadata derivation that is used to identify query methods, base implementation methods and custom implementations. The following summary outlines the key aspects:

- `repositoryBaseClass` : The repository base class defines which methods are implemented by the base class and which methods require additional handling through aspects or custom implementations.
- `repositoryFragmentsContributor` : A [RepositoryFragmentsContributor](#) allows contributions to repository composition after all standard fragments have been collected. Store modules use this mechanism to add features such as Querydsl or Query-by-Example support. It also serves as an SPI for third-party extensions.
- `exposeMetadata` : Controls whether invocation metadata is available through `RepositoryMethodContext.getContext()` .

Using JpaContext in Custom Implementations

When working with multiple `EntityManager` instances and custom repository implementations, you need to wire the correct `EntityManager` into the repository implementation class. You can do so by explicitly naming the `EntityManager` in the `@PersistenceContext` annotation or, if the `EntityManager` is `@Autowired`, by using `@Qualifier` .

As of Spring Data JPA 1.9, Spring Data JPA includes a class called `JpaContext` that lets you obtain the `EntityManager` by managed domain class, assuming it is managed by only one of the `EntityManager` instances in the application. The following example shows how to use `JpaContext` in a custom repository:

Example 5. Using JpaContext in a custom repository implementation

```
class UserRepositoryImpl implements UserRepositoryCustom {  
  
    private final EntityManager em;  
  
    @Autowired  
    public UserRepositoryImpl(JpaContext context) {  
        this.em = context.getEntityManagerByManagedType(User.class);  
    }  
  
    ...  
}
```

JAVA

The advantage of this approach is that, if the domain type gets assigned to a different persistence unit, the repository does not have to be touched to alter the reference to the persistence unit.

Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Publishing Events from Aggregate Roots

Entities managed by repositories are aggregate roots. In a Domain-Driven Design application, these aggregate roots usually publish domain events. Spring Data provides an annotation called `@DomainEvents` that you can use on a method of your aggregate root to make that publication as easy as possible, as shown in the following example:

Exposing domain events from an aggregate root

```
class AnAggregateRoot {  
  
    @DomainEvents 1  
    Collection<Object> domainEvents() {  
        // ... return events you want to get published here  
    }  
  
    @AfterDomainEventPublication 2  
    void callbackMethod() {  
        // ... potentially clean up domain events list  
    }  
}
```

JAVA

- 1 The method that uses `@DomainEvents` can return either a single event instance or a collection of events. It must not take any arguments.
- 2 After all events have been published, we have a method annotated with `@AfterDomainEventPublication`. You can use it to potentially clean the list of events to be published (among other uses).

The methods are called every time one of the following a Spring Data repository methods are called:

- `save(...)`, `saveAll(...)`
- `delete(...)`, `deleteAll(...)`, `deleteAllInBatch(...)`, `deleteInBatch(...)`

Note, that these methods take the aggregate root instances as arguments. This is why `deleteById(...)` is notably absent, as the implementations might choose to issue a query deleting the instance and thus we would never have access to the aggregate instance in the first place.

Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Null Handling of Repository Methods

Null Handling of Repository Methods

Nullability Annotations

JSpecify

Spring Framework Nullability and JSR-305 Annotations

Nullability in Kotlin-based Repositories

Repository CRUD methods that return an individual aggregate instances can use `Optional` to indicate the potential absence of a value. Besides that, Spring Data supports returning the following wrapper types on query methods:

- `com.google.common.base.Optional`
- `scala.Option`
- `io.vavr.control.Option`

Alternatively, query methods can choose not to use a wrapper type at all. The absence of a query result is then indicated by returning `null`. Repository methods returning collections, collection alternatives, wrappers, and streams are guaranteed never to return `null` but rather the corresponding empty representation. See "[Repository query return types](#)" for details.

Nullability Annotations

JSpecify

As on Spring Framework 7 and Spring Data 4, you can express nullability constraints for repository methods by using `JSpecify`. `JSpecify` is well integrated into IntelliJ and Eclipse to provide a tooling-friendly approach and opt-in `null` checks during runtime, as follows:

- [@NotNullMarked](#): Used on the module-, package- and class-level to declare that the default behavior for parameters and return values is, respectively, neither to accept nor to produce `null` values.
- [@Nonnull](#): Used on a type level for parameter or return values that must not be `null` (not needed value where `@NotNullMarked` applies).

- [@Nullable](#): Used on the type level for parameter or return values that can be null.
- [@NullUnmarked](#): Used on the package-, class-, and method-level to roll back nullness declaration and opt-out from a previous [@Nullable](#). Nullness changes to unspecified in such a case.

@Nullable at the package level via a package-info.java file

```
@Nullable
package org.springframework.core;

import org.jspecify.annotations.Nullable;
```

JAVA

In the various Java files belonging to the package, nullable type usages are defined explicitly with [@Nullable](#). It is recommended that this annotation is specified just before the related type.

For example, for a field:

```
private @Nullable String fileEncoding;
```

JAVA

Or for method parameters and return value:

```
public static @Nullable String buildMessage(@Nullable String message,
                                           @Nullable Throwable cause) {
    // ...
}
```

JAVA

When overriding a method, nullness annotations are not inherited from the superclass method. That means those nullness annotations should be repeated if you just want to override the implementation and keep the same API nullness.

With arrays and varargs, you need to be able to differentiate the nullness of the elements from the nullness of the array itself. Pay attention to the syntax defined by the Java specification which may be initially surprising:

- `@Nullable Object[] array` means individual elements can be null but the array itself can't.
- `Object @Nullable [] array` means individual elements can't be null but the array itself can.
- `@Nullable Object @Nullable [] array` means both individual elements and the array can be null.

The Java specifications also enforces that annotations defined with `@Target(ElementType.TYPE_USE)` like JSpecify `@Nullable` should be specified after the last `.` with inner or fully qualified types:

- `Cache.@Nullable ValueWrapper`

- `jakarta.validation.@Nullable` Validator

[@NonNull](#) and [@NotNullUnmarked](#) should rarely be needed for typical use cases.

Spring Framework Nullability and JSR-305 Annotations

You can express nullability constraints for repository methods by using [Spring Framework's nullability annotations](#).

NOTE

As on Spring Framework 7, Spring's nullability annotations are deprecated in favor of JSpecify. Consult the framework documentation on [Migrating from Spring null-safety annotations to JSpecify](#) for more information.

They provide a tooling-friendly approach and opt-in `null` checks during runtime, as follows:

- [@NonNullApi](#) : Used on the package level to declare that the default behavior for parameters and return values is, respectively, neither to accept nor to produce `null` values.
- [@NonNull](#) : Used on a parameter or return value that must not be `null` (not needed on a parameter and return value where [@NonNullApi](#) applies).
- [@Nullable](#) : Used on a parameter or return value that can be `null`.

Spring annotations are meta-annotated with [JSR 305](#) annotations (a dormant but widely used JSR). JSR 305 meta-annotations let tooling vendors (such as [IDEA](#), [Eclipse](#), and [Kotlin](#)) provide null-safety support in a generic way, without having to hard-code support for Spring annotations. To enable runtime checking of nullability constraints for query methods, you need to activate non-nullability on the package level by using Spring's [@NonNullApi](#) in `package-info.java`, as shown in the following example:

Declaring Non-nullability in package-info.java

JAVA

Once non-null defaulting is in place, repository query method invocations get validated at runtime for nullability constraints. If a query result violates the defined constraint, an exception is thrown. This happens when the method would return `null` but is declared as non-nullable (the default with the annotation defined on the package in which the repository resides). If you want to opt-in to nullable results again, selectively use [@Nullable](#) on individual methods. Using the result wrapper types mentioned at the start of this section continues to work as expected: an empty result is translated into the value that represents absence.

The following example shows a number of the techniques just described:

Using different nullability constraints

```

package com.acme; 1

interface UserRepository extends Repository<User, Long> {

    User getByEmailAddress(EmailAddress emailAddress); 2

    @Nullable
    User findByEmailAddress(@Nullable EmailAddress emailAddress); 3

    Optional<User> findOptionalByEmailAddress(EmailAddress emailAddress); 4
}

```

- 1 The repository resides in a package (or sub-package) for which we have defined non-null behavior.
- 2 Throws an `EmptyResultDataAccessException` when the query does not produce a result. Throws an `IllegalArgumentException` when the `emailAddress` handed to the method is `null`.
- 3 Returns `null` when the query does not produce a result. Also accepts `null` as the value for `emailAddress`.
- 4 Returns `Optional.empty()` when the query does not produce a result. Throws an `IllegalArgumentException` when the `emailAddress` handed to the method is `null`.

Nullability in Kotlin-based Repositories

Kotlin has the definition of nullability constraints baked into the language. Kotlin code compiles to byte-code, which does not express nullability constraints through method signatures but rather through compiled-in metadata. Make sure to include the `kotlin-reflect` JAR in your project to enable introspection of Kotlin's nullability constraints. Spring Data repositories use the language mechanism to define those constraints to apply the same runtime checks, as follows:

Using nullability constraints on Kotlin repositories

```

interface UserRepository : Repository<User, String> {

    fun findByUsername(username: String): User 1

    fun findByFirstname(firstname: String?): User? 2
}

```

- 1 The method defines both the parameter and the result as non-nullable (the Kotlin default). The Kotlin compiler rejects method invocations that pass `null` to the method. If the query yields an empty result, an `EmptyResultDataAccessException` is thrown.
- 2 This method accepts `null` for the `firstname` parameter and returns `null` if the query does not produce a result.

Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Spring Data Extensions

Spring Data Extensions

Querydsl Extension

Setting up Annotation Processing

Web support

Basic Web Support

Creating JSON representations for Page

Spring Data Jackson Modules

Web Databinding Support

Querydsl Web Support

Repository Populators

This section documents a set of Spring Data extensions that enable Spring Data usage in a variety of contexts. Currently, most of the integration is targeted towards Spring MVC.

Querydsl Extension

[Querydsl](#) is a framework that enables the construction of statically typed SQL-like queries through its fluent API.

NOTE

Querydsl maintenance has slowed down to a point where the community has forked the project under OpenFeign at github.com/OpenFeign/querydsl (groupId `io.github.openfeign.querydsl`). Spring Data supports the fork on a best-effort basis.

Several Spring Data modules offer integration with Querydsl through `QuerydslPredicateExecutor`, as the following example shows:

QuerydslPredicateExecutor interface

```
public interface QuerydslPredicateExecutor<T> {  
  
    Optional<T> findById(Predicate predicate);    1  
  
    Iterable<T> findAll(Predicate predicate);    2
```

JAVA

```

    long count(Predicate predicate);           3

    boolean exists(Predicate predicate);       4

    // ... more functionality omitted.
}

```

- 1 Finds and returns a single entity matching the Predicate .
- 2 Finds and returns all entities matching the Predicate .
- 3 Returns the number of entities matching the Predicate .
- 4 Returns whether an entity that matches the Predicate exists.

To use the Querydsl support, extend `QuerydslPredicateExecutor` on your repository interface, as the following example shows:

Querydsl integration on repositories

```

interface UserRepository extends CrudRepository<User, Long>,
    QuerydslPredicateExecutor<User> {
}

```

JAVA

The preceding example lets you write type-safe queries by using Querydsl Predicate instances, as the following example shows:

```

Predicate predicate = user.firstname.equalsIgnoreCase("dave")
    .and(user.lastname.startsWithIgnoreCase("mathews"));

userRepository.findAll(predicate);

```

JAVA

Setting up Annotation Processing

To use Querydsl with Spring Data JPA, you need to set up annotation processing in your build system that generates the Q classes. While you could write the Q classes by hand, it is recommended to use the Querydsl annotation processor to generate them for you to keep your Q classes in sync with your domain model.

Most Spring Data users do not use Querydsl, so it does not make sense to require additional mandatory dependencies for projects that would not benefit from Querydsl. Hence, you need to activate annotation processing in your build system.

The following example shows how to set up annotation processing by mentioning dependencies and compiler config changes in Maven and Gradle:

Maven	Gradle	Maven (OpenFeign)	Gradle (OpenFeign)
-------	--------	-------------------	--------------------

XML

```

<dependencies>
  <dependency>
    <groupId>com.querydsl</groupId>
    <artifactId>querydsl-jpa</artifactId>
    <version>${querydslVersion}</version>
    <classifier>jakarta</classifier>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <configuration>
        <annotationProcessorPaths>
          <!-- Explicit opt-in required via annotationProcessors or
annotationProcessorPaths on Java 22+, see
https://bugs.openjdk.org/browse/JDK-8306819 -->
          <annotationProcessorPath>
            <groupId>com.querydsl</groupId>
            <artifactId>querydsl-apt</artifactId>
            <version>${querydslVersion}</version>
            <classifier>jakarta</classifier>
          </annotationProcessorPath>
          <annotationProcessorPath>
            <groupId>jakarta.persistence</groupId>
            <artifactId>jakarta.persistence-api</artifactId>
          </annotationProcessorPath>
        </annotationProcessorPaths>

        <!-- Recommended: Some IDE's might require this configuration to include
generated sources for IDE usage -->
        <generatedTestSourcesDirectory>target/generated-test-
sources</generatedTestSourcesDirectory>
        <generatedSourcesDirectory>target/generated-
sources</generatedSourcesDirectory>
      </configuration>
    </plugin>
  </plugins>
</build>

```

Note that the setup above shows the simplest usage omitting any other options or dependencies that your project might require.

Web support

Spring Data modules that support the repository programming model ship with a variety of web support. The web related components require Spring MVC JARs to be on the classpath. Some of them even provide integration with [Spring HATEOAS](#). In general, the integration support is enabled by using the `@EnableSpringDataWebSupport` annotation in your JavaConfig configuration class, as the following example shows:

Enabling Spring Data web support

Java	XML
------	-----

```
@Configuration
@EnableWebMvc
@EnableSpringDataWebSupport
class WebConfiguration {}
```

JAVA

The `@EnableSpringDataWebSupport` annotation registers a few components. We discuss those later in this section. It also detects Spring HATEOAS on the classpath and registers integration components (if present) for it as well.

Basic Web Support

Enabling Spring Data web support in XML

The configuration shown in the previous section registers a few basic components:

- A Using the [DomainClassConverter](#) Class to let Spring MVC resolve instances of repository-managed domain classes from request parameters or path variables.
- [HandlerMethodArgumentResolver](#) implementations to let Spring MVC resolve `Pageable` and `Sort` instances from request parameters.
- Jackson Modules to de-/serialize types like `Point` and `Distance`, or store specific ones, depending on the Spring Data Module used.

Using the `DomainClassConverter` Class

The `DomainClassConverter` class lets you use domain types in your Spring MVC controller method signatures directly so that you need not manually lookup the instances through the repository, as the following example shows:

A Spring MVC controller using domain types in method signatures


```

@Controller
@RequestMapping("/users")
class UserController {

    @RequestMapping("/{id}")
    String showUserForm(@PathVariable("id") User user, Model model) {

        model.addAttribute("user", user);
        return "userForm";
    }
}

```

The method receives a `User` instance directly, and no further lookup is necessary. The instance can be resolved by letting Spring MVC convert the path variable into the `id` type of the domain class first and eventually access the instance through calling `findById(...)` on the repository instance registered for the domain type.

NOTE

Currently, the repository has to implement `CrudRepository` to be eligible to be discovered for conversion.

HandlerMethodArgumentResolvers for Pageable and Sort

The configuration snippet shown in the previous section also registers a `PageableHandlerMethodArgumentResolver` as well as an instance of `SortHandlerMethodArgumentResolver`. The registration enables `Pageable` and `Sort` as valid controller method arguments, as the following example shows:

Using Pageable as a controller method argument

```

@Controller
@RequestMapping("/users")
class UserController {

    private final UserRepository repository;

    UserController(UserRepository repository) {
        this.repository = repository;
    }

    @RequestMapping
    String showUsers(Model model, Pageable pageable) {

        model.addAttribute("users", repository.findAll(pageable));
        return "users";
    }
}

```

```

    }
}

```

The preceding method signature causes Spring MVC try to derive a `Pageable` instance from the request parameters by using the following default configuration:

Table 1. Request parameters evaluated for `Pageable` instances

page	Page you want to retrieve. 0-indexed and defaults to 0.
size	Size of the page you want to retrieve. Defaults to 20.
sort	Properties that should be sorted by in the format <code>property,property(,ASC DESC)(,IgnoreCase)</code> . The default sort direction is case-sensitive ascending. Use multiple <code>sort</code> parameters if you want to switch direction or case sensitivity — for example, <code>?sort=firstname&sort=lastname,asc&sort=city,ignorecase</code> .

To customize this behavior, register a bean that implements the `PageableHandlerMethodArgumentResolverCustomizer` interface or the `SortHandlerMethodArgumentResolverCustomizer` interface, respectively. Its `customize()` method gets called, letting you change settings, as the following example shows:

```

@Bean SortHandlerMethodArgumentResolverCustomizer sortCustomizer() {
    return s -> s.setPropertyDelimiter("<-->");
}

```

JAVA

If setting the properties of an existing `MethodArgumentResolver` is not sufficient for your purpose, extend either `SpringDataWebConfiguration` or the HATEOAS-enabled equivalent, override the `pageableResolver()` or `sortResolver()` methods, and import your customized configuration file instead of using the `@Enable` annotation.

If you need multiple `Pageable` or `Sort` instances to be resolved from the request (for multiple tables, for example), you can use Spring's `@Qualifier` annotation to distinguish one from another. The request parameters then have to be prefixed with `${qualifier}_`. The following example shows the resulting method signature:

```

String showUsers(Model model,
    @Qualifier("thing1") Pageable first,
    @Qualifier("thing2") Pageable second) { ... }

```

JAVA

You have to populate `thing1_page`, `thing2_page`, and so on.

The default `Pageable` passed into the method is equivalent to a `PageRequest.of(0, 20)`, but you can customize it by using the `@PageableDefault` annotation on the `Pageable` parameter.

Creating JSON representations for Page

It's common for Spring MVC controllers to try to ultimately render a representation of a Spring Data page to clients. While one could simply return `Page` instances from handler methods to let Jackson render them as is, we strongly recommend against this as the underlying implementation class `PageImpl` is a domain type. This means we might want or have to change its API for unrelated reasons, and such changes might alter the resulting JSON representation in a breaking way.

With Spring Data 3.1, we started hinting at the problem by issuing a warning log describing the problem. We still ultimately recommend to leverage the integration with Spring HATEOAS for a fully stable and hypermedia-enabled way of rendering pages that easily allow clients to navigate them. But as of version 3.3 Spring Data ships a page rendering mechanism that is convenient to use but does not require the inclusion of Spring HATEOAS.

Using Spring Data' PagedModel

At its core, the support consists of a simplified version of Spring HATEOAS' `PagedModel` (the Spring Data one located in the `org.springframework.data.web` package). It can be used to wrap `Page` instances and result in a simplified representation that reflects the structure established by Spring HATEOAS but omits the navigation links.

JAVA

```
@Controller
class MyController {

    private final MyRepository repository;

    // Constructor omitted

    @GetMapping("/page")
    PagedModel<?> page(Pageable pageable) {
        return new PagedModel<>(repository.findAll(pageable)); 1
    }
}
```

1 Wraps the `Page` instance into a `PagedModel`.

This will result in a JSON structure looking like this:

JAVASCRIPT

```
{
  "content" : [
```

```

    ... // Page content rendered here
  ],
  "page" : {
    "size" : 20,
    "totalElements" : 30,
    "totalPages" : 2,
    "number" : 0
  }
}

```

Note how the document contains a `page` field exposing the essential pagination metadata.

Globally enabling simplified Page rendering

If you don't want to change all your existing controllers to add the mapping step to return `PagedModel` instead of `Page` you can enable the automatic translation of `PageImpl` instances into `PagedModel` by tweaking `@EnableSpringDataWebSupport` as follows:

```

@EnableSpringDataWebSupport(pageSerializationMode = VIA_DTO)
class MyConfiguration { }

```

JAVA

This will allow your controller to still return `Page` instances and they will automatically be rendered into the simplified representation:

```

@Controller
class MyController {

    private final MyRepository repository;

    // Constructor omitted

    @GetMapping("/page")
    Page<?> page(Pageable pageable) {
        return repository.findAll(pageable);
    }
}

```

JAVA

Hypermedia Support for Page and Slice

Spring HATEOAS ships with a representation model class (`PagedModel` / `SlicedModel`) that allows enriching the content of a `Page` or `Slice` instance with the necessary `Page` / `Slice` metadata as well as links to let the clients easily navigate the pages. The conversion of a `Page` to a `PagedModel` is done by

an implementation of the Spring HATEOAS `RepresentationModelAssembler` interface, called the `PagedResourcesAssembler`. Similarly `Slice` instances can be converted to a `SlicedModel` using a `SlicedResourcesAssembler`. The following example shows how to use a `PagedResourcesAssembler` as a controller method argument, as the `SlicedResourcesAssembler` works exactly the same:

Using a `PagedResourcesAssembler` as controller method argument

```
@Controller
class PersonController {

    private final PersonRepository repository;

    // Constructor omitted

    @GetMapping("/people")
    ResponseEntity<PagedModel<Person>> people(Pageable pageable,
        PagedResourcesAssembler assembler) {

        Page<Person> people = repository.findAll(pageable);
        return ResponseEntity.ok(assembler.toModel(people));
    }
}
```

JAVA

Enabling the configuration, as shown in the preceding example, lets the `PagedResourcesAssembler` be used as a controller method argument. Calling `toModel(...)` on it has the following effects:

- The content of the `Page` becomes the content of the `PagedModel` instance.
- The `PagedModel` object gets a `PageMetadata` instance attached, and it is populated with information from the `Page` and the underlying `Pageable`.
- The `PagedModel` may get `prev` and `next` links attached, depending on the page's state. The links point to the URI to which the method maps. The pagination parameters added to the method match the setup of the `PageableHandlerMethodArgumentResolver` to make sure the links can be resolved later.

Assume we have 30 `Person` instances in the database. You can now trigger a request (`GET localhost:8080/people`) and see output similar to the following:

```
{ "links" : [
  { "rel" : "next", "href" : "http://localhost:8080/persons?page=1&size=20" }
],
  "content" : [
    ... // 20 Person instances rendered here
  ],
}
```

JAVASCRIPT

```
"page" : {
  "size" : 20,
  "totalElements" : 30,
  "totalPages" : 2,
  "number" : 0
}
```

| WARNING

The JSON envelope format shown here doesn't follow any formally specified structure and it's not guaranteed stable and we might change it at any time. It's highly recommended to enable the rendering as a hypermedia-enabled, official media type, supported by Spring HATEOAS, like [HAL](#). Those can be activated by using its `@EnableHypermediaSupport` annotation. Find more information in the [Spring HATEOAS reference documentation](#).

The assembler produced the correct URI and also picked up the default configuration to resolve the parameters into a `Pageable` for an upcoming request. This means that, if you change that configuration, the links automatically adhere to the change. By default, the assembler points to the controller method it was invoked in, but you can customize that by passing a custom `Link` to be used as base to build the pagination links, which overloads the `PagedResourcesAssembler.toModel(...)` method.

Spring Data Jackson Modules

The core module, and some of the store specific ones, ship with a set of Jackson Modules for types, like `org.springframework.data.geo.Distance` and `org.springframework.data.geo.Point`, used by the Spring Data domain.

Those modules are imported once web support is enabled and `tools.jackson.databind.ObjectMapper` is available.

During initialization `SpringDataJackson3Modules`, like the `SpringDataJackson3Configuration`, get picked up by the infrastructure, so that the declared `tools.jackson.databind.JsonModule`s are made available to the Jackson `ObjectMapper`.

Data binding mixins for the following domain types are registered by the common infrastructure.

```
org.springframework.data.geo.Distance
org.springframework.data.geo.Point
org.springframework.data.geo.Box
org.springframework.data.geo.Circle
org.springframework.data.geo.Polygon
```

| NOTE

NOTE

The individual module may provide additional `SpringDataJackson3Modules`. Please refer to the store specific section for more details.

NOTE

Jackson 2 support is deprecated and will be removed in a future release.

Web Databinding Support

You can use Spring Data projections (described in [Projections](#)) to bind incoming request payloads by using either [JSONPath](#) expressions (requires [Jayway JsonPath](#)) or [XPath](#) expressions (requires [XmlBeam](#)), as the following example shows:

HTTP payload binding using JSONPath or XPath expressions

```
@ProjectedPayload
public interface UserPayload {

    @XBRead("//firstname")
    @JsonPath("$.firstname")
    String getFirstname();

    @XBRead("/lastname")
    @JsonPath({ "$.lastname", "$.user.lastname" })
    String getLastname();
}
```

JAVA

You can use the type shown in the preceding example as a Spring MVC handler method argument or by using `ParameterizedTypeReference` on one of methods of the `RestTemplate`. The preceding method declarations would try to find `firstname` anywhere in the given document. The `lastname` XML lookup is performed on the top-level of the incoming document. The JSON variant of that tries a top-level `lastname` first but also tries `lastname` nested in a `user` sub-document if the former does not return a value. That way, changes in the structure of the source document can be mitigated easily without having clients calling the exposed methods (usually a drawback of class-based payload binding).

Nested projections are supported as described in [Projections](#). If the method returns a complex, non-interface type, a Jackson `ObjectMapper` is used to map the final value.

For Spring MVC, the necessary converters are registered automatically as soon as `@EnableSpringDataWebSupport` is active and the required dependencies are available on the classpath. For usage with `RestTemplate`, register a `ProjectingJacksonHttpMessageConverter` (JSON) or `XmlBeamHttpMessageConverter` manually.

For more information, see the [web projection example](#) in the canonical [Spring Data Examples repository](#).

Querydsl Web Support

For those stores that have [Querydsl](#) integration, you can derive queries from the attributes contained in a `Request` query string.

Consider the following query string:

```
?firstname=Dave&lastname=Matthews
```

TEXT

Given the `User` object from the previous examples, you can resolve a query string to the following value by using the `QuerydslPredicateArgumentResolver`, as follows:

```
QUser.user.firstname.eq("Dave").and(QUser.user.lastname.eq("Matthews"))
```

TEXT

NOTE

The feature is automatically enabled, along with `@EnableSpringDataWebSupport`, when `Querydsl` is found on the classpath.

Adding a `@QuerydslPredicate` to the method signature provides a ready-to-use `Predicate`, which you can run by using the `QuerydslPredicateExecutor`.

TIP

Type information is typically resolved from the method's return type. Since that information does not necessarily match the domain type, it might be a good idea to use the `root` attribute of `QuerydslPredicate`.

The following example shows how to use `@QuerydslPredicate` in a method signature:

```
@Controller
class UserController {

    @Autowired UserRepository repository;

    @RequestMapping(value = "/", method = RequestMethod.GET)
    String index(Model model, @QuerydslPredicate(root = User.class) Predicate predicate,
1         Pageable pageable, @RequestParam MultiValueMap<String, String> parameters) {

        model.addAttribute("users", repository.findAll(predicate, pageable));

        return "index";
    }
}
```

JAVA


```
}  
}
```

- 1 Resolve query string arguments to matching Predicate for User.

The default binding is as follows:

- Object on simple properties as eq.
- Object on collection like properties as contains.
- Collection on simple properties as in.

You can customize those bindings through the bindings attribute of @QuerydslPredicate or by making use of Java 8 default methods and adding the QuerydslBinderCustomizer method to the repository interface, as follows:

```
interface UserRepository extends CrudRepository<User, String>, JAVA  
    QuerydslPredicateExecutor<User>, 1  
    QuerydslBinderCustomizer<QUser> { 2  
  
    @Override  
    default void customize(QuerydslBindings bindings, QUser user) {  
  
        bindings.bind(user.username).first((path, value) -> path.contains(value)) 3  
        bindings.bind(String.class)  
            .first((StringPath path, String value) -> path.containsIgnoreCase(value)); 4  
        bindings.excluding(user.password); 5  
    }  
}
```

- 1 QuerydslPredicateExecutor provides access to specific finder methods for Predicate.
- 2 QuerydslBinderCustomizer defined on the repository interface is automatically picked up and shortcuts @QuerydslPredicate(bindings=...).
- 3 Define the binding for the username property to be a simple contains binding.
- 4 Define the default binding for String properties to be a case-insensitive contains match.
- 5 Exclude the password property from Predicate resolution.

TIP

You can register a QuerydslBinderCustomizerDefaults bean holding default Querydsl bindings before applying specific bindings from the repository or @QuerydslPredicate.

Repository Populators

If you work with the Spring JDBC module, you are probably familiar with the support for populating a `DataSource` with SQL scripts. A similar abstraction is available on the repositories level, although it does not use SQL as the data definition language because it must be store-independent. Thus, the populators support XML (through Spring's OXM abstraction) and JSON (through Jackson) to define data with which to populate the repositories.

Assume you have a file called `data.json` with the following content:

Data defined in JSON

JAVASCRIPT

```
[ { "_class" : "com.acme.Person",  
  "firstname" : "Dave",  
  "lastname" : "Matthews" },  
  { "_class" : "com.acme.Person",  
    "firstname" : "Carter",  
    "lastname" : "Beauford" } ]
```

You can populate your repositories by using the populator elements of the repository namespace provided in Spring Data Commons. To populate the preceding data to your `PersonRepository`, declare a populator similar to the following:

Declaring a Jackson repository populator

XML

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xmlns:repository="http://www.springframework.org/schema/data/repository"  
  xsi:schemaLocation="http://www.springframework.org/schema/beans  
    https://www.springframework.org/schema/beans/spring-beans.xsd  
    http://www.springframework.org/schema/data/repository  
    https://www.springframework.org/schema/data/repository/spring-repository.xsd">  
  
  <repository:jackson2-populator locations="classpath:data.json" />  
  
</beans>
```

The preceding declaration causes the `data.json` file to be read and deserialized by a Jackson `ObjectMapper`.

The type to which the JSON object is unmarshalled is determined by inspecting the `_class` attribute of the JSON document. The infrastructure eventually selects the appropriate repository to handle the object that was deserialized.

To instead use XML to define the data the repositories should be populated with, you can use the `unmarshaller-populator` element. You configure it to use one of the XML marshaller options available

in Spring OXM. See the [Spring reference documentation](#) for details. The following example shows how to unmarshall a repository populator with JAXB:

Declaring an unmarshalling repository populator (using JAXB)

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:repository="http://www.springframework.org/schema/data/repository"
  xmlns:oxm="http://www.springframework.org/schema/oxm"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/data/repository
    https://www.springframework.org/schema/data/repository/spring-repository.xsd
    http://www.springframework.org/schema/oxm
    https://www.springframework.org/schema/oxm/spring-oxm.xsd">

  <repository:unmarshaller-populator locations="classpath:data.json"
    unmarshaller-ref="unmarshaller" />

  <oxm:jaxb2-marshaller contextPath="com.acme" />

</beans>
```



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Repository query keywords

Repository query keywords

Supported query method subject keywords

Reserved methods

Supported query method predicate keywords and modifiers

Supported query method subject keywords

The following table lists the subject keywords generally supported by the Spring Data repository query derivation mechanism to express the predicate. Consult the store-specific documentation for the exact list of supported keywords, because some keywords listed here might not be supported in a particular store.

Table 1. Query subject keywords

Keyword	Description
<code>find...By</code> , <code>read...By</code> , <code>get...By</code> , <code>query...By</code> , <code>search...By</code> , <code>stream...By</code>	General query method returning typically the repository type, a <code>Collection</code> or <code>Streamable</code> subtype or a result wrapper such as <code>Page</code> , <code>GeoResults</code> or any other store-specific result wrapper. Can be used as <code>findBy...</code> , <code>findMyDomainTypeBy...</code> or in combination with additional keywords.
<code>exists...By</code>	Exists projection, returning typically a <code>boolean</code> result.
<code>count...By</code>	Count projection returning a numeric result.
<code>delete...By</code> , <code>remove...By</code>	Delete query method returning either no result (<code>void</code>) or the delete count.
<code>...First<number>...</code> , <code>...Top<number>...</code>	Limit the query results to the first <code><number></code> of results. This keyword can occur in any place of the subject between <code>find</code> (and the other keywords) and <code>by</code> .
<code>...Distinct...</code>	Use a distinct query to return only unique results. Consult the store-specific documentation whether that feature is supported. This keyword can occur in any place of the subject between <code>find</code> (and the other keywords) and <code>by</code> .

Reserved methods

The following table lists reserved methods that use predefined functionality (as defined in `CrudRepository`). These methods are directly invoked on the backing (store-specific) implementation of the repository proxy. See also [“Defining Query Methods”](#).

Table 2. Reserved methods

<code>deleteAllById(Iterable<ID> identifiers)</code>
<code>deleteById(ID identifier)</code>
<code>existsById(ID identifier)</code>
<code>findAllById(Iterable<ID> identifiers)</code>
<code>findById(ID identifier)</code>

Supported query method predicate keywords and modifiers

The following table lists the predicate keywords generally supported by the Spring Data repository query derivation mechanism. However, consult the store-specific documentation for the exact list of supported keywords, because some keywords listed here might not be supported in a particular store.

Table 3. Query predicate keywords

Logical keyword	Keyword expressions
AND	And
OR	Or
AFTER	After, IsAfter
BEFORE	Before, IsBefore
CONTAINING	Containing, IsContaining, Contains
BETWEEN	Between, IsBetween
ENDING_WITH	EndingWith, IsEndingWith, EndsWith
EXISTS	Exists
FALSE	False, IsFalse
GREATER_THAN	GreaterThan, IsGreaterThan
GREATER_THAN_EQUALS	GreaterThanEqual, IsGreaterThanEqual

Logical keyword	Keyword expressions
IN	In , IsIn
IS	Is , Equals , (or no keyword)
IS_EMPTY	IsEmpty , Empty
IS_NOT_EMPTY	IsNotEmpty , NotEmpty
IS_NOT_NULL	NotNull , IsNotNull
IS_NULL	Null , IsNull
LESS_THAN	LessThan , IsLessThan
LESS_THAN_EQUAL	LessThanEqual , IsLessThanEqual
LIKE	Like , IsLike
NEAR	Near , IsNear
NOT	Not , IsNot
NOT_IN	NotIn , IsNotIn
NOT_LIKE	NotLike , IsNotLike
REGEX	Regex , MatchesRegex , Matches
STARTING_WITH	StartingWith , IsStartingWith , StartsWith
TRUE	True , IsTrue
WITHIN	Within , IsWithin

In addition to filter predicates, the following list of modifiers is supported:

Table 4. Query predicate modifier keywords

Keyword	Description
IgnoreCase , IgnoringCase	Used with a predicate keyword for case-insensitive comparison.
AllIgnoreCase , AllIgnoringCase	Ignore case for all suitable properties. Used somewhere in the query method predicate.

Keyword	Description
OrderBy...	Specify a static sorting order followed by the property path and direction (e.g. OrderByFirstnameAscLastnameDesc).



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Repository query return types

Repository query return types

Supported Query Return Types

Supported Query Return Types

The following table lists the return types generally supported by Spring Data repositories. However, consult the store-specific documentation for the exact list of supported return types, because some types listed here might not be supported in a particular store.

NOTE

Geospatial types (such as `GeoResult`, `GeoResults`, and `GeoPage`) are available only for data stores that support geospatial queries. Some store modules may define their own result wrapper types.

Table 1. Query return types

Return type	Description
<code>void</code>	Denotes no return value.
Primitives	Java primitives.
Wrapper types	Java wrapper types.
<code>T</code>	A unique entity. Expects the query method to return one result at most. If no result is found, <code>null</code> is returned. More than one result triggers an <code>IncorrectResultSizeDataAccessException</code> .
<code>Iterator<T></code>	An <code>Iterator</code> .
<code>Collection<T></code>	A <code>Collection</code> .
<code>List<T></code>	A <code>List</code> .
<code>Optional<T></code>	A Java 8 or Guava <code>Optional</code> . Expects the query method to return one result at most. If no result is found, <code>Optional.empty()</code> or <code>Optional.absent()</code> is returned. More than one result triggers an <code>IncorrectResultSizeDataAccessException</code> .

Return type	Description
<code>Option<T></code>	Either a Scala or Vavr <code>Option</code> type. Semantically the same behavior as Java 8's <code>Optional</code> , described earlier.
<code>Stream<T></code>	A Java 8 <code>Stream</code> .
<code>Streamable<T></code>	A convenience extension of <code>Iterable</code> that directly exposes methods to stream, map and filter results, concatenate them etc.
Types that implement <code>Streamable</code> and take a <code>Streamable</code> constructor or factory method argument	Types that expose a constructor or <code>...of(...)</code> / <code>...valueOf(...)</code> factory method taking a <code>Streamable</code> as argument. See Returning Custom Streamable Wrapper Types for details.
Vavr <code>Seq</code> , <code>List</code> , <code>Map</code> , <code>Set</code>	Vavr collection types. See Support for Vavr Collections for details.
<code>Future<T></code>	A <code>Future</code> . Expects a method to be annotated with <code>@Async</code> and requires Spring's asynchronous method execution capability to be enabled.
<code>CompletableFuture<T></code>	A Java 8 <code>CompletableFuture</code> . Expects a method to be annotated with <code>@Async</code> and requires Spring's asynchronous method execution capability to be enabled.
<code>Slice<T></code>	A sized chunk of data with an indication of whether there is more data available. Requires a <code>Pageable</code> method parameter.
<code>Page<T></code>	A <code>Slice</code> with additional information, such as the total number of results. Requires a <code>Pageable</code> method parameter.
<code>Window<T></code>	A <code>Window</code> of results obtained from a scroll query. Provides <code>ScrollPosition</code> to issue the next scroll query. Requires a <code>ScrollPosition</code> method parameter.
<code>GeoResult<T></code>	A result entry with additional information, such as the distance to a reference location.
<code>GeoResults<T></code>	A list of <code>GeoResult<T></code> with additional information, such as the average distance to a reference location.
<code>GeoPage<T></code>	A <code>Page</code> with <code>GeoResult<T></code> , such as the average distance to a reference location.
<code>SearchResult<T></code>	A result entry with additional information, such as the score in relation to the reference search term.
<code>SearchResults<T></code>	A list of <code>SearchResult<T></code> .
<code>Mono<T></code>	A Project Reactor <code>Mono</code> emitting zero or one element using reactive repositories. Expects the query method to return one result at most. If no result is found, <code>Mono.empty()</code> is returned. More than one result triggers an <code>IncorrectResultSizeDataAccessException</code> .

Return type	Description
<code>Flux<T></code>	A Project Reactor <code>Flux</code> emitting zero, one, or many elements using reactive repositories. Queries returning <code>Flux</code> can emit also an infinite number of elements.
<code>Single<T></code>	A RxJava <code>Single</code> emitting a single element using reactive repositories. Expects the query method to return one result at most. If no result is found, <code>Mono.empty()</code> is returned. More than one result triggers an <code>IncorrectResultSizeDataAccessException</code> .
<code>Maybe<T></code>	A RxJava <code>Maybe</code> emitting zero or one element using reactive repositories. Expects the query method to return one result at most. If no result is found, <code>Mono.empty()</code> is returned. More than one result triggers an <code>IncorrectResultSizeDataAccessException</code> .
<code>Flowable<T></code>	A RxJava <code>Flowable</code> emitting zero, one, or many elements using reactive repositories. Queries returning <code>Flowable</code> can emit also an infinite number of elements.



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Ahead of Time Optimizations

Ahead of Time Optimizations

Best Practices

- Annotate your Domain Types

- Ahead of Time Code Generation

- Ahead of Time Repositories

- Repository Metadata

- Native Image Runtime Hints

- JPA Ahead of Time Repositories

This chapter covers Spring Data's Ahead of Time (AOT) optimizations that build upon [Spring's Ahead of Time Optimizations](#).

Best Practices

Annotate your Domain Types

During application startup, Spring scans the classpath for domain classes for early processing of entities. By annotating your domain types with Spring Data Store specific `@Table`, `@Document` or `@Entity` annotations you can aid initial entity scanning and ensure that those types are registered with `ManagedTypes` for Runtime Hints. Classpath scanning is not possible in native image arrangements and so Spring has to use `ManagedTypes` for the initial entity set.

Ahead of Time Code Generation

Ahead of time code generation is not limited to usage with GraalVM Native Image but also offers benefits when working with regular deployments and can help optimize startup performance on the jvm.

NOTE

With AOT optimizations some decisions (like database dialects for example) will be frozen at build time and get included as is in the application setup.

If Ahead of Time compilation is enabled Spring Data can (depending on the actual Module in use) contribute several components during the AOT phase of your build.

- Bytecode for generated Type/Property Accessors
- Sourcecode for the defined Repository Interfaces
- Repository Metadata in JSON format

Each of the above is enabled by default. However, there users may fine tune the configuration with following options.

<code>spring.aot.data.accessors.enabled</code>	Boolean flag to control contribution of Bytecode for generated Type/Property Accessors
<code>spring.aot.data.accessors.include</code>	Comma separated list of FQCN for which to contribute Bytecode for generated Type/Property Accessors. Ant-style include patterns matching package names (e.g. <code>example.springdata.**</code>) or type names inclusion. If a type is matched by an inclusion and an exclusion, the inclusion wins and the type is considered included.
<code>spring.aot.data.accessors.exclude</code>	Comma separated list of FQCN for which to skip contribution of Bytecode for generated Type/Property Accessors. Ant-style exclude patterns matching package names (e.g. <code>example.springdata.**</code>) or type names exclusion. If a type is matched by an inclusion and an exclusion, the inclusion wins and the type is considered included.
<code>spring.aot.repositories.enabled</code>	Boolean flag to control contribution of Source Code for Repository Interfaces
<code>spring.aot.[module-name].repositories.enabled</code>	Boolean flag to control contribution of Source Code for Repository Interfaces for a certain module (e.g. <code>cassandra</code> , <code>jdbc</code> , <code>jpa</code> , <code>mongodb</code>)
<code>spring.aot.repositories.metadata.enabled</code>	Boolean flag to control contribution of JSON repository metadata containing query methods and actual query strings. Requires <code>spring.aot.repositories.enabled</code> to be enabled.

Ahead of Time Repositories

NOTE

Ahead of Time repositories are only available for imperative (non reactive) repository interfaces of certain modules. The criteria identifying eligible query methods varies between the implementing modules.

AOT Repositories are an extension to AOT processing by pre-generating eligible query method implementations. Query methods are opaque to developers regarding their underlying queries being exe-

cuted in a query method call. AOT repositories contribute query method implementations based on derived, annotated, and named queries that are known at build-time. This optimization moves query method processing from runtime to build-time, which can lead to a significant performance improvement as query methods do not need to be analyzed reflectively upon each application start.

The resulting AOT repository fragment follows the naming scheme of `<Repository FQCN>Impl__AotRepository` and is placed in the same package as the repository interface.

| WARNING

Consider AOT repository classes an internal optimization. Do not use them directly in your code as generation and implementation details may change in future releases.

Repository Metadata

AOT processing introspects query methods and collects metadata about repository queries. Spring Data stores this metadata in JSON files that are named after the source repository within the same package. Repository JSON Metadata contains details about queries and fragments. An example for the following repository is shown below:

Metadata	Repository
----------	------------

```
{
  "name": "example.springdata.UserRepository",
  "module": "JDBC",
  "type": "IMPERATIVE",
  "methods": [
    {
      "name": "findBy",
      "signature": "public abstract java.util.List<example.springdata.User>
example.springdata.UserRepository.findBy()",
      "query": {
        "query": "SELECT * FROM User"
      }
    },
    {
      "name": "findByLastnameStartingWith",
      "signature": "public abstract
org.springframework.data.domain.Page<example.springdata.User>
example.springdata.UserRepository.findByLastnameStartingWith(java.lang.String,org.springframework.data.domain.Pageable)",
      "query": {
        "query": "SELECT * FROM User u WHERE lastname LIKE :lastname",
        "count-query": "SELECT COUNT(*) FROM User WHERE lastname LIKE :lastname"
      }
    }
  ]
}
```

JSON

```
{
  "name": "findByEmailAddress",
  "signature": "public abstract example.springdata.User
example.springdata.UserRepository.findByEmailAddress(java.lang.String)",
  "query": {
    "query": "select * from User where emailAddress = ?1"
  }
},
```

| NOTE

Creating JSON metadata can be controlled via the `spring.aot.repositories.metadata.enabled` flag.

Native Image Runtime Hints

Running an application as a native image requires additional information compared to a regular JVM runtime. Spring Data contributes [Runtime Hints](#) during AOT processing for native image usage. These are in particular hints for:

- Auditing
- `ManagedTypes` to capture the outcome of class-path scans
- Repositories
 - Reflection hints for entities, return types, and Spring Data annotations
 - Repository fragments
 - Querydsl Q classes
 - Kotlin Coroutine support
- Web support (Jackson Hints for `PagedModel`)

JPA Ahead of Time Repositories

AOT repositories filter methods that are eligible for AOT processing. These are typically all query methods that are not backed by an [implementation fragment](#).

| NOTE

AOT processing avoids database access. Therefore, it initializes an in-memory Hibernate instance for metadata collection. Types for the Hibernate configuration are determined by our AOT metadata collector. We prefer using a `PersistentEntityTypes` bean if available and fall back to `PersistenceUnitInfo` or our own discovered types. If our type scanning is not sufficient for your arrangement, you can enable direct `EntityManagerFactory` usage by configuring the `spring.aot.jpa.repositories.use-entitymanager=true` property.

Supported Features

- Derived query methods, `@Query` / `@NativeQuery` and named query methods
- Stored procedure query methods annotated with `@Procedure`
- `@Modifying` methods returning `void` or `int`
- `@QueryHints` support
- `Pagination`, `Slice`, `Stream`, and `Optional` return types
- Sort query rewriting
- Interface and DTO Projections
- Value Expressions (Those require a bit of reflective information. Mind that using Value Expressions requires expression parsing and contextual information to evaluate the expression)

Limitations

- Requires Hibernate for AOT processing.
- `QueryRewriter` must be a no-args class. `QueryRewriter` beans are not yet supported.
- Methods accepting `ScrollPosition` (e.g. `Keyset` pagination) are not yet supported.
- Custom Collection return types (e.g. `io.vavr.collection`, `"org.eclipse.collections"`) are not yet supported.

Excluded methods

- `CrudRepository`, `Querydsl`, `Query by Example`, and other base interface methods as their implementation is provided by the base class respective fragments
- Methods whose implementation would be overly complex
 - Methods accepting `ScrollPosition` (e.g. `Keyset` pagination)
 - Dynamic projections



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle

and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Frequently Asked Questions

Frequently Asked Questions

Common

Auditing

Common

1. *I'd like to get more detailed logging information on what methods are called inside `JpaRepository` for example. How can I gain them?*

You can make use of `CustomizableTraceInterceptor` provided by Spring, as shown in the following example:

```
<bean id="customizableTraceInterceptor" class="
    org.springframework.aop.interceptor.CustomizableTraceInterceptor">
    <property name="enterMessage" value="Entering ${methodName}(${arguments})"/>
    <property name="exitMessage" value="Leaving ${methodName}(): ${returnValue}"/>
</bean>

<aop:config>
    <aop:advisor advice-ref="customizableTraceInterceptor"
        pointcut="execution(public *
org.springframework.data.jpa.repository.JpaRepository+.*(..))"/>
</aop:config>
```

XML

Auditing

1. *I want to use Spring Data JPA auditing capabilities but have my database already configured to set modification and creation date on entities. How can I prevent Spring Data from setting the date programmatically?*

Set the `set-dates` attribute of the `auditing` namespace element to `false`.

Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Glossary

AOP

Aspect oriented programming

Commons DBCP

Commons DataBase Connection Pools - a library from the Apache foundation that offers pooling implementations of the DataSource interface.

CRUD

Create, Read, Update, Delete - Basic persistence operations.

DAO

Data Access Object - Pattern to separate persisting logic from the object to be persisted

Dependency Injection

Pattern to hand a component's dependency to the component from outside, freeing the component to lookup the dependent itself. For more information, see en.wikipedia.org/wiki/Dependency_Injection.

EclipseLink

Object relational mapper implementing JPA - www.eclipse.org/eclipselink/

Hibernate

Object relational mapper implementing JPA - hibernate.org/

JPA

Jakarta Persistence API

Spring

Java application framework - spring.io/projects/spring-framework

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Envers

This chapter points out the specialties for repository support for Envers. This builds on the core repository support explained earlier. Make sure you have a sound understanding of the basic concepts explained there.

Section Summary

- [Introduction](#)
- [Configuration](#)
- [Usage](#)

Introduction

Introduction

What is Spring Data Envers?

What is Envers?

What is Spring Data Envers?

Spring Data Envers makes typical Envers queries available in repositories for Spring Data JPA. It differs from other Spring Data modules in that it is always used in combination with another Spring Data Module: Spring Data JPA.

What is Envers?

Envers is a [Hibernate module](#) that adds auditing capabilities to JPA entities. This documentation assumes you are familiar with Envers, just as Spring Data Envers relies on Envers being properly configured.

Configuration

As a starting point for using Spring Data Envers, you need a project with Spring Data JPA on the class-path and an additional `spring-data-envers` dependency:

```
<dependencies>

<!-- other dependency elements omitted -->

<dependency>
  <groupId>org.springframework.data</groupId>
  <artifactId>spring-data-envers</artifactId>
  <version>4.0.2</version>
</dependency>

</dependencies>
```

XML

This also brings `hibernate-envers` into the project as a transient dependency.

To enable Spring Data Envers and Spring Data JPA, we need to configure two beans and a special `repositoryFactoryBeanClass`:

```
@Configuration
@EnableEnversRepositories
@EnableTransactionManagement
public class EnversDemoConfiguration {

    @Bean
    public DataSource dataSource() {

        EmbeddedDatabaseBuilder builder = new EmbeddedDatabaseBuilder();
        return builder.setType(EmbeddedDatabaseType.HSQL).build();
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory() {
```

JAVA

```

        HibernateJpaVendorAdapter vendorAdapter = new HibernateJpaVendorAdapter();
        vendorAdapter.setGenerateDdl(true);

        LocalContainerEntityManagerFactoryBean factory = new
LocalContainerEntityManagerFactoryBean();
        factory.setJpaVendorAdapter(vendorAdapter);
        factory.setPackagesToScan("example.springdata.jpa.envers");
        factory.setDataSource(dataSource());
        return factory;
    }

    @Bean
    public PlatformTransactionManager transactionManager(EntityManagerFactory
entityManagerFactory) {

        JpaTransactionManager txManager = new JpaTransactionManager();
        txManager.setEntityManagerFactory(entityManagerFactory);
        return txManager;
    }
}

```

To actually use Spring Data Envers, make one or more repositories into a [RevisionRepository](#) by adding it as an extended interface:

```

interface PersonRepository
    extends CrudRepository<Person, Long>,
        RevisionRepository<Person, Long, Long> 1
{
}

```

JAVA

- ¹ The first type parameter (Person) denotes the entity type, the second (Long) denotes the type of the id property, and the last one (Long) is the type of the revision number. For Envers in default configuration, the revision number parameter should be Integer or Long .

The entity for that repository must be an entity with Envers auditing enabled (that is, it must have an @Audited annotation):

```

@Entity
@Audited
class Person {

    @Id @GeneratedValue
    Long id;
}

```

JAVA


```
String name;  
@Version Long version;  
}
```



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. “AWS” and “Amazon Web Services” are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Usage

Usage

Further Resources

You can now use the methods from `RevisionRepository` to query the revisions of the entity, as the following test case shows:

```
@ExtendWith(SpringExtension.class)
@Import(EnversDemoConfiguration.class) 1
class EnversIntegrationTests {

    final PersonRepository repository;
    final TransactionTemplate tx;

    EnversIntegrationTests(@Autowired PersonRepository repository, @Autowired
PlatformTransactionManager tm) {
        this.repository = repository;
        this.tx = new TransactionTemplate(tm);
    }

    @Test
    void testRepository() {

        Person updated = preparePersonHistory();

        Revisions<Long, Person> revisions = repository.findRevisions(updated.id);

        Iterator<Revision<Long, Person>> revisionIterator = revisions.iterator();

        checkNextRevision(revisionIterator, "John", RevisionType.INSERT);
        checkNextRevision(revisionIterator, "Jonny", RevisionType.UPDATE);
        checkNextRevision(revisionIterator, null, RevisionType.DELETE);
        assertThat(revisionIterator.hasNext()).isFalse();

    }

    /**
```

JAVA

```

    * Checks that the next element in the iterator is a Revision entry referencing a
    Person
    * with the given name after whatever change brought that Revision into existence.
    * <p>
    * As a side effect the Iterator gets advanced by one element.
    *
    * @param revisionIterator the iterator to be tested.
    * @param name the expected name of the Person referenced by the Revision.
    * @param revisionType the type of the revision denoting if it represents an insert,
    update or delete.
    */
    private void checkNextRevision(Iterator<Revision<Long, Person>> revisionIterator,
    String name,
        RevisionType revisionType) {

        assertThat(revisionIterator.hasNext()).isTrue();
        Revision<Long, Person> revision = revisionIterator.next();
        assertThat(revision.getEntity().name).isEqualTo(name);
        assertThat(revision.getMetadata().getRevisionType()).isEqualTo(revisionType);
    }

    /**
    * Creates a Person with a couple of changes so it has a non-trivial revision
    history.
    * @return the created Person.
    */
    private Person preparePersonHistory() {

        Person john = new Person();
        john.setName("John");

        // create
        Person saved = tx.execute(__ -> repository.save(john));
        assertThat(saved).isNotNull();

        saved.setName("Jonny");

        // update
        Person updated = tx.execute(__ -> repository.save(saved));
        assertThat(updated).isNotNull();

        // delete
        tx.executeWithoutResult(__ -> repository.delete(updated));
        return updated;
    }
}

```

¹ This references the application context configuration presented earlier (in the [Configuration](#) section).

Further Resources

You can download the [Spring Data Envers example](#) in the [Spring Data Examples repository](#) and play around with to get a feel for how the library works.

You should also check out the [Javadoc](#) for [RevisionRepository](#) and related classes.

You can ask questions at [Stackoverflow](#) by using the [spring-data-envers](#) tag.

The [source code](#) and [issue tracker](#) for Spring Data Envers is hosted at [GitHub](#) (as a module of Spring Data JPA).



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#) • [Thank you](#) • [Your California Privacy Rights](#) • [Cookies Settings](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

Spring Data JPA Parent 4.0.2 API

Packages

Package	Description
<code>org.springframework.data.envers.repository.config</code>	Classes for Envers Repositories configuration support.
<code>org.springframework.data.envers.repository.support</code>	Spring Data JPA specific converter infrastructure.
<code>org.springframework.data.jpa.convert</code>	Spring Data JPA specific converter infrastructure.
<code>org.springframework.data.jpa.convert.threeten</code>	Spring Data JPA specific JSR-310 converters.
<code>org.springframework.data.jpa.domain</code>	JPA specific support classes to implement domain classes.
<code>org.springframework.data.jpa.domain.support</code>	Implementation classes for auditing with JPA.
<code>org.springframework.data.jpa.mapping</code>	JPA specific support classes for the Spring Data mapping subsystem.
<code>org.springframework.data.jpa.projection</code>	JPA specific support projection support.
<code>org.springframework.data.jpa.provider</code>	JPA provider-specific utilities.
<code>org.springframework.data.jpa.repository</code>	Interfaces and annotations for JPA specific repositories.
<code>org.springframework.data.jpa.repository.aot</code>	Ahead-of-Time (AOT) generation for Spring Data JPA repositories.
<code>org.springframework.data.jpa.repository.cdi</code>	CDI support for Spring Data JPA Repositories.
<code>org.springframework.data.jpa.repository.config</code>	Classes for JPA namespace configuration.
<code>org.springframework.data.jpa.repository.query</code>	Query implementation to execute queries against JPA.
<code>org.springframework.data.jpa.repository.support</code>	JPA repository implementations.
<code>org.springframework.data.jpa.support</code>	Various helper classes useful when working with JPA.
<code>org.springframework.data.jpa.util</code>	Spring Data JPA utilities.

